

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

INHERITED FROM constitution/CLAUDE.md

All rules in constitution/CLAUDE.md (and the constitution/Constitution.md it references) apply unconditionally. Lava-specific rules below extend them — they **MUST NOT** weaken any inherited rule. Where Lava already has equivalent or stricter mechanics (Anti-Bluff Pact §6.A–§6.AC, Local-Only CI/CD, §6.W mirror policy, host-stability §6.M, no-hardcoding §6.R, CONTINUATION mandate §6.S, etc.), the existing Lava rule wins **only when it is at least as strict** as the Helix universal rule. The HelixConstitution submodule was incorporated 2026-05-14 (29th §6.L cycle). Implementation gaps (project-side gates not yet wired: CM-COMMIT-DOCS-EXISTS, CM-FIXED-COLUMN-ALIGNMENT, CM-SCRIPT-DOCS-SYNC, CM-BUILD-RESOURCE-STATS-TRACKER, etc.) are tracked under §6.AD-debt below; until they close, the inherited rules are operator-and-reviewer-verified manually.

Claude Code's @path/to/file import syntax also resolves: @constitution/CLAUDE.md at the top of this document is equivalent to the pointer block above.

See also (read in this order on a cold start):

- **constitution/CLAUDE.md + constitution/Constitution.md — universal Helix rules.** Inherited unconditionally per the block above.
- **docs/CONTINUATION.md — READ FIRST.** Single-file source-of-truth handoff per §6.S. Tells you the active phase, current pin index, latest release, open known issues. A fresh session resumes from here in under five minutes.
- **CHANGELOG.md** — per-release distribution log per §6.P; per-version snapshots live under .lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md.
- **AGENTS.md** — longer companion guide (tech stack versions, deployment, security notes). Read this when CLAUDE.md is too brief on a given subject.
- **core/CLAUDE.md, app/CLAUDE.md, feature/CLAUDE.md** — scoped Anti-Bluff rules that apply only inside those trees.
- **lava-api-go/CLAUDE.md and lava-api-go/CONSTITUTION.md** — scoped instructions and constitutional addenda for the Go API service. **Read both before touching lava-api-go/.**
- **submodules/<Name>/CLAUDE.md** — each of the 16 vasic-digital/* submodules + 1 HelixDevelopment-owned (HelixQA, adopted Phase 4; Phase 4-debt closed 2026-05-16 — HelixQA upstream commit b13ba7c ships helix-deps.yaml + install_upstreams.sh at parity with the 16 vasic-digital submodules) ships its own scoped rules, inherited per

6.F. All 17 own-org submodules now satisfy CM-HELIX-DEPS-MANIFEST + CM-CANONICAL-ROOT-CLARITY + CM-INSTALL-UPSTREAMS-RAN in fully STRICT mode (0 waived). Honour them before editing any code under submodules/.

- docs/ARCHITECTURE.md, docs/LOCAL_NETWORK_DISCOVERY.md — architecture diagrams and the mDNS discovery flow.
- docs/CODEGRAPH.md — the **codegraph** code-intelligence integration (install, per-agent MCP wiring for all 5 CLI agents, the anti-bluff verification suite). codegraph is mandatory per the inherited HelixConstitution codegraph mandate.
- docs/superpowers/specs/2026-04-28-sp2-go-api-migration-design.md — full SP-2 design doc (Go API service migration).
- docs/superpowers/plans/2026-04-28-sp2-go-api-migration.md — SP-2 implementation plan (14 phases, 39 tasks).

Project

Lava is an unofficial Android client for **multiple Russian torrent trackers** (RuTracker, RuTor, plus Internet Archive and additional providers wired through the Tracker SDK), plus a companion **Ktor proxy server** that scrapes upstream sites and exposes a JSON API to the app. Two artifacts share one Gradle build:

- :app — Android app, Kotlin + Jetpack Compose, App ID `digital.vasic.lava.client`.
- :proxy — Ktor/Netty headless server, packaged as a fat JAR + Docker image.

The Tracker SDK lives in `core/tracker/*` (api + per-tracker plugin modules + the registry in `:core:tracker:client`); see `README.md` for the per-tracker capability matrix and `docs/sdk-developer-guide.md` for the seven-step recipe to add a new one. The repo is a fork of `andrikeev/Flow`, rebranded to Lava. All code/comments/docs are English.

Commands

```
# Android app
./gradlew :app:assembleDebug           # debug APK
      (applicationIdSuffix .dev)
./gradlew :app:assembleRelease         # release APK (signed via
      keystores/)

# Proxy server
./gradlew :proxy:buildFatJar            # → proxy/build/libs/app.jar
./build_and_push_docker_image.sh       # builds + pushes to
      DigitalOcean registry

# Build all artifacts and copy to releases/
./build_and_release.sh                 # → releases/{version}/
      android-{debug|release}/, releases/{version}/proxy/

# Run the proxy locally (with LAN mDNS discovery)
```

```

./start.sh
    # builds JAR + image, starts container, advertises
    _lava._tcp
./stop.sh
    # stops + removes the
    container
./tools/lava-containers/bin/lava-containers -cmd=status #
    runtime, health, advertised IPs

# Code style (Spotless + ktlint – the only enforced checker)
./gradlew spotlessApply # run before committing
./gradlew spotlessCheck

# Tests (coverage is minimal; one unit test exists today)
./gradlew test
./gradlew :core:preferences:test --tests
    "lava.securestorage.EndpointConverterTest"

# Single Compose UI Challenge Test – Sixth Law clause 4 acceptance
    gate
# (requires a connected Android device or running emulator)
./gradlew :app:connectedDebugAndroidTest \
    --tests
    "lava.app.challenges.Challenge01AppLaunchAndTrackerSelectionTest"

# Local CI gate – IS the project's CI/CD apparatus (Local-Only CI/
    CD rule)
./scripts/ci.sh --changed-only
    # pre-push subset: Spotless, changed-module unit tests,
    # constitution parser,
    forbidden-files check
./scripts/ci.sh --full # all gates: every module,
    parity, mutation tests,
    # fixture freshness, real-
    device Challenge Tests
    # (used by scripts/tag.sh
    at tag time)

# Constitution + bluff enforcement
./scripts/check-constitution.sh # parser/validator; greps
    tracked files for forbidden
    # commands and credential
    patterns; run by pre-push
./scripts/bluff-hunt.sh # phase-gate bluff hunt
    (Seventh Law clause 5)
./scripts/check-fixture-freshness.sh # detects stale network
    fixtures
./scripts/scan-no-hardcoded-uuid.sh # $6.R UUID-literal scan
    (active enforcement)
./scripts/scan-no-hardcoded-ipv4.sh # $6.R IPv4-literal scan
    (active 2026-05-13)
./scripts/scan-no-hardcoded-hostport.sh # $6.R host:port-literal
    scan (active 2026-05-13)

# Real-device Challenge Tests (Sixth Law clause 4 acceptance gate)
./scripts/run-emulator-tests.sh
    # Android emulator container + connectedAndroidTest

```

```

# Distribution – §6.P enforces strictly increasing version code +
# mandatory CHANGELOG entry
./scripts/firebase-distribute.sh      # uploads to Firebase App
# Distribution; refuses if
#                                     #   versionCode ≤ last
#                                     #   published or CHANGELOG.md
#                                     #   lacks an entry for
#                                     #   current versionName(versionCode)

# Release tagging – refuses without local CI evidence + real-
# device attestation
./scripts/tag.sh <tag>

# Mirror sync (2 upstreams: GitHub, GitLab – per §6.W)
./Upstreams/GitHub.sh
./Upstreams/GitLab.sh
./scripts/sync-tracker-sdk-mirrors.sh

# Code intelligence – codegraph (semantic code graph over MCP; see
# docs/CODEGRAPH.md)
codegraph index                      # rebuild the .codegraph/
# semantic knowledge graph
codegraph sync                      # incremental index update
# after edits
codegraph status                    # index statistics (files /
# nodes / edges)
scripts/verify-codegraph.sh         # anti-bluff verification
# suite – 6 layers, all 5 agents
scripts/verify-codegraph.sh --quick # deterministic layers only
# (skip the slow LLM layer 05)

```

./start.sh delegates to the Lava-domain CLI at tools/lava-containers/, which auto-detects Podman or Docker, builds the proxy fat JAR + image, and runs it via docker-compose.yml. Generic container-runtime concerns are owned by the upstream vasic-digital/Containers submodule mounted at submodules/containers/ (pinned hash); the local CLI is thin glue per the Decoupled Reusable Architecture constitutional rule. The compose file uses **network_mode: host** so JmDNS broadcasts reach the LAN — the Android debug build then auto-discovers the proxy via NsdManager (lava._tcp.local.).

Signing requires a .env at the repo root with KEYSTORE_PASSWORD and KEYSTORE_ROOT_DIR (see .env.example); keystores live in keystores/ (gitignored). The app build also expects app/google-services.json (gitignored) for Firebase.

Architecture

Module layout

There is **no root build.gradle.kts** — all build logic lives in buildSrc/ as convention plugins (lava.android.application, lava.android.library, lava.android.library.compose, lava.android.feature,

lava.android.hilt, lava.kotlin.library, lava.kotlin.serialization, lava.kotlin.ksp, lava.ktor.application). Module build.gradle.kts files apply these by ID — **do not duplicate config in module scripts; extend the convention plugin instead.**

Three top-level Gradle groupings (all listed in settings.gradle.kts):

- core/* — 17 modules. Several use the **api/impl split** (auth, network, work) so consumers depend on :api only. Pure-Kotlin modules with no Android dep: core:models, core:common, core:auth:api, core:network:api, core:network:rutracker, core:work:api.
- feature/* — 15 screen-level modules; each applies lava.android.feature (which auto-wires Hilt, Orbit, and the standard core deps).
- :app — Compose entry point, depends on every core + feature module. Holds MainActivity, TvActivity, top-level navigation graph, and rutracker.org deep-link handling.

Dependency direction: app → feature:* → core:domain → core:data → core:network:impl + core:database, plus core:auth:impl, core:work:impl, core:navigation, core:ui, core:designsystem.

Two non-Gradle artifacts also live at the repo root and are NOT built by ./gradlew:

- lava-api-go/ — Go service replacing the Ktor :proxy in SP-2. Independent build (Makefile, go.mod), independent constitution (lava-api-go/CONSTITUTION.md), real-Postgres integration tests gated by -Pintegration=true running under podman/docker. Inherits 6.D and 6.E for its rutracker bridge work.
- submodules/ — 16 pinned vasic-digital/* Git submodules (Auth, Cache, Challenges, Concurrency, Config, Containers, Database, Discovery, HTTP3, Mdns, Middleware, Observability, RateLimiter, Recovery, Security, Tracker-SDK). Every component with a non-Lava-specific use case lives here per the Decoupled Reusable Architecture rule. Pins are frozen by default — never auto-update.

Patterns to follow

- **MVI via Orbit** in every feature: XxxViewModel (@HiltViewModel, ContainerHost<State, SideEffect>) + sealed XxxState / XxxAction / XxxSideEffect. Mirror the shape of neighboring features.
- **100% Jetpack Compose** — no XML layouts, no Fragments. Theme + components live in core:designsystem (LavaTheme).
- **Custom navigation DSL** in core:navigation, built on Navigation-Compose. Several modules enable -Xcontext-receivers; the context(NavigationGraphBuilder) calls rely on it. Each feature exposes addXxx() / openXxx() extensions.
- **DI:** Dagger Hilt on Android, **Koin** on the proxy server.
- **Database:** Room with KSP. Schemas are checked into core/database/schemas/.
- **Serialization:** kotlinx-serialization-json — apply lava.kotlin.serialization rather than configuring it ad hoc.
- **Versions / deps:** add to gradle/libs.versions.toml and reference via the version catalog (libs.findLibrary(...)) in convention plugins, libs.xxx in module scripts). Don't hard-code versions.

- **TV support:** `TvActivity` extends `MainActivity` and flips `PlatformType` to `TV`; manifest declares `android.software.leanback` and `android.hardware.touchscreen` as not required.
- **Local network discovery:** `core:data` exposes `LocalNetworkDiscoveryService` (`AndroidNsdManager`, service type `_lava._tcp.local.`) consumed by `DiscoverLocalEndpointsUseCase` in `core:domain` and triggered from `feature:menu`. See docs/`LOCAL_NETWORK_DISCOVERY.md` for the full flow.

Release build quirks

`isMinifyEnabled = true` but `isObfuscate = false`. ProGuard rules in `app/proguard-rules.pro` keep `lava.network.dto.**` and `Tink` classes. `android:usesCleartextTraffic="true"` is set in the manifest — be deliberate if changing it.

Anti-Bluff Testing Pact (Constitutional Law)

This project adheres to **Anti-Bluff Testing**. A "bluff test" is one that passes while the corresponding real feature is broken for end users. Bluff tests are worse than no tests because they create a false sense of security. They are forbidden.

First Law — Tests Must Guarantee Real User-Visible Behavior

- Every test **MUST** verify an outcome that matters to end users (data correctness, UI state, persistence, network behavior).
- A test that only asserts "function did not crash" or "mock was called" is a bluff test.
- If a bug ships to production, there **MUST** be a retroactive test that would have caught it before the fix is merged.

Second Law — No Mocking of Internal Business Logic

- **ViewModel tests MUST use real UseCase implementations**, not stubbed or mocked use cases.
- **UseCase tests MUST use real Repository implementations** (or fakes that enforce identical constraints to the real database / network layer).
- Mocking is permitted **ONLY** at the outermost boundaries (Android system services, actual HTTP sockets, hardware sensors).
- If a bug exists in a UseCase, a ViewModel test wired to the real UseCase **MUST** fail.

Third Law — Fakes Must Be Behaviorally Equivalent

- A test fake that is "simpler" than reality in a way that could hide a bug is a bluff fake.
- `TestEndpointsRepository` **MUST** enforce duplicate rejection (Room primary-key conflict) and default-endpoint seeding just like `EndpointsRepositoryImpl`.

- `TestLocalNetworkDiscoveryService` MUST simulate real `NsdManager` behaviors (e.g., `_lava._tcp.local.` service-type suffix) that affect production parsing logic.
- If the real implementation has a branch, the fake MUST have a matching branch or a documented limitation.

Fourth Law — Integration Challenge Tests

- Every feature MUST include at least one **Integration Challenge Test** that exercises the real implementation stack end-to-end: `ViewModel` → `UseCase` → `Repository` → (Fake) `Service`.
- Challenge Tests must use the **actual production classes** at every layer; only external boundaries may be faked.
- A Challenge Test that passes MUST guarantee the feature works for a real user under the tested conditions.

Fifth Law — Regression Immunity

- When a production bug is discovered, the fix commit MUST include a test that would have failed before the fix.
- If such a test cannot be written, the code is untestable and must be refactored for testability before the fix is accepted.
- Code coverage metrics are meaningless if the tests are bluffs; behavioral guarantees are the only metric that matters.

Sixth Law — Real User Verification (Anti-Pseudo-Test Rule)

A test that passes while the feature it covers is broken for end users is the most expensive kind of test in this codebase — it converts unknown breakage into believed safety. This has happened in this project before: tests and Integration Challenge Tests executed green while large parts of the product were unusable on a real device. That outcome is a constitutional failure, not a coverage failure, and it MUST NOT recur.

Every test added to this codebase from this point on MUST satisfy ALL of the following. Violation of any of them is a release blocker, irrespective of coverage metrics, CI status, reviewer sign-off, or schedule pressure.

1. **Same surfaces the user touches.** The test must traverse the production code path the user's action triggers, end to end, with no shortcut that bypasses real wiring. If the user's action is "open screen → tap button → see result", the test exercises the same screen, the same button handler, and the same result-rendering code — not a synthetic call into the `ViewModel` that skips the screen, and not a hand-rolled flow that skips the network/database boundary the real action crosses.
2. **Provably falsifiable on real defects.** Before merging, the author MUST run the test once with the underlying feature deliberately broken (throw inside the use case, return the wrong row from the repository, return the wrong status from the API) and confirm the test fails with a clear assertion message. The PR description MUST state which deliberate break was used and what failure the test produced. A test that cannot be made to fail by breaking the thing it claims to verify is a bluff test by definition.

3. **Primary assertion on user-visible state.** The chief failure signal of the test **MUST** be on something a real user could see or measure: rendered UI text, persisted database row, HTTP response body / status / header, file written to disk, packet on the wire. "Mock was invoked N times" is a permitted secondary assertion, never the primary one.
4. **Integration Challenge Tests are the load-bearing acceptance gate.** A green Challenge Test means a real user can complete the flow on a real device against real services — not "the wiring compiles". A feature for which a Challenge Test cannot be written is, by definition, not shippable. Refactor for testability or remove the feature.
5. **CI green is necessary, not sufficient.** "All tests pass" is not a release signal. Before any release tag is cut, a human (or a scripted black-box runner that drives the real UI / real HTTP API) **MUST** have used the feature on a real device or environment and observed the user-visible outcome described in the feature spec. Tag scripts (e.g. `scripts/tag.sh`) **MUST NOT** be run until that verification is documented.
6. **Inheritance.** This Sixth Law applies recursively to every submodule, every feature, and every new artifact added to the project (including the Go API service). Submodule constitutions **MAY** add stricter rules but **MUST NOT** relax this one.

Seventh Law — Tests **MUST** Confirm User-Reachable Functionality (Anti-Bluff Enforcement)

The Sixth Law states what tests must satisfy. The Seventh Law states how we enforce it mechanically — because the project has shipped passing-tests with broken-features at least once, and the operator's standing mandate (2026-04-30) is that this **MUST NOT** recur. The Seventh Law is the teeth.

Every commit that adds, modifies, or removes a test in this codebase, **AND** every commit that adds or modifies a user-facing feature, is bound by the following clauses. Violation is a release blocker — pre-push hooks reject the push, tag scripts refuse to operate, and reviewers **MUST** raise the violation as a blocking review comment.

1. **Bluff Audit Stamp on every test commit.** When a commit adds or modifies a `*Test.kt` / `*_test.go` / equivalent file, the commit message body **MUST** contain a "Bluff-Audit:" block in this exact format:

```
Bluff-Audit: <test-name-or-file>
  Mutation: <what was deliberately broken in the production
code>
  Observed-Failure: <copy-paste of the test failure message
after mutation>
  Reverted: yes
```

At least one mutation per test class added/modified. Pre-push hook rejects test commits without this stamp. The mutation **MUST** target the production code path the test claims to cover — mutating an unrelated branch is itself a bluff.

2. **Real-Stack Verification Gate per feature.** For every feature whose acceptance criterion mentions user-visible behavior (a screen, an HTTP response, a downloaded file, a persisted row), the feature is not "complete" until a real-stack test exists that:

- Hits the real third-party service over the network where the service is third-party (rutracker.org, rutor.info, etc.) — gated by `-PrealTrackers=true` so default test runs don't make outbound calls
- Runs against a real Postgres instance where the service is our own (lava-api-go) — gated by `-Pintegration=true` and using podman/docker for the database
- Drives the real Compose UI on a real Android emulator/device where the feature is UI — gated by `-PdeviceTests=true` and using `adb shell` instrument against `connectedAndroidTest`

A feature lacking such a real-stack test is documented as **not user-reachable** until the gap is closed. The matching coverage exemption ledger entry is mandatory.

3. **Pre-Tag Real-Device Attestation.** Before any release tag is cut, the operator **MUST** execute the user-visible flows (login, search, browse, view topic, download .torrent) on a real Android device against the real backend services and record a JSON attestation file at `.lava-ci-evidence/<tag-name>/real-device-attestation.json`. The attestation **MUST** include: device model, Android version, app version, timestamp, command-by-command checklist of executed user actions, and at least 3 screenshots OR a video recording referenced by hash. `scripts/tag.sh` **MUST** refuse to operate on a commit lacking the matching attestation. There is no exception. "Operator was busy" is not an exception. "Test environment was unstable" is not an exception. Untested-on-device commits do not get tags.

4. **Forbidden Test Patterns (per-language).** The following patterns are bluffs by construction. Pre-push hooks reject diffs that introduce them:

- **Mocking the System Under Test.** A `@MockK` / `@Mock` on the class whose name appears in the test class name (e.g. `mockk<RuTrackerSearch>` in `RuTrackerSearchTest`) is the canonical bluff. The SUT must be a real instance. Mocks are permitted **ONLY** at the boundaries below the SUT (Android system services, real HTTP sockets when not using `MockWebServer`, real database drivers when not using in-memory variants).
- **Verification-only assertions.** A test whose **ONLY** assertion is `verify { mock.foo() }` or `coVerify { ... }` is a call-counting test, not a behavior test. The Sixth Law clause 3 requires the primary assertion to be on user-visible state.
- **@Ignore'd tests with no follow-up issue.** An ignored test asserts nothing and adds noise. If a test must be temporarily disabled, the `@Ignore("reason")` argument **MUST** cite an issue number tracking re-enablement; the issue **MUST** be triaged in the next phase.
- **Tests that build the SUT but never invoke its methods.** A test that constructs a `ViewModel`, wires its dependencies, and asserts only `assertNotNull(viewModel)` is a compilation test, not a behavior test.
- **Acceptance gates whose chief assertion is BUILD SUCCESSFUL.** A Gradle task green-light is necessary but never sufficient — the gate **MUST** also assert on a meaningful outcome (correct test count, presence of evidence file, field-level data equality, etc.).

- **Hardcoded-port multi-target test runners (added 2026-05-05 from the matrix-runner port-collision incident).** A multi-target test runner (multi-AVD matrix, multi-VM matrix, multi-host distributed test) that hardcodes a single port/serial/endpoint and relies on per-iteration cleanup to free that target for the next iteration is a bluff by construction. When cleanup is unreliable (which it is under normal operating conditions: race conditions, residual state, partially-failed teardowns), the runner targets the LAST surviving target instead of the new one — silently testing the wrong thing N times while reporting N successful test rows for N different targets. The remediation pattern is **dynamic discovery**: pre/post snapshot of the target list (e.g. adb devices for emulators) to detect the NEW target appearing after each launch. Forensic anchor: the 7-day-old port-collision bluff in submodules/containers/pkg/emulator/Boot() exposed by the 2026-05-05 ultrathink-driven systematic-debugging session — see .lava-ci-evidence/sixth-law-incidents/2026-05-05-matrix-runner-port-collision-bluff.json. The architectural fix landed in submodules/containers commit 648a4bb.
- **Forward-compat skip without a tracking citation (added 2026-05-05).** A @SdkSuppress(maxSdkVersion = N) (or analogous skip) without a comment citing an open issue + an incident JSON is a green-on-skip — the test reports PASS by not running, which is identical in failure-mode to @Ignore without a tracking issue. Per clause 6.J, every skip MUST have a tracking citation. The forward-compat-skip pattern itself is permitted; the unmotivated skip is the bluff. Forensic anchor: the Pixel_9a (API 36) Compose-LazyLayout-Looper failure surfaced by the matrix-runner architectural fix; see .lava-ci-evidence/sixth-law-incidents/2026-05-05-pixel9a-espresso-api36-incompatibility.json.

5. **Recurring Bluff Hunt.** Once per development phase (every 2-4 weeks of active work), the team performs a bluff hunt. Procedure:

```
# Pick 5 random *Test.kt files from the project
bluff_targets=$(find core feature -name '*Test.kt' | shuf -n 5)

# For each target:
for t in $bluff_targets; do
    # 1. Read the test, identify the production class it claims
    #    to cover
    # 2. Apply a deliberate mutation to that production class
    # 3. Run the test
    # 4. If the test still passes, the test is a bluff — file an
    #    issue,
    #    classify the bluff (mock-the-SUT, count-only assertion,
    #    etc.),
    #    and either rewrite or remove the test
    # 5. Revert the mutation; the test must pass again
done
```

Output recorded in .lava-ci-evidence/bluff-hunt/<date>.json with: targets, mutations, observed failures or surviving passes, and any issues opened. The bluff hunt is a phase gate — a phase cannot be marked complete until the bluff hunt has run and any surviving bluffs are addressed.

5.a — Cadence tightening (added 2026-05-05, formalized in §6.N.1). The 2-4 week phase-end cycle remains the baseline, but three additional triggers fire IN-cycle:

1. **Per operator anti-bluff-mandate invocation.** First invocation in any 24h window: full 5+2 hunt (5 *Test.kt files per the baseline rule + 2 production-code files per §6.N.2). Subsequent invocations within the same 24h: lighter "incident-response hunt" — 1-2 files most relevant to the invocation context (e.g., the area of code the latest discovery flagged).
2. **Per matrix-runner / gate change** (pre-push enforced — owed via §6.N-debt). Any commit that touches submodules/containers/pkg/emulator/, scripts/run-emulator-tests.sh, scripts/tag.sh, or scripts/check-constitution.sh MUST be accompanied by a 1-target falsifiability rehearsal in the area of change.
3. **Per phase-gating attestation file added** (pre-push enforced — owed via §6.N-debt). Any new file under .lava-ci-evidence/sp3a-challenges/, .lava-ci-evidence/<tag>/real-device-verification.{md,json}, or .lava-ci-evidence/sixth-law-incidents/ MUST be accompanied by a falsifiability rehearsal of the production code path the attestation claims to cover.

5.b — Production-code coverage (added 2026-05-05, formalized in §6.N.2). Bluff hunts MUST sample production code, not just test files. Layered:

- **Mandatory minimum (per phase):** 2 files from gate-shaping production code. Canonical list: scripts/tag.sh helpers, scripts/check-constitution.sh, scripts/bluff-hunt.sh, submodules/containers/pkg/emulator/, submodules/containers/cmd/emulator-matrix/, the matrix runner's writeAttestation function. The list grows as new gate-shaping code lands.
- **Recommended additional (per phase):** 0-2 files from broader CI-touched code — anything invoked by scripts/ci.sh or scripts/run-emulator-tests.sh.
- **Conceptual filter:** for each candidate file, ask "would a bug here be invisible to existing tests?" Prefer files where the answer is yes.

The mutation-rehearsal protocol from clause 5 baseline applies unchanged. Output recorded under .lava-ci-evidence/bluff-hunt/<date>.json.

6. Bluff Discovery Protocol. When a real user reports a bug whose corresponding tests are green, a Seventh Law incident is declared. The protocol:

- The fix commit for the user-visible bug MUST include a regression test that, before the fix, fails with a clear assertion message
- The bluff that hid the bug MUST be diagnosed and recorded in .lava-ci-evidence/sixth-law-incidents/<date>.json with: which test was the bluff, why it was a bluff (which clause was violated), the mutation that would have caught it
- The bluff classification (e.g. "mocked the network call", "asserted only on call count") MUST be added to the Forbidden Test Patterns list above if not already there
- The Seventh Law itself MUST be reviewed for a new clause that prevents the same class of bluff in the future

7. Inheritance and Propagation. The Seventh Law applies recursively to every submodule, every feature, and every new artifact (including all `vasic-digital/*` submodules and the `lava-api-go` service). Each submodule's `CLAUDE.md` MUST contain either the verbatim Seventh Law or a link to this section in the parent. Submodules MAY add stricter clauses but MUST NOT relax any clause. New submodule adoption is conditional on Seventh Law compliance — non-compliant external submodules MUST be forked under `vasic-digital/` and brought into compliance before adoption.

Sixth Law extensions (lessons-learned addenda)

The clauses above are immutable. The clauses below are added when a real bug ships green and we need to prevent the *class* of bug, not just the specific instance.

Operator-authorized pin-policy waiver — HelixQA always-track-upstream (2026-05-16, Phase 4-C-1 Q9). The Decoupled Reusable Architecture rule's pins-frozen-by-default policy applies to every owned-by-us submodule EXCEPT HelixQA. HelixQA's pin (`submodules/helixqa/`) is **always-track-upstream** per operator decision Q9 of Phase 4-C-1 (`docs/plans/2026-05-16-helixqa-go-package-linking-design.md §G.9`). Rationale: HelixQA is in active development (per its `IMMEDIATE_EXECUTION_PLAN.md`); locking Lava to a stale HelixQA pin would defeat the Phase 4-C objective of consuming HelixQA's evolving adapter surface. Mechanical implication: when HelixQA's main advances, the next Lava commit MAY bump `submodules/helixqa/` to that HEAD without a separate operator approval ceremony per submodule — the standing Q9 authorization covers it. Other submodules (`auth`, `cache`, `challenges`, `concurrency`, `config`, `containers`, `database`, `discovery`, `http3`, `mdns`, `middleware`, `observability`, `ratelimiter`, `recovery`, `security`, `tracker_sdk`) remain pins-frozen by default; their bumps still require deliberate per-submodule operator authorization. The `./constitution/` submodule itself remains pinned + advanced only per CONST-049's 7-step pipeline. Per §6.AC.6 inheritance: this waiver is project-specific to Lava's HelixQA consumption and does NOT propagate to other consuming projects automatically — each consuming project decides per Q9 whether their own HelixQA pin tracks upstream.

Open constitutional debt: §6.X-debt (emulator process inside a podman/docker container managed by `submodules/containers/pkg/emulator/`; PARTIAL CLOSE landed via Containers 562069e7 + parent 888378a6; darwin/arm64 sub-debt RESOLVED 2026-05-20 (§6.L 67th cycle) via the per-OS-acceleration model in Containers c1871138 + 6aff7ea8 — `AccelProfileForOS / ResolveRunner / GateEligibleForOS` in `pkg/emulator/accel.go` + `emulator-matrix --runner=auto`: the OS-correct accelerated GATE runner is per-OS — containerized+KVM on Linux, host-direct+HVF on macOS, host-direct+WHPX on Windows. A Linux container on macOS physically cannot reach HVF (a host-only API), so host-direct+HVF IS the macOS gate runner, not a workstation-only fallback. PROVEN by the C00 cold-start canary + the full 37-class / 46-test Challenge suite on Pixel_8/API35 = 43 pass / 3 credential-skip / 0 fail. Linux x86_64 containerized+KVM remains the Linux gate-host path; `./lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json` records the original gap); **§6.Y-debt (added 2026-05-14)** — pre-push hook + `scripts/check-constitution.sh`

enforcement of the post-distribution version-bump-first rule; documented but not yet mechanically enforced; **§6.Z-debt (added 2026-05-14)** — `scripts/firebase-distribute.sh` Phase 1 Gate 6 + pre-push hook check that every distribute carries a test-evidence file with matching commit SHA + recent timestamp + BUILD SUCCESSFUL for the mandatory test set; documented but not yet mechanically enforced (operator + reviewer manually verify until close); **§6.AA-debt (added 2026-05-14, PARTIAL CLOSE)** — paired `last-version-{debug, release}` per-channel pointer split landed in commit TBD (`firebase-distribute.sh` now writes per-channel pointers + the §6.P monotonic-version gate consults the channel-matched pointer); default-mode flip to `--debug-only` + refusal of out-of-order `--release-only` without companion debug-stage evidence still OWED; **§6.AB-debt (added 2026-05-14)** — Detekt rule + Go-vet check for the §6.AB anti-bluff completeness checklist (per-feature rendering / state-machine / gating); documented but not yet mechanically enforced (operator + reviewer + sampled bluff-hunt manually verify until close); **§6.AC-debt (added 2026-05-14)** — Detekt + Go-vet rules flagging `try / catch` blocks lacking `recordNonFatal / recordWarning / explicit // no-telemetry: <reason> opt-out`; pre-push hook integration; documented but not yet mechanically enforced; **§6.AD-debt (added 2026-05-14, 29th §6.L cycle; PARTIAL CLOSE rolling through 2026-05-17)** — HelixConstitution submodule incorporated at `./constitution/`. **Closed sub-items (2026-05-17 audit):** ~~per-submodule + per-scoped CLAUDE.md inheritance pointer-block propagation~~ CLOSED (98 per-scope docs verified by `scripts/check-constitution.sh`); ~~no-guessing-vocabulary grep gate~~ CLOSED (extracted to standalone `scripts/check-no-guessing-vocabulary.sh` + 7-fixture hermetic test at `tests/check-constitution/test_no_guessing_vocabulary.sh`); ~~Classification: line pre-push enforcement~~ CLOSED (Check 8 in `.github/hooks/pre-push` line 214 + hermetic test at `tests/pre-push/check8_test.sh`); ~~docs/scripts/commit_all.sh.md~~ external user guide CLOSED (144-line guide present); ~~build-resource-stats-tracker (HelixConstitution §11.4.24)~~ CLOSED (`scripts/build-stats-sample.sh` + `scripts/build-stats-report.sh` + `.lava-ci-evidence/build-stats/registry.tsv` + `docs/build-stats/Stats.md` all present). **Remaining OWED:** HelixConstitution CM-* gate set explicit wiring (CM-COMMIT-DOCS-EXISTS, CM-FIXED-COLUMN-ALIGNMENT, CM-SCRIPT-DOCS-SYNC, CM-ITEM-STATUS-TRACKING, CM-ITEM-TYPE-TRACKING, CM-ITEM-OPERATOR-BLOCKED-DETAILS, CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT, CM-SUBAGENT-DELEGATION-AUDIT) — note that §6.AD.3 Path B equivalence-maps CM-ITEM-STATUS-TRACKING / CM-ITEM-TYPE-TRACKING / CM-FIXED-COLUMN-ALIGNMENT / CM-ITEM-OPERATOR-BLOCKED-DETAILS / CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT to the existing `.lava-ci-evidence/layout` (CLOSED-BY-EQUIVALENCE per the constitution clause); CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION CLOSED by Check 8; what truly remains: CM-COMMIT-DOCS-EXISTS (verify referenced docs in commit messages resolve), CM-SCRIPT-DOCS-SYNC (script [?] `docs/scripts/*.md` sync — Phase 8 `scripts/check-canonical-root-and-upstreams.sh` covers root inheritance but not per-script sync), CM-SUBAGENT-DELEGATION-AUDIT (subagent delegation audit trail).

Recently resolved debt (kept as forensic anchor, no longer load-bearing):
§6.K-debt (closed 2026-05-07, pkg/vm + image-cache spec), §6.N-debt
(closed 2026-05-05 evening, Group A-prime spec), §6.S
CONTINUATION.md mandate (active and enforced).

6.A — Real-binary contract tests (added 2026-04-29)

Forensic anchor: the lava-api-go container ran 569 consecutive failing healthchecks in production while serving real HTTP/3 + HTTP/2 traffic. `docker-compose.yml` invoked `healthprobe --http3 https://localhost:8443/health`, but the binary at `lava-api-go/cmd/healthprobe/main.go` only registered `-url / -insecure / -timeout`. `flag.Parse()` rejected the unknown `--http3` and the probe exited 1 every 10 seconds. `docker compose config --quiet` was happy with the YAML; every functional test was green; the orchestrator labelled the container "unhealthy" and there was no test asserting the probe could even start.

Rule. Every place where one of our scripts/compose files invokes a binary we own (or build) MUST be covered by a contract test that:

1. Builds (or locates) the target binary.
2. Recovers the binary's registered flag set from its actual `Usage` output (or by importing the package's flag set, where structurally possible).
3. Asserts every flag passed by the script/compose is a strict subset of the binary's registered flag set.
4. Carries a falsifiability rehearsal sub-test that re-introduces the historical buggy flag and confirms the contract checker rejects it. The recorded run must appear in the commit body (Test/Mutation/Observed/Reverted protocol).

The canonical implementation is `lava-api-go/tests/contract/healthcheck_contract_test.go`. New compose healthchecks, new lifecycle scripts, new build glue, new release scripts MUST gain an analogous contract test before the change can be merged.

6.B — Container "Up" is not application-healthy (added 2026-04-29)

`docker ps / podman ps` reporting Up only means PID 1 is alive — the application inside may be crash-looping, stuck on startup, or (as 6.A shows) green-on-the-outside-broken-on-the-inside. A test that asserts `service.State.Status == "running"` is a bluff test by clauses 1 and 3. Use:

- The same probe the orchestrator uses (or a real-protocol probe at the same surface).
- An end-to-end functional request that exercises the user-visible code path, not the lifecycle plumbing.

The lava-api-go integration tests already do this (real Gin engine via `httptest`, real Postgres in podman, real cache). New container-bearing features MUST follow the same pattern.

6.C — *Mirror-state mismatch checks before tagging (added 2026-04-29)*

When mirroring across N upstreams, "all mirrors push succeeded" is one assertion; "all mirrors converge to the same SHA at HEAD" is a stronger one. A push that fences against branch protection on one mirror but goes through on the others produces a green-looking session log and a divergent state at rest. `scripts/tag.sh` MUST verify post-push that both upstreams report the same tip SHA before reporting success. Future releases of `tag.sh` SHOULD record the per-mirror SHA in the evidence file alongside the pretag-verify probe results.

6.D — *Behavioral Coverage Contract (added 2026-04-30, SP-3a)*

Coverage is measured behaviorally, not lexically. Every public method of every interface added under `core/tracker/api/`, `submodules/tracker_sdk/api/`, or any future SDK contract module MUST have at least one real-stack test that traverses the same code path a user's action triggers. Line coverage is reported as a secondary metric. Uncovered lines after the behavioral pass are exempted only via an entry in the per-spec exemption ledger (`docs/superpowers/specs/<spec>-coverage-exemptions.md`) naming the line, the reason, the reviewer, and the date. Blanket coverage waivers are forbidden. The SP-3a exemption ledger lives at `docs/superpowers/specs/2026-04-30-sp3a-coverage-exemptions.md`.

6.E — *Capability Honesty (added 2026-04-30, SP-3a)*

A `TrackerDescriptor` (or any future descriptor of a feature-bearing component) that declares a capability MUST cause `getFeature()` to return a non-null implementation for the corresponding feature interface. The historical "Not implemented" stub pattern (e.g., `ProxyNetworkApi.checkAuthorized()` returning `false` despite being declared) is a constitutional violation. Capability declared $\Rightarrow$ feature interface returned $\Rightarrow$ at least one real-stack test exists for the capability. CI gate: a unit test enumerates every descriptor, every declared capability, and asserts the corresponding `getFeature()` call returns non-null. The SP-3a Phase 2/3 implementations of `RuTrackerClient` and `RuTorClient` already gate `getFeature<T>()` on capability set; new tracker modules MUST follow the same pattern.

6.F — *Anti-Bluff Submodule Inheritance (added 2026-04-30, SP-3a)*

Clauses 6.A through 6.E inherit recursively to every `vasic-digital` submodule mounted in this repository, to every future submodule, and to every code module added to a submodule. A submodule constitution MAY add stricter rules (e.g. Tracker-SDK's "no domain shape" rule) but MUST NOT relax 6.A–6.F. Adopting an externally maintained dependency that does not satisfy these clauses requires forking it under `vasic-digital/` first. The Seventh Law (Anti-Bluff Enforcement) inherits under the same rule. The Go API service (`lava-api-go/`) inherits 6.D and 6.E binding on its `rutracker` bridge work in the SP-3a-bridge follow-up.

6.G — End-to-End Provider Operational Verification (added 2026-05-04)

Forensic anchor: The Internet Archive provider (archiveorg) shipped with `AuthType.NONE` but the `ProviderLoginScreen` → `ProviderLoginViewModel` → `LavaTrackerSdk.login()` flow hung on `CircularProgressIndicator` because `onSubmitClick()` did not short-circuit for no-auth providers and never reset `isLoading = false` when `sdk.login()` returned null. All Challenge Tests C1–C8 passed green while the provider was completely unusable.

Rule: Every provider declared in `TrackerDescriptor` and shown in the provider list **MUST** have at least one end-to-end test that exercises the real production stack from the user's first tap to a successful outcome on a real device or real service. A provider that appears selectable but cannot complete its primary flow (search, browse, download, or anonymous login) is a constitutional violation, irrespective of unit-test coverage.

1. **Auth-type 诚实.** A provider with `AuthType.NONE` must have a test that taps "Continue" and reaches the main app. A provider with `AuthType.FORM_LOGIN` must have a test that enters real credentials (from `.env`) and authenticates successfully against the real service. A provider with `AuthType.API_KEY` must have a test that enters a real key and succeeds.
2. **No synthetic-only coverage for provider flows.** Unit tests of `ProviderLoginViewModel` in isolation are insufficient. The test must traverse the real screen, the real `ViewModel`, the real `LavaTrackerSdk`, and the real HTTP client hitting the real base URL declared in the descriptor.
3. **Falsifiability rehearsal per provider.** Before any new provider is added, the author **MUST** deliberately break the provider's primary flow (e.g., return `AuthResult.Error` from `login()`, throw in `search()`, return empty results from `browse()`) and confirm the end-to-end test fails with a clear assertion. The PR description **MUST** state which break was used and what failure the test produced.
4. **Challenge Test expansion.** Challenge Tests C1–C8 cover `rutracker` and `rutur`. For every additional provider, a new Challenge Test **MUST** be added in `app/src/androidTest/kotlin/lava/app/challenges/` before that provider is declared "supported." Unsupported providers **MUST NOT** appear in the provider list shipped to end users.
5. **Real-device or emulator mandate.** Provider end-to-end tests **MUST** run on a real Android device or on an Android emulator inside a container. Running them on the JVM with Robolectric or on a mocked `NavHost` is a bluff test by clause 1.

6.H — Credential Security Inviolability (added 2026-05-04)

Forensic anchor: Real tracker credentials (`RUTRACKER_USERNAME`, `RUTRACKER_PASSWORD`, etc.) were stored in `../Boba/.env` and had to be copied into this repository for integration testing. Accidental commit of these credentials would be a security incident.

Rule: Files containing real credentials, signing keys, or API secrets **MUST** never be committed to any upstream. Violation is a release blocker and a security incident.

1. **.env and .env.* are inviolable.** The root `.gitignore` **MUST** contain `.env`, `.env.*`, and `.env.local`. No `.env` file shall ever appear in `git ls-files`.
2. **keystores/ are inviolable.** The root `.gitignore` **MUST** contain `keystores/`. Keystore files contain private signing keys.
3. **app/google-services.json is inviolable.** The root `.gitignore` **MUST** contain this file. It contains Firebase API keys.
4. **No credential strings in source code.** No hardcoded username, password, API key, or token shall exist in any `.kt`, `.java`, `.go`, `.gradle`, `.md`, or `.xml` file. The only permissible location for real credentials is `.env` (gitignored) or a local secrets manager.
5. **Pre-push credential scan.** The pre-push hook **MUST** reject any push that introduces a file matching `.env*` or `*.keystore` or that adds a line matching credential patterns (`password=`, `api_key=`, etc.) to tracked files. `scripts/check-constitution.sh` **MUST** verify this.
6. **Leak response protocol.** If credentials are accidentally committed, the commit **MUST** be purged from all upstreams immediately, credentials **MUST** be rotated, and the incident **MUST** be recorded in `.lava-ci-evidence/sixth-law-incidents/` per the Seventh Law clause 6 protocol.
7. **Inheritance.** This clause inherits recursively to every submodule. Submodule constitutions **MAY** add stricter rules but **MUST NOT** relax this one.

6.I — Multi-Emulator Container Matrix as Real-Device Equivalent (added 2026-05-04)

Forensic anchor: clauses 6.G and the Sixth Law clause 5 + Seventh Law clause 3 originally required **operator real-device attestation** before a release tag. In practice, "real device" became a single Pixel-class phone, which (a) only exercises one Android version + one screen size at a time, (b) creates an operator-availability bottleneck where releases stall waiting on hardware, and (c) silently encourages "looks fine on my Pixel" to substitute for "works on the matrix of devices users actually run." That is the same class of bluff the Sixth Law was written to prevent — a single physical device is not a "matrix" any more than a single test is a "suite."

Rule. Real-device verification, where required by clause 6.G clause 5, the Sixth Law clause 5, or the Seventh Law clause 3, is satisfied by **a multi-emulator container matrix** that meets **ALL** of:

1. **Container-bound.** The emulators **MUST** run inside the project's emulator container (`docker-compose.test.yml` + `docker/emulator/Dockerfile`), not on a developer's host directly. Host-emulator runs are permitted for development iteration; the constitutional gate is the container path because it is reproducible across operator workstations and self-hosted runners (Local-Only CI/CD rule).
2. **Android version coverage.** At minimum: API 28 (Android 9), API 30 (Android 11), API 34 (Android 14), and the latest stable API the project's `compileSdk` targets. A subset is permitted **ONLY** if a clause-6.G evidence entry names exactly which API levels were skipped and why; "fast iteration" is not a permitted reason.

3. **Screen-size coverage.** At minimum: one phone-class device (any Pixel small/medium), one tablet-class device (Pixel Tablet or Nexus 9), and one TV-class device when the feature touches TvActivity or the leanback manifest entries. Form factor matters: layouts that pass on a 6" phone routinely break on a 10" tablet, and the project ships a TvActivity exactly because of that.
4. **Per-AVD attestation row.** The evidence file at `.lava-ci-evidence/<tag>/real-device-verification.md` (or `real-device-attestation.json`) MUST contain one row per AVD-test pair: AVD name, Android API level, screen size, test class executed, pass/fail, screenshot or test-report path, timestamp. A green matrix is a SET of rows, not a single PASS line. Missing rows are missing evidence; "all green" without per-AVD detail is a bluff in itself. **Group B extension (added 2026-05-05 evening):** every row additionally carries `diag` (`target/sdk/device/adb_devices_state` — the per-AVD forensic snapshot captured immediately before instrumentation invocation), `failure_summaries` (parsed JUnit `<failure>` + `<error>` entries; empty array on pass), and `concurrent` (the matrix runner's `--concurrent` setting at the time the row ran; 1 = serial). The run-level gating field (boolean; `true` $\Leftrightarrow$ `--concurrent == 1 AND --dev=false`) is the constitutional eligibility flag — `scripts/tag.sh` refuses to operate on attestations whose gating is false (Group B Gate 2) OR whose any row has `concurrent != 1` (Group B Gate 1) OR whose any row's `diag.sdk` does not equal `api_level` (Group B Gate 3, the AVD-shadow bluff). These three additional gates are tag-time enforcement of the per-row evidence the Group B spec at `docs/superpowers/specs/2026-05-05-anti-bluff-mandate-reinforcement-group-b-design.md` (commit `e2a3af2`) introduces.
5. **Falsifiability rehearsal still applies per matrix.** For at least one AVD per Android-version-class, the deliberate-mutation rehearsal (Sixth Law clause 2) MUST be performed and recorded — the matrix exists to detect divergence between API levels, so a mutation that breaks on API 34 but not on API 28 is a finding worth keeping.
6. **Cold-boot only for the gate run.** The matrix run that produces the evidence MUST start from cold-booted emulators (no snapshot reload). Snapshot reuse during dev is fine; for the gate it is forbidden, because snapshot drift is a known source of false-green ("worked yesterday" $\neq$ "works after upgrade").
7. **scripts/tag.sh enforcement.** `scripts/tag.sh` MUST refuse to operate on a commit whose `.lava-ci-evidence/<tag>/real-device-verification.md` lacks AT LEAST one row per minimum-coverage AVD listed in clause 2 + clause 3, with all such rows reporting pass. There is no exception. "Operator was busy" is not an exception. "Emulator was flaky" is not an exception — flakiness must be diagnosed and fixed; intermittent green is constitutionally indistinguishable from intermittent red.
8. **Inheritance.** Clause 6.I applies recursively to every feature, every submodule, and every new artifact. Submodule constitutions MAY add stricter coverage requirements (e.g. "tablet-class is mandatory for this feature even though the matrix minimum is phone-class") but MUST NOT relax this clause.

The matrix is the gate. A single passing emulator is not.

6.I-debt — Containers-submodule extraction (constitutional debt, 2026-05-04)

The current `docker-compose.test.yml + docker/emulator/Dockerfile + scripts/run-emulator-tests.sh` are Lava-side container-orchestration code. Per the Decoupled Reusable Architecture rule, generic container-orchestration concerns **MUST** live in `vasic-digital/Containers`. The matrix-runner machinery required by clause 6.I (multi-AVD orchestration, per-AVD attestation collection, cold-boot enforcement, falsifiability-rehearsal hooks) is the same kind of capability another `vasic-digital` project would want — therefore it **MUST** be extracted to a new submodules/`containers/pkg/emulator/` package and a `cmd/emulator-matrix/` CLI. Lava's `scripts/run-emulator-tests.sh` then becomes thin glue invoking the Containers CLI with Lava-specific arguments (the AVD list, the test class, the evidence path).

This is recorded as constitutional debt because the immediate-session work (writing + running Challenge Tests C9-C12 on the four hidden providers) needs the matrix capability NOW, while the proper extraction is multi-hour work that includes Anti-Bluff falsifiability rehearsal of the new Containers package's own tests. Acceptable transitional state: development-iteration runs use Lava-side compose; the constitutional-gate run (the one that produces `.lava-ci-evidence/<tag>/real-device-verification.md` and unblocks `scripts/tag.sh`) **MUST** go through the Containers package once it ships. No release tag is cut while this debt is open.

Tracking: the next phase that touches release tagging **MUST** close this debt before its tag.

6.J — Anti-Bluff Functional Reality Mandate (added 2026-05-04)

Forensic anchor (recurring): the operator has now twice flagged that "tests pass green while features don't actually work for users" (Internet Archive stuck-on-loading + the broader concern that Sixth Law clauses 1-5 alone are insufficient enforcement). The Anti-Bluff Pact is mature; what is needed is the most direct possible re-statement so that no future agent or contributor can read past it.

Rule. Every test, every Challenge Test, and every CI gate added to or maintained in this codebase **MUST** do exactly one job: **confirm that the feature it claims to cover actually works for an end user, end-to-end, on the user's real surfaces.**

If a test passes while the feature is broken — irrespective of what the test's author intended, irrespective of what the test's name says, irrespective of how green CI looks — the test has failed at its only job. There is no "but the assertion was technically correct" defense. There is no "the test was just for the wiring" defense. There is no "we'll add the real test next sprint" defense.

The execution of tests and Challenge Tests in this project has ONE purpose: **guarantee the quality, the completion, and the full usability of the product by end users.** Anything that does not contribute to that guarantee is removed, refactored, or rewritten.

Mechanical implications:

1. CI green is necessary, NEVER sufficient. (Sixth Law clause 5 in even more direct words.)

2. A green Challenge Test means a real user can complete the flow on the gating matrix (clause 6.I). It does not mean "the wiring compiles" or "the mock returned the expected value."
3. Any agent or contributor may invoke clause 6.J to remove a test they can demonstrate is bluff (the test passes against a deliberately-broken production code path) — even retroactively, even if the test was sacred for years. The bluff classification + the broken-mutation evidence go to `.lava-ci-evidence/sixth-law-incidents/<date>.json`, and the removal is a NORMAL change, not a controversial one.
4. Adding `@Ignore` to a failing test without an open issue is a clause-6.J violation by construction — "ignored" is not "covered."
5. Adding a Challenge Test that the agent CANNOT execute against the gating matrix is itself a bluff, regardless of the assertion's plausibility. Either the test runs on the matrix and passes truthfully, or it does not exist.

This clause exists not to add new mechanics — clauses 6.A through 6.I + the Sixth and Seventh Laws already provide the mechanics — but to be the unambiguous restatement that a future agent or contributor cannot rationalize away. **Tests must guarantee the product works. Anything else is theatre.**

Inheritance. Applies recursively to every submodule, every feature, every new artifact. Submodule constitutions MAY emphasize this clause more loudly (e.g. by repeating it verbatim in their `CLAUDE.md`) but MUST NOT relax it.

6.K — Builds-Inside-Containers Mandate (added 2026-05-04)

Forensic anchor: the project's emulator-matrix gate (clause 6.I) requires test runs inside containers, but every build that PRODUCES the artifacts those tests exercise (the debug APK, the release APK, the Go API binary, the Ktor proxy fat JAR, the OCI image tarballs) currently runs on the developer's bare host. That is a constitutional inconsistency: the gate run is reproducible, the artifact under test is not. A debug APK built on developer A's machine with their installed Java toolchain + their `~/.gradle/caches/` + their `local.properties` is NOT byte-equivalent to the one built on developer B's machine, and neither is byte-equivalent to the one the CI gate would have produced — any of them might pass on the matrix while the others fail, and we would have no way to tell because there is no shared build surface.

Rule. Every build that produces an artifact placed in `releases/`, every build whose output is run inside the emulator-matrix or any clause-6.I gate, and every build whose output is signed for release MUST run inside the project's container path — specifically inside the `vasic-digital/Containers` submodule's build orchestration (existing `cmd/distributed-build` + `pkg/distribution` + `pkg/runtime` primitives), invoked by thin Lava-side glue.

1. Container-bound

builds. `:app:assembleDebug`, `:app:assembleRelease`, `:proxy:buildFatJar`, `lava-api-go` static-binary build, OCI image builds — all run inside a build container (a JDK-bearing container for Gradle, a Go-bearing container for the Go service) brought up by `submodules/containers`. Local incremental dev builds on the host are PERMITTED for iteration; the constitutional gate, the release-artifact build, and the build whose output goes through the emulator matrix MUST go through Containers.

2. **Containers MUST natively support Android emulators.** The existing `cmd/distributed-test` already routes Android-test execution through container orchestration. Per this clause, `submodules/containers/pkg/emulator/` is added as a first-class package alongside `pkg/runtime`, `pkg/compose`, `pkg/orchestrator`, `pkg/health`, `pkg/lifecycle`, `pkg/distribution`. Its responsibility: spin up Android emulators in containers (cold-boot per clause 6.I), wire adb to the host, install APKs, drive instrumentation tests, collect per-AVD attestation evidence, tear down. Lava's `scripts/run-emulator-tests.sh` becomes thin glue invoking this package.
3. **Containers SHOULD investigate non-Android emulators.** Roadmap items for `pkg/emulator/` (or a sibling `pkg/vm/` if architecturally cleaner), recorded as 6.K-debt and tracked but NOT release-blocking for the next Lava tag:
 - **QEMU full-system emulation** — boot non-x86 OS images (ARM, RISC-V, MIPS) inside KVM-accelerated containers for cross-architecture testing. Useful for Lava's signing/keystore code that may rely on JCA providers behaving differently across CPU architectures.
 - **Other OS emulators** — Linux distributions (Alpine, Debian, Fedora, Arch) for testing the Lava proxy's distroless container behavior across base-image families; FreeBSD for verifying `lava-api-go`'s POSIX-only assumptions; minimal Windows for verifying the gradle wrapper's `gradlew.bat` parity.
 - **iOS / macOS emulation** — out of scope unless and until Lava ships an iOS client; recorded here so the roadmap item exists.
4. **Sixth Law alignment.** Per 6.J ("tests must guarantee the product works"), an artifact built outside the gating container path and tested inside the gating emulator path is constitutionally suspect — green tests against an artifact the gate did not build are a bluff vector by construction. Closing 6.K closes that vector. Until 6.K is closed, every release-evidence note MUST cite which artifact came from which build path; tags with mismatched-path builds are forbidden.
5. **scripts/check-constitution.sh enforcement.** Once `submodules/containers/pkg/emulator/` ships, the constitution checker MUST verify the existence of: (a) the `pkg/emulator/` package in the pinned Containers submodule, (b) Lava-side thin-glue scripts referencing the package's CLI, (c) at least one passing test inside `submodules/containers/pkg/emulator/` exercising a real container-emulator boot via `pkg/runtime`'s auto-detected Docker/Podman. Failure of any of (a)/(b)/(c) is a clause-6.K violation; pre-push rejects.
6. **Falsifiability rehearsal applies to 6.K's own infrastructure.** Per 6.J + clause 6.A, the new Containers `pkg/emulator/` package's tests MUST themselves be falsifiable — deliberate-mutation rehearsal recorded in commit body, observed-failure captured, reverted. The same standard the Lava-side Anti-Bluff Pact applies.
7. **Inheritance.** Applies recursively to every submodule, every feature, every new artifact. Submodule constitutions MAY add stricter rules (e.g. "this submodule's binary MUST also be built inside Containers' multi-arch matrix") but MUST NOT relax this clause.

6.K-debt — Containers extension implementation (constitutional debt, 2026-05-04)

The clauses above are the contract. The implementation is owed: submodules/containers/pkg/emulator/ does not yet exist. Until it does, the Lava-side docker-compose.test.yml + docker/emulator/Dockerfile + scripts/run-emulator-tests.sh are the transitional path — they remain in this repo as Lava-domain glue rather than being extracted. The next phase that touches release tagging or the emulator-matrix gate **MUST** close this debt before its tag, and the close **MUST**: (1) add pkg/emulator/ to Containers, (2) add at least the QEMU baseline to a sibling pkg/vm/, (3) extract Lava's emulator-orchestration glue, (4) update the constitution checker per clause 6.K clause 5. No release tag is cut while this debt is open, except for hotfixes whose changeset does not touch the emulator-matrix gate's output.

RESOLVED 2026-05-07 via pkg/vm + image-cache bundled spec at docs/superpowers/specs/2026-05-05-pkg-vm-image-cache-design.md (commit c8dc198) and plan at docs/superpowers/plans/2026-05-05-pkg-vm-image-cache.md. Implementation chain: Containers commits on lava-pin/2026-05-07-pkg-vm (Phase A introduces pkg/cache/, pkg/vm/, cmd/vm-matrix/; Phase B refactors pkg/emulator/ to route image fetch through pkg/cache/). Lava parent commits on master (Phase C ships tools/lava-containers/vm-images.json + scripts/run-vm-{signing,distro}-matrix.sh + tests/vm-{signing,distro}/* + this RESOLVED note; Phase D bumps the pin + closure attestation). The §6.K-debt entry stays in this CLAUDE.md as a forensic record but is no longer load-bearing.

6.M — Host-Stability Forensic Discipline (added 2026-05-04 evening, recurrence forensics)

Forensic anchor: the operator reported a second possible host-stability incident on 2026-05-04 evening: "computer has stuck or it was suspended / sent to standby / hibernated or signed out (somehow by someone or by something — it is unclear)". A 5-step audit (uptime, who, journalctl logind events, free memory, OOM kernel events) confirmed that **no actual host event occurred** — the host had been continuously up 9h43m, the same user was logged in throughout, no logind transitions appeared in the journal, no OOM kills, no critical kernel events, no forbidden commands invoked anywhere in the session's command stream. The operator's perception was real, but no constitutional violation occurred and no project code path was implicated. Full audit recorded at .lava-ci-evidence/sixth-law-incidents/2026-05-04-perceived-host-instability.json.

The audit revealed that **the Forbidden Command List + the existing Host Machine Stability Directive were sufficient** — no command in the session's history matched any forbidden pattern. The clause that this incident motivates is not a new prohibition; it is a forensic discipline so the next perceived-instability event can be classified in 60 seconds rather than rederived from scratch.

Rule. Every perceived-instability event — whether a real host suspend, a real sign-out, OR an operator-perceived freeze without forensic evidence — MUST be classified into one of three categories AND recorded with the same audit set:

1. **Class I** — verifiable host event (poweroff, suspend, hibernate, sign-out). uptime reset; logged-in users changed; journalctl shows logind transition. Example: docs/INCIDENT_2026-04-28-HOST-POWEROFF.md. Triggers the post-poweroff recovery procedure (orphan-container audit, pre-push verification, etc.).
2. **Class II** — measurable resource pressure causing partial freeze (kernel OOM kill of a session process, swap exhaustion, thermal throttling, fs full). uptime continuous but journalctl shows the kernel intervening. No observed example yet in this project.
3. **Class III** — operator-perceived instability with NO forensic evidence. uptime continuous, same user, no journal events, no OOM, no resource pressure. Example: .lava-ci-evidence/sixth-law-incidents/2026-05-04-perceived-host-instability.json. The user's perception is real (often a long-running gradle build that paused GUI responsiveness, an emulator GPU lockup, or a remote SSH session disconnect), but no project code path is implicated and no host event is recorded.

Audit protocol (60-second forensic). Every perceived-instability response MUST run this set BEFORE concluding anything:

1. Uptime + logged-in users (Class I detector)

```
uptime; who
```

2. Logind events since session start (Class I detector)

```
journalctl --user --since '<start-time>' | grep -iE 'suspend|hibernate|poweroff|halt|kill-user|kill-session|terminate-user|terminate-session|sign-out'
```

3. Kernel critical events (Class II detector)

```
journalctl --since '10 min ago' | grep -iE 'oom|killed process|emergency|critical'
```

4. Memory + swap (Class II detector)

```
free -h
```

5. Disk pressure (Class II detector)

```
df -h /run/media/<volume>
```

6. Forbidden-command-in-tracked-files cross-check

```
grep -rE 'systemctl[[:space:]]+(suspend|hibernate|...)|logind[[:space:]]+(...)|pm-suspend|...' \
  --include='*.sh' --include='*.kt' --include='*.go' --include='*.kts' . | \
  grep -v '\.git\|build\|/INCIDENT_\|forbidden'
```

7. Container state (operator-directive: "destroy + rebuild" mandate)

```
podman ps -a | head
docker ps -a 2>/dev/null # may be "command not found" on hosts
                        using only podman
```

Outputs from each step go into the incident record. A perceived-instability event without an audit record is itself a Seventh Law violation — clause 6.J's "tests must guarantee the product works" applies to the project's response to incidents too.

Container-runtime safety analysis (recorded once, referenced forever).

- **Podman (rootless)** — the runtime in use on the operator's primary host. Rootless Podman has NO host-level power-management privileges by design. It cannot invoke `systemctl suspend`, cannot trigger login transitions, cannot sign out the user. The per-user `podman.service` is itself a session service; stopping it stops containers but does NOT affect the user's login session. Read-only operations (`podman ps`, `podman images`) are safe to invoke from any audit. Mutating operations (`podman rm`, `podman stop`) only affect containers, not the host. **Conclusion: Podman cannot cause Class I host events.** Recorded once here so future sessions don't re-derive.
- **Docker (rootful daemon)** — NOT installed on this host (`command not found`). When present elsewhere, the docker daemon runs as root, so in principle a maliciously-scripted container restart cycle could pressure the host. Lava does not install or invoke Docker; if a future host requires it, the rootless Podman compatibility layer (docker symlink to podman via `podman-docker` package) is preferred. **Conclusion for Lava-on-this-host: Docker is not a factor.**
- **Container builds (`./build_and_release.sh`)** — invokes `podman build + podman save` for image export. Both are session-scoped operations. Image-tar exports can be I/O heavy on `/run/media/...` filesystems (the audit's disk-pressure step catches this); archive-rotation hygiene is in `releases/` directory cleanup, not in container management.

Emulator zombie-cleanup hook (action item from this incident). `scripts/run-emulator-tests.sh` MUST be extended with a pre-boot hook that kills any qemu-system processes left over from interrupted matrix runs before launching new ones. Zombies don't cause host events but they (a) hold ADB ports, (b) consume RAM, (c) confuse the operator's audit. Owed in the next phase that touches the script.

Inheritance. Clause 6.M applies recursively to every submodule, every feature, and every new artifact. Submodule constitutions MAY add stricter forensic requirements (e.g., submodules/containers SHOULD record per-container-runtime audit findings as it's the source of truth for runtime detection) but MUST NOT relax this clause.

6.N — Bluff-Hunt Cadence Tightening + Production Code Coverage (added 2026-05-05)

Forensic anchor: the 7-day-old architectural bluff in `submodules/containers/pkg/emulator/Boot()` (hardcoded `ADBPort=5555`) exposed by ultrathink-driven systematic-debugging on 2026-05-05; recorded at `.lava-ci-evidence/sixth-law-incidents/2026-05-05-matrix-runner-port-collision-bluff.json`. The bluff was invisible to all existing `*Test.kt`-targeted bluff hunts because the buggy code was production Go code that no test could mutate-rehearse — only end-to-end multi-AVD matrix runs would have caught it, and those runs were themselves

rendered green by the bluff (textbook clause-6.J failure mode). The operator's 8th invocation of the anti-bluff mandate landed concurrently with this discovery, demanding tightened cadence and broader scope for bluff hunts.

6.N.1 — Tightened cadence. The 2-4 week phase-end cycle (Seventh Law clause 5 baseline) remains. Three additional triggers fire IN-cycle:

1. **Per operator anti-bluff-mandate invocation.** First invocation in any 24h window: full 5+2 hunt (5 *Test.kt per Seventh Law clause 5 baseline + 2 production-code files per §6.N.2). Subsequent invocations within the same 24h: lighter "incident-response hunt" — 1-2 files most relevant to the invocation context.
2. **Per matrix-runner / gate change** (pre-push enforced — see §6.N-debt below). Any commit that touches submodules/containers/pkg/emulator/, scripts/run-emulator-tests.sh, scripts/tag.sh, or scripts/check-constitution.sh MUST carry a Bluff-Audit stamp recording a 1-target falsifiability rehearsal in the area of change. The hook checks for the Bluff-Audit: block AND verifies the mutation reasoning targets a file in the diff.
3. **Per phase-gating attestation file added** (pre-push enforced — see §6.N-debt below). Any new file under .lava-ci-evidence/sp3a-challenges/, .lava-ci-evidence/<tag>/real-device-verification.{md,json}, or .lava-ci-evidence/sixth-law-incidents/ MUST be accompanied by a falsifiability rehearsal of the production code path the attestation claims to cover. The rehearsal evidence MAY be embedded in the attestation OR live as a companion file in the same directory.

6.N.2 — Production-code coverage in bluff hunts. Bluff hunts MUST sample production code beyond *Test.kt / *_test.go. Layered scope:

- **Mandatory minimum (per phase):** 2 files from gate-shaping production code. Gate-shaping = files whose output determines pass/fail of a constitutional gate. Canonical list: scripts/tag.sh helpers, scripts/check-constitution.sh, scripts/bluff-hunt.sh, submodules/containers/pkg/emulator/, submodules/containers/cmd/emulator-matrix/, the matrix runner's writeAttestation function. The list grows as new gate-shaping code lands.
- **Recommended additional (per phase):** 0-2 files from broader CI-touched code — anything invoked by scripts/ci.sh or scripts/run-emulator-tests.sh, including Lava-domain Kotlin/Go production paths.
- **Conceptual filter:** for each candidate file, ask "would a bug here be invisible to existing tests?" Prefer files where the answer is yes — those are the bluff-rich targets.

The mutation-rehearsal protocol from Seventh Law clause 5 applies unchanged: pick file → apply deliberate mutation that affects the gate's verdict → run the gate → confirm the gate fails (or surfaces the wrong outcome) → revert → re-run → confirm green again. Record outcome in .lava-ci-evidence/bluff-hunt/<date>.json.

6.N.3 — §6.N-debt (forward-reference to Group A-prime spec). The pre-push hook enforcement clauses (6.N.1.2 + 6.N.1.3 above) are documented but NOT yet implemented. Implementation is deferred to the Group A-prime spec, which the parent Group A spec spawns as the next brainstorming + writing-plans cycle (docs/superpowers/specs/<TBD-Group-A-prime>.md). Until Group A-prime ships:

- 6.N.1.1 (per-invocation hunt) is operator-driven manual cadence — no mechanical enforcement
- 6.N.1.2 + 6.N.1.3 are documented requirements only — no mechanical enforcement
- The §6.N-debt entry stays open in this CLAUDE.md until Group A-prime closes it. The constitution checker (scripts/check-constitution.sh) MAY warn but MUST NOT yet hard-fail on missing rehearsals.

Inheritance. Clause 6.N applies recursively to every submodule, every feature, and every new artifact. Submodule constitutions MAY add stricter cadence requirements (e.g., submodules/containers SHOULD bluff-hunt every change to pkg/emulator/ since it is the source of truth for the matrix gate) but MUST NOT relax this clause.

6.N-debt — Pre-push hook enforcement of 6.N.1 clauses 2 + 3 (constitutional debt, 2026-05-05)

The clauses above are the contract. Mechanical enforcement (pre-push hook code that rejects non-compliant commits) is owed via the Group A-prime spec, which is the next brainstorming target after Group A lands. Until Group A-prime ships:

- 6.N.1.2 (per matrix-runner/gate change) is documented requirement; reviewer MUST manually verify Bluff-Audit stamps in commit messages touching the named files
- 6.N.1.3 (per attestation) is documented requirement; reviewer MUST manually verify falsifiability rehearsal evidence accompanies new attestation files

The next phase that touches scripts/check-constitution.sh MUST close 6.N-debt before its commit lands, and the close MUST: (1) parse commit messages for Bluff-Audit: stamps when 6.N.1.2-listed files appear in the diff, (2) check for falsifiability rehearsal evidence (in-attestation or companion file) when 6.N.1.3-listed paths gain new files, (3) update the constitution checker's gate set accordingly.

RESOLVED 2026-05-05 evening via Group A-prime spec at docs/superpowers/specs/2026-05-05-anti-bluff-mandate-reinforcement-group-a-prime-design.md (commit bb2d6a1). Implementation chain: submodules/containers commit (Cleanup API + matrix.go gradle-log persistence) + Lava parent commit (pre-push Check 4 + Check 5 + check-constitution.sh §6.N awareness + scripts/run-emulator-tests.sh refactor). Pre-push hook Checks 4 + 5 active; constitution checker hard-fails on missing §6.N propagation OR missing rehearsal stamps. The §6.N-debt entry stays in this CLAUDE.md as a forensic record but is no longer load-bearing.

6.O — Crashlytics-Resolved Issue Coverage Mandate (added 2026-05-05, ELEVENTH §6.L invocation)

Forensic anchor: within minutes of the first Firebase-instrumented APK distribution (Lava-Android-1.2.3-1023, commit e9de508, distributed 2026-05-05 22:33), Firebase Crashlytics recorded 2 crashes that the operator surfaced via the dashboard. The constitutional concern: Crashlytics issues that are "resolved" by a code fix tend to drift back to broken when the same class of bug recurs months later — because the original incident left no test record, the regression is invisible to the test suite. The operator has now invoked the §6.L mandate for the **ELEVENTH TIME** to make this binding.

Rule. Every Crashlytics-recorded issue (fatal OR non-fatal) that is closed/resolved by any commit **MUST** be covered by **AT LEAST ONE** validation test **AND ONE** Challenge Test that, before the fix, fails with a clear assertion message. There is no exception. "It's an environmental flake" is not an exception. "We can't reproduce it locally" is not an exception — the absence of a reproducer is itself a Sixth Law clause 4 violation, which means the feature is not shippable until the reproducer exists.

1. **Validation test (unit / integration level).** A test in the language of the crashing surface (Kotlin/JUnit for Android, Go for lava-api-go) that reproduces the conditions that triggered the Crashlytics record. Lives at the appropriate `*Test.kt` or `*_test.go` path. Falsifiability rehearsal recorded in commit body (Bluff-Audit stamp).
2. **Challenge Test (end-to-end level).** A Compose UI Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/` (for client-side issues) or `tests/e2e/` test (for lava-api-go issues) that drives the same user-facing path that produced the original crash, asserts on user-visible state, and would fail before the fix.
3. **Crashlytics issue closure log.** Each closed issue **MUST** have an entry in `.lava-ci-evidence/crashlytics-resolved/<YYYY-MM-DD>--<short-slug>.md` containing: the original Crashlytics issue ID (or shareable link from the Console), the stack trace summary, the root-cause analysis, the fix commit SHA, and links to the validation test + Challenge Test that cover the regression. The constitution checker (`scripts/check-constitution.sh`) **MAY** warn if a commit references a Crashlytics fix but no closure log exists; the next phase that touches the checker **SHOULD** make this a hard fail.
4. **Pre-tag verification.** Before any release tag is cut on a commit that closes one or more Crashlytics issues, the operator **MUST** verify (a) the validation tests pass, (b) the Challenge Tests pass on the gating matrix per §6.I, (c) the closure logs exist. `scripts/tag.sh` **MUST** refuse to operate on commits whose `CHANGELOG` mentions Crashlytics fixes without matching closure logs. There is no exception.
5. **Closing the Crashlytics dashboard issue.** Marking an issue "closed" in the Firebase Console requires an interactive Console action (no public REST API). The operator does this **AFTER** the commit lands **AND** the closure log + tests are in place — never before, because the dashboard close-mark loses the ability to track regressions if the test coverage is missing.

6. **Inheritance.** Clause 6.O applies recursively to every submodule, every feature, every new artifact (including lava-api-go and every vasic-digital/* submodule). Submodule constitutions MAY add stricter requirements (e.g. "this submodule's Crashlytics issues MUST also gain a chaos-engineering scenario") but MUST NOT relax this clause.

7. **Programmatic closure-log authoring (added 2026-05-16, Phase 4-C-3).** As of Phase 4-C-3 (docs/plans/2026-05-16-helixqa-go-package-linking-design.md), the lava-api-go/internal/qa/ticket adapter is an AUTHORIZED programmatic path for producing §6.O closure logs. The adapter wraps HelixQA's pkg/ticket.Generator and emits markdown that matches this clause's schema byte-for-byte at the heading level. Operator-written and adapter-generated logs are both compliant; the trailer line Generated by lava-api-go/internal/qa/ticket distinguishes provenance per §6.J's honesty requirement (a generated log claiming "operator-authored" would be a bluff). Existing closure logs under .lava-ci-evidence/crashlytics-resolved/ remain untouched; only NEW logs at the operator's discretion route through the adapter.

The Crashlytics dashboard records what user devices saw. The validation + Challenge tests guarantee the same user-visible defect cannot recur silently. Both surfaces — the Crashlytics close-mark AND the test suite — MUST be in lockstep; otherwise the test suite is a bluff per §6.J/§6.L.

6.P — Distribution Versioning + Changelog Mandate (added 2026-05-05, TWELFTH §6.L invocation)

Forensic anchor: the operator (TWELFTH §6.L invocation, 2026-05-05 23:11): "when distributing new build it must have version code bigger by at least one than the last version code available for download (already distributed). Every distributed build MUST CONTAIN changelog with the details what it includes compared to previous one we have published! Make sure all these points are in Constitution, CLAUDE.MD and AGENTS.MD."

Rule. Every distribution channel — Firebase App Distribution, container registry pushes, releases/ directory snapshots, scripts/tag.sh artifact production — MUST:

1. **Strictly increasing version code per artifact, per channel.** No re-distribution of an already-published versionCode is permitted. A bug fix to an already-distributed build MUST bump the version code (e.g., 1023 → 1024) AND the version name (e.g., 1.2.3 → 1.2.4 for user-visible bug fixes; 1.2.3 → 1.2.3.1 is NOT acceptable since Android version names are 3-component). For the Go API: Code integer monotonic increment + corresponding Name semver bump. For the Ktor proxy: apiVersionCode integer + apiVersionName semver bump in lockstep. ServiceAdvertisement.API_VERSION MUST track apiVersionName (per the §6.A real-binary contract test added in commit 4411def).

2. **Mandatory changelog per distribution.** Every distribute action MUST include a CHANGELOG entry detailing what changed since the previous published build:

- **Source:** CHANGELOG.md at repo root (canonical) + per-version snapshot at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md`.
- **Content:** Bullet list of user-visible changes since the previous published version. Each bullet MUST cite the commit SHA, the high-level change, and (where applicable) the §6.O Crashlytics issue closure log it relates to.
- **Distribution payload:** `scripts/firebase-distribute.sh` MUST inject the changelog into the App Distribution release-notes field via `--release-notes` (truncated to 16KB if needed; full text remains in CHANGELOG.md and the per-version snapshot).
- **Operator-visible:** the testers' Firebase invitation email surface the changelog AND a link to the canonical CHANGELOG.md.

3. **Mechanical version-code bump enforcement.** `scripts/firebase-distribute.sh` MUST refuse to operate when:

- The CURRENT `versionCode` (parsed from `app/build.gradle.kts`) is $\leq$ the LAST distributed `versionCode` for the same channel (read from `.lava-ci-evidence/distribute-changelog/<channel>/last-version`).
- The CHANGELOG.md does not contain an entry for the current `versionName` (`versionCode`) header.
- The per-version snapshot file is missing.

4. **Pre-tag enforcement.** `scripts/tag.sh` MUST refuse to operate when CHANGELOG.md lacks an entry for the tag's version, OR when the per-version snapshot file is missing under `.lava-ci-evidence/distribute-changelog/`. There is no exception. "Operator was busy" is not an exception.

5. **Re-distribution of an existing version is forbidden.** If a build needs re-uploading (e.g., signing-key rotation), the version MUST be bumped first. The exception: the same `versionCode/versionName` MAY be re-uploaded WITHIN A SINGLE distribute session (idempotent retry on transient network failure) but MUST NOT cross session boundaries — the persisted last-version file is the authority.

6. **Inheritance.** Clause 6.P applies recursively to every submodule, every feature, every new artifact (Android client, Ktor proxy, lava-api-go, vasic-digital submodule binaries, future iOS clients, future container images). Submodule constitutions MAY add stricter rules (e.g., "this submodule's artifacts also require a SBOM in the changelog") but MUST NOT relax this clause.

6.Q — Compose Layout Antipattern Guard (added 2026-05-05, THIRTEENTH §6.L invocation)

Forensic anchor: 2026-05-05 23:51 operator report: "Opening Trackers from Settings crashes the app." Root cause: `TrackerSelectorList` Composable used `LazyColumn` nested inside `TrackerSettingsScreen's Column(verticalScroll(rememberScrollState()))`. Compose's measurement protocol throws `IllegalStateException: Vertically scrollable component was measured with an infinite maximum height constraint`

when a lazy layout receives unbounded height from a parent's `verticalScroll`. The crash had been latent in production code since SP-3a Phase 4 (Task 4.12) — multiple Crashlytics-fix cycles passed before any tester actually opened the screen and surfaced it. Closure log at [.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md](#).

Rule. Lava's Compose UI code MUST NOT nest a vertically-scrolling lazy layout (`LazyColumn`, `LazyVerticalGrid`, `LazyVerticalStaggeredGrid`) inside a parent that gives it unbounded vertical space. The forbidden parents include — but are not limited to — `Modifier.verticalScroll`, `Modifier.wrapContentHeight(unbounded = true)`, and any `LinearLayout-with-weight` wrapper. Equivalent rule for horizontal layouts: no `LazyRow` / `LazyHorizontalGrid` / `LazyHorizontalStaggeredGrid` inside `Modifier.horizontalScroll` or unbounded-width parents.

1. **Mechanical enforcement.** A static structural test under `tests/compose-layout/` (or per-feature equivalent — see `feature/tracker_settings/src/test/.../TrackerSelectorListLazyColumnRegressionTest.kt` for the pattern) MUST scan source files and reject:

- Imports of `androidx.compose.foundation.lazy.LazyColumn` (and siblings) in files where the enclosing screen uses `verticalScroll`.
- The opposite case: Composables that import `verticalScroll` AND are invoked from a Composable that already vertical-scrolls.

Per-screen overrides are permitted ONLY when a comment cites the `LazyListState`-or-equivalent pattern that bounds the lazy layout's height (e.g. `Modifier.heightIn(max = X.dp)` on the `LazyColumn`). The override MUST link to a real-stack screenshot test or Compose UI Challenge Test that proves the bounded layout renders.

2. **The list of forbidden patterns is canonical, not exhaustive.** Other measurement-conflict antipatterns also fall under §6.Q:

- `Modifier.fillMaxHeight()` on a child of `verticalScroll` — bounded child of unbounded parent → unexpected layout.
- `Modifier.weight(1f)` on a `Row` whose parent is `horizontalScroll` — weight requires bounded width.
- Multiple `LazyColumns` as siblings inside a non-lazy parent — each gets unbounded height; only the first composes its viewport correctly.

3. **Acceptance gate per layout-touching feature.** Every Compose UI module that adds OR modifies a screen-level Composable MUST gain (a) a structural test asserting no nested-scroll antipattern in the new file, and (b) a Compose UI Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/C<N>_<Name>Test.kt` that drives the rendered screen and asserts on user-visible state. Falsifiability rehearsal recorded per §6.N.

4. **Inheritance.** Clause 6.Q applies recursively to every submodule, every feature, every new artifact. Submodule constitutions MAY add stricter Compose-layout rules (e.g. "no lazy layouts at all" for a constrained-list-only screen) but MUST NOT relax this clause.

6.R — No-Hardcoding Mandate (added 2026-05-06, *FOURTEENTH* §6.L invocation)

Forensic anchor: 2026-05-06 operator directive during Phase 1 brainstorm: "Pay attention that we **MUST NOT** hardcode anything ever!" — restating the spirit of §6.J for an entire class of bluffs (literal values that drift silently from their intended source-of-truth).

Rule. No connection address, port, header field name, credential, key, salt, secret, schedule, algorithm parameter, or domain literal shall appear as a string/int constant in tracked source code (.kt, .java, .go, .gradle, .kts, .xml, .yaml, .yml, .json, .sh). Every such value **MUST** come from a config source: .env (gitignored), generated config class (build-time codegen reading .env), runtime env var, or mounted file.

The placeholder file .env.example (committed) carries dummy values for every variable so a developer cloning the repo knows what to set.

scripts/check-constitution.sh **MUST** grep tracked files for forbidden literal patterns:

- Any IPv4 address outside .env.example and incident docs
- The header name from .env.example's LAVA_AUTH_FIELD_NAME
- Any 36-char UUID outside .env.example
- Hardcoded host:port pairs in HTTP/HTTPS URLs

Pre-push rejects on match. Bluff-Audit stamp required on any commit that adds new config-driven values, demonstrating the no-hardcoding contract test fails when a literal is reintroduced.

Exemptions (test fixtures, incident docs, design specs):

- .env.example — by definition carries placeholders
- .lava-ci-evidence/sixth-law-incidents/*.json — forensic anchors quoting historical literals
- docs/superpowers/specs/*.md, docs/superpowers/plans/*.md — design docs and implementation plans may show example values for clarity (placeholders preferred but examples permitted)
- *_test.go, *Test.kt, *Tests.kt, *Test.java — test fixtures may use synthetic literals, **MUST NOT** use real production values

Enforcement status (updated 2026-05-13, §4.5.10 closure): UUID literals are enforced by scripts/check-constitution.sh → scripts/scan-no-hardcoded-uuid.sh (since 2026-05-06). IPv4 + host:port literals are now enforced by scripts/scan-no-hardcoded-ipv4.sh + scripts/scan-no-hardcoded-hostport.sh (staged-enforcement closure landed 2026-05-13). Schedule and algorithm-parameter literal classes remain staged — algorithm parameters in particular cannot be reliably matched by regex (PBKDF2 iteration counts, AES key sizes, HMAC widths are all just integers in context) and require a code-review gate instead. Open a forensic-anchor entry in .lava-ci-evidence/sixth-law-incidents/ if a literal of any forbidden class ships in production.

Inheritance: applies recursively to every submodule and every new artifact. Submodule constitutions **MAY** add stricter rules but **MUST NOT** relax this clause.

6.S — Continuation Document Maintenance Mandate (added 2026-05-06, FIFTEENTH §6.L invocation)

Forensic anchor: 2026-05-06 operator directive: "during any work we perform, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document MUST BE maintained and it MUST NOT BE out of sync with current work we are doing! If for any reason we stop our work, we MUST BE able to continue any time, with current work, exactly where we have left off and from any CLI agent or any LLM model we chose! Nothing can be broken or faulty in maintained Continuation document!"

Rule. The file docs/CONTINUATION.md is the single-file source-of-truth handoff document for resuming the project's work across any CLI session. It MUST be maintained continuously and MUST NOT drift out of sync with the actual repository state. Every commit that changes any of the following MUST update docs/CONTINUATION.md in the SAME COMMIT:

- A phase completes (mark "DONE" in §1)
- A phase starts (move from §4 NOT STARTED into §1 with progress)
- A new spec or plan lands under docs/superpowers/{specs,plans}/ (record in §1 / §4)
- A submodule pin bumps (update §3 pin index)
- A release artifact ships (Firebase upload, container registry push, tag) — update §0 + §2
- A known issue is discovered (add to §4.5 known-issues subsection)
- A known issue is resolved (move from §4.5 to commit-history reference)
- An operator scope directive lands (update §0 "Last updated" + relevant section)
- A bug is fixed in a way that changes user-visible behavior (record under §4.5 + the closure log path)

The §0 "Last updated" line MUST be the date of the most recent CONTINUATION-touching commit. A Last updated: YYYY-MM-DD value older than the most recent state-changing commit is a §6.S violation.

Mechanical enforcement. scripts/check-constitution.sh MUST verify:

1. docs/CONTINUATION.md exists at repo root
2. Contains a §0 with a "Last updated" line
3. Contains §7 with the operator-pasteable RESUME PROMPT
4. The §6.S clause is present in CLAUDE.md
5. The §6.S inheritance reference appears in every submodules/*/CLAUDE.md and in lava-api-go/CLAUDE.md

Pre-push rejects on any of (1)-(5) failing.

Why this clause exists. The operator has had multiple sessions interrupted (host poweroff incident 2026-04-28, mandate restatements across 14 §6.L invocations, model rate-limits, organic context exhaustion). Without a maintained CONTINUATION.md, every session-restart costs hours re-establishing state. With a maintained CONTINUATION.md + the §7 RESUME PROMPT, ANY CLI agent (any model, any session) can pick up the work in under five minutes. The cost of stale CONTINUATION.md is not abstract — it is exactly the lost-work-time the operator's directive cited.

The bluff this clause prevents. A future agent rationalizing "I'll update CONTINUATION.md at the end of this phase, not commit-by-commit" — and then losing the session before that end. Or "the file is out of date but it's close enough." Or "I'll fix CONTINUATION.md as a follow-up commit." Each of these reads CONTINUATION.md as a maintenance chore. It is not. It is a §6.J primary-on-user-visible-state assertion: the document that says the project is at state X must STILL say so at every commit's HEAD, or the project IS NOT at state X — the document is lying, and the next agent acts on the lie.

Inheritance. Applies recursively to every submodule, every feature, every new artifact (including `lava-api-go`). Submodule constitutions MAY add stricter rules (e.g., "this submodule maintains its own CONTINUATION.md in addition to the parent") but MUST NOT relax this clause. The pre-push hook in the parent enforces presence of the §6.S clause in submodule CLAUDE.md; per-submodule CONTINUATION files are not yet mandated but recommended for any submodule that develops independently.

6.T — Universal Quality Constraints (added 2026-05-06 from ../HelixCode constitution mining)

Forensic anchor: 2026-05-06 operator directive: "Pay attention to rules we may be missing from ../HelixCode Constitution, AGENTS.MD and CLAUDE.MD! We are interested in generic, common sense, universal mandatory constraints which improve stability and quality of the project and all its codebase!"

Adopted from HelixCode: four constitutional anchors that are universally-applicable to any vasic-digital / HelixDevelopment-stack project, not just HelixCode-specific. Each is binding here verbatim.

§6.T.1 — Reproduction-Before-Fix (HelixCode CONST-014).

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script (or equivalent test that exercises the real production path) BEFORE any fix is attempted. Sequence:

1. Write the Challenge / regression test first
2. Run it; confirm it fails (it reproduces the bug)
3. Fix the production code path
4. Run again; confirm it passes
5. Commit the test + fix together with the failure-message-from-step-2 in the Bluff-Audit stamp

A fix without a prior reproducing test is a §6.J spirit violation by construction: there is no falsifiability evidence the fix addresses the root cause vs masking it.

§6.T.2 — Resource Limits for Tests & Challenges (HelixCode CONST-011).

ALL test and challenge execution MUST be strictly limited so it does not starve the operator's host or other workloads on shared infrastructure. Concretely:

- go test invocations: `GOMAXPROCS=2 nice -n 19 ionice -c 3 -p 1` prefix recommended for full-suite runs
- Container test runs: explicit CPU + memory limits via the runtime's `--cpus + --memory` flags

- Gradle test runs: `--max-workers=2` for full-tree invocations on shared hosts
- Long-running matrix-gate runs: must be backgrounded + monitored, never run in the foreground of an interactive session

The 30-40% host-resource ceiling applies; full-resource runs are reserved for dedicated CI machines and require explicit operator authorization.

§6.T.3 — No-Force-Push (HelixCode §12.2 / CONST-043).

No force push, force-with-lease push, history rewrite, branch deletion of `main/` `master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`), signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the vasic-digital stack and every Lava mirror.

The 4-mirror reconciliation pattern Lava already uses (`-s ours` merge when a sibling mirror has diverged, push the merge to all four) is the prescribed alternative when push-conflicts arise.

§6.T.4 — Bugfix Documentation (HelixCode CONST-012).

All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` (created on first fix; appended thereafter) with: root cause analysis, affected files, fix description, link to the verification test/challenge, and the commit SHA that landed the fix. The §6.O Crashlytics-Resolved Issue Coverage Mandate already requires `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` for each Crashlytics-recorded issue; §6.T.4 extends this to ALL bug fixes (Crashlytics-tracked or not, user-reported or self-discovered).

The closure log is the operator-visible audit trail: a bug "fixed" without an entry in `BUGFIXES.md` is silently fixed — the next agent that encounters a regression has no record of why the original fix was made or what the failure mode was.

Inheritance. All four sub-points apply recursively to every submodule, every feature, every new artifact (including `lava-api-go`). Submodule constitutions MAY add stricter rules but MUST NOT relax any of §6.T.1, §6.T.2, §6.T.3, §6.T.4.

6.U — No sudo/su Mandate (added 2026-05-08)

Forensic anchor: 2026-05-08 operator directive: "Any use of `sudo` or `su` is strictly forbidden! This MUST BE part of all submodules and main project Constitution, CLAUDE.MD and AGENTS.MD! Any action that requires use of `su` or `sudo` MUST BE performed within local project or Submodule context with adding local dependencies (as new additional Submodules) which can be fully automatically built and used in local context!"

Rule. Every invocation of `sudo` or `su` (including `sudo -u`, `sudo -E`, `su -`, `su -l`, `sudo su`) is strictly forbidden in any committed script, agent tool call, Makefile, Dockerfile, compose file, test, or any other artifact in this codebase. Operations that require elevated privileges MUST be performed via:

1. **Container-based solutions** using the `vasic-digital/Containers` submodule (rootless Podman/Docker). If a build or runtime environment requires system-

level dependencies, containerize the dependency rather than escalating privileges on the host.

2. **Local project or Submodule dependencies** — add new Submodules that provide the required capability in a fully self-contained, automatically buildable manner.
3. **User-level alternatives** for every privileged operation: user systemd units instead of system-level service files, local port forwarding instead of privileged port binding, user namespaces instead of root-owned resources.

Why this matters. Privilege escalation is:

- **A security risk** — any bug or exploit in code run via sudo gains root access to the host.
- **A reproducibility hazard** — operations that require sudo depend on the operator having sudo rights and a specific sudoers configuration, which is not portable across developer machines or CI runners.
- **A Constitutional violation directly** — the Host Machine Stability Directive (below) already forbids commands that lead to host suspend/poweroff; sudo on an untrusted script has exactly that blast radius.

When you encounter a suggestion requiring sudo/su:

1. STOP immediately.
2. Find a user-level alternative (rootless Podman, user namespaces, local-only ports).
3. If no user-level alternative exists, add a Submodule that provides the capability in a self-contained containerized build.
4. Never modify the suggestion to "just use sudo" or "add the user to sudoers" — those are constitutional violations.

Mechanical enforcement. `scripts/check-constitution.sh` MUST grep tracked files for `sudo`, `sudo\t`, `su`, `su -`, and `su -l` patterns. Pre-push hook rejects on any match. Exemptions: `.env.example` (may show sudo patterns for documentation), incident docs under `.lava-ci-evidence/sixth-law-incidents/` (forensic records may quote historical invocations). `scripts/run-emulator-tests.sh` and every script in the repository MUST NOT contain `sudo` or `su`.

Inheritance. Applies recursively to every submodule, every feature, every new artifact (including `lava-api-go`). Submodule constitutions MAY add stricter rules but MUST NOT relax this clause.

6.X — Container-Submodule Emulator Wiring Mandate (added 2026-05-13)

Forensic anchor: 2026-05-13 operator directive (TWENTY-FIRST §6.L invocation, immediately after F.2 + F.2.6 + §5 follow-ups landed): "when we rely / depend on emulator(s) needed for the testing of the System, make sure we boot up Container running Android emulator in it using ours Containers Submodule. It is supported and it works, it just need proper connecting into the flows. Do this now and continue!"

§6.V (added 2026-05-08) requires emulators to run inside containers managed by the Containers submodule. The 2026-05-13 operator directive is stricter: the emulator process itself MUST execute INSIDE a podman/docker container managed

by submodules/containers/, not be host-direct-launched by Containers-submodule code that merely runs on the host. The Containers submodule supports this — pkg/runtime/ provides the rootless podman/docker abstraction; pkg/emulator/ provides the AVD lifecycle; the wiring that combines them into "Android emulator process inside a Linux container with ADB port forwarded to the host" is the explicit deliverable.

Rule. Every Android emulator instance the project depends on for testing — Challenge Tests, integration testing, `-PdeviceTests=true` runs, real-device verification per §6.G/6.I, release-tagging matrix attestation, ANY emulator-bearing gate — MUST satisfy ALL of:

1. **The emulator process runs INSIDE a container.** `qemu-system-x86_64 / qemu-system-aarch64 / emulator` executes as PID inside a podman or docker container, not as a process on the host. The container's image bundles the Android SDK + AVD images + KVM/HVF passthrough config. adb connects from the host (or from a sibling test container) over the forwarded ADB console port.
2. **The container is launched via the Containers submodule.** submodules/containers/pkg/runtime/ (rootless podman/docker auto-detection) brings the container up. submodules/containers/pkg/emulator/ orchestrates the AVD lifecycle inside it via the runtime's exec interface. No host-side emulator `-avd ...` invocation is permitted for gate runs. The Containers submodule is the SOLE provider of the boot path.
3. **Lava-side glue invokes the Containers CLI.** `scripts/run-emulator-tests.sh` becomes (or remains) thin Lava-domain glue that forwards Lava-specific arguments (AVD list, test class, APK path, evidence directory) to the Containers CLI. The CLI's exit code (0 = matrix passed, 1 = at least one AVD failed, 2 = config error per §6.J anti-bluff posture) is the gate's source of truth.
4. **The container path is the gate.** A test that passes via host-direct emulator launch is NOT gated and MUST NOT appear in `.lava-ci-evidence/<tag>/real-device-verification.md` as a passing row. Workstation iteration during development MAY use host-direct emulators for speed, but the constitutional gate run (tag time, release verification) MUST go through the container path. `scripts/tag.sh` enforces this by refusing matrix attestations whose runner field is not `containers-submodule`.
5. **Pinned Containers commit at the latest.** Per §6.V clause 3 + §6.W, before any emulator gate run, submodules/containers/ MUST be at the latest commit pushed to `vasic-digital/Containers` on GitHub and GitLab. Stale pins for gate runs are constitutional violations.
6. **Multi-architecture + multi-API coverage inside containers.** Per §6.I + §6.V clause 5: minimum API 28, 30, 34, latest stable, multiplied by phone / tablet / (where applicable) TV form factors. Each (API × form-factor) is a distinct container instance; the matrix runner spins them up sequentially (or concurrently per §6.I clause 4 Group B `concurrent==1` gate). Each AVD's attestation row identifies the container image + runtime (runner: `containers-submodule`, runtime: `podman` or `docker`, image: `<sha256>`).

Why this clause exists despite §6.V. §6.V said "the Containers submodule MUST be the canonical emulator runtime" — true at the orchestration level, but it left the door open to a Containers-submodule implementation that host-direct-launched the emulator. The operator's 2026-05-13 directive closes that door: the emulator itself executes inside the container. This eliminates the "tested-against-different-graphics-stack-than-prod" class of bluffs and the "/dev/kvm-on-host vs. /dev/kvm-not-in-container" divergence class.

Mechanical enforcement. `scripts/check-constitution.sh` (after §6.X wiring lands) MUST verify:

- (a) `submodules/containers/pkg/emulator/` has a `Containerized*` or `RuntimeBacked*` lifecycle implementation distinct from the host-direct path.
- (b) `scripts/run-emulator-tests.sh` invokes a Containers-submodule CLI that, when traced, ends in a `podman run` or `docker run` of an emulator-bearing image.
- (c) `.lava-ci-evidence/<tag>/real-device-verification.{md,json}` attestation rows declare `runner: containers-submodule` for every gating row; rows lacking this declaration are rejected by `scripts/tag.sh`.
- (d) Every submodule's `CLAUDE.md`, `AGENTS.md`, and `CONSTITUTION.md` contains a §6.X inheritance reference.
- (e) `lava-api-go/CLAUDE.md`, `lava-api-go/AGENTS.md`, `lava-api-go/CONSTITUTION.md` contain a §6.X inheritance reference.

Failures of (a)–(c) are pre-push rejections once §6.X-debt closes. Failures of (d)/(e) are pre-push rejections immediately.

§6.X-debt — wiring implementation (constitutional debt, 2026-05-13). Per §6.V's transitional allowance, `scripts/run-emulator-tests.sh` currently delegates to `submodules/containers/cmd/emulator-matrix/` which runs the emulator process on the host via `exec.Command`. The container-bound wiring (the emulator PID lives inside a `podman/docker` container managed by `pkg/runtime/`) is OWED to the Containers submodule. Acceptable transitional state until this debt closes: workstation iteration uses host-direct (the matrix runner today); release-tagging is blocked until the container-bound path ships. The next phase that touches release tagging or the emulator-matrix gate MUST close this debt before its tag, and the close MUST: (1) add a `Containerized` (or equivalent) `Emulator` implementation in `submodules/containers/pkg/emulator/` that boots the emulator inside a `pkg/runtime/-managed` container, (2) update `cmd/emulator-matrix/main.go` to accept a `--runner=containerized|host-direct` flag defaulting to `containerized` for `--gating=true` invocations, (3) bake or fetch an Android-SDK-bearing container image documented in `tools/lava-containers/vm-images.json`, (4) update Lava-side `scripts/run-emulator-tests.sh` to forward `--runner=containerized` for gate runs, (5) update `scripts/check-constitution.sh` enforcement clauses (a)–(c) above. The §6.V-debt entry on `darwin/arm64` (recorded at `.lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json`) is a sub-debt under §6.X-debt and closes when the Containers image supports HVF passthrough OR when a Linux `x86_64` self-hosted runner is provisioned per the recorded remediation Option C.

Inheritance. Applies recursively to every submodule (including the Containers submodule itself, whose internal emulator code MUST converge on this rule), every feature, every new artifact (including `lava-api-go` test-bench infrastructure).

Submodule constitutions MAY add stricter rules (e.g., "this submodule's emulator matrix MUST additionally cover Android Automotive emulators inside containers") but MUST NOT relax this clause.

6.V — Container Emulators Mandate (added 2026-05-08)

Forensic anchor: 2026-05-08 operator directive: "For emulator we MUST use Containers submodule (latest codebase to be fetched and pulled) and emulators ran under the Podman or Docker container(s)! Then, all tests to be executed on them!"

Rule. Every Android emulator instance used for Challenge Tests, integration testing, or UI verification MUST run inside a container managed by the `vasic-digital/Containers` submodule (mounted at `submodules/containers/`). The following requirements are mandatory:

1. **Containers submodule is the canonical emulator runtime.** The `submodules/containers/pkg/emulator/` package (or its successor) MUST be the sole provider of emulator lifecycle management. No host-direct emulator invocation (`emulator -avd ...` or `$ANDROID_HOME/emulator/emulator ...`) is permitted for gate runs. Local dev iteration on the host is permitted; the constitutional gate (Challenge Tests, tag verification, real-device verification) MUST go through Containers.
2. **Rootless Podman/Docker only.** Emulator containers MUST run under rootless Podman or rootless Docker. No privileged containers, no `--privileged` flag, no `sudo` to launch the container runtime.
3. **Latest Containers codebase.** Before any emulator gate run, `submodules/containers/` MUST be at the latest available commit from `vasic-digital/Containers` on GitHub and/or GitLab (per §6.W). The pin update MUST be deliberate and committed before the gate run; stale pins are a constitutional violation for gate runs.
4. **All tests execute inside containers.** Every test that depends on an Android emulator (all Compose UI Challenge Tests, all `connectedAndroidTest` runs, all `-PdeviceTests=true` invocations) MUST be executed on an emulator running inside a container. A test that passes on a host-direct emulator but has not been verified inside a container is not gated — the container environment may surface hardware-acceleration, network, or timing differences that produce different outcomes.
5. **Multi-architecture and API-level coverage.** Per §6.I, the emulator matrix inside containers MUST cover minimum API 28, 30, 34, latest stable; minimum phone + tablet + (where TV is touched) TV. Each combination is a distinct container run.

Transitional allowance. Until `submodules/containers/pkg/emulator/` and `submodules/containers/pkg/vm/` fully support all required AVD configurations, the Lava-side `docker-compose.test.yml`, `docker/emulator/Dockerfile`, and `scripts/run-emulator-tests.sh` are the transitional thin glue per the existing §6.K debt note. The transition is complete when every Challenge Test can be executed via `submodules/containers` alone.

Inheritance. Applies recursively to every submodule, every feature, every new artifact (including lava-api-go). Submodule constitutions MAY add stricter rules but MUST NOT relax this clause.

6.W — GitHub + GitLab Only Remote Mandate (added 2026-05-08)

Forensic anchor: 2026-05-08 operator directive: "Do not create any new Git remotes besides GitHub and GitLab under vasic-digital and HelixDevelopment organizations since we have full CLI accesses for GitHub and GitLab only. Other Git providers do not have CLI equivalents!"

Rule. Only two Git providers are permitted for any remote in this repository or any vasic-digital/HelixDevelopment submodule it depends on:

1. **GitHub** — `github.com/vasic-digital/*` and `github.com/HelixDevelopment/*`
2. **GitLab** — `gitlab.com/vasic-digital/*` and `gitlab.com/HelixDevelopment/*`

The following are **explicitly forbidden** as remotes:

- GitFlic (`gitflic.ru`, `gitflic.space`, or any `gitflic.*` domain)
- GitVerse (`gitverse.ru`, `gitverse.space`, or any `gitverse.*` domain)
- Bitbucket, Azure Repos, AWS CodeCommit, Gitee, SourceForge, or any other Git hosting provider

Why this matters. The operator has full CLI access (`gh`, `glab`) only for GitHub and GitLab. Other providers lack CLI equivalents, which means:

- Mirror pushes to GitFlic/GitVerse cannot be automated or verified via CLI tooling
- Repository creation, fork, merge, and issue management are manual-only on unsupported providers
- Constitutional enforcement (pre-push hooks, bluff audits, continuous verification) cannot reach those mirrors
- Mirror divergence at unsupported providers becomes undetectable until tag/verify time

Mandatory consequences.

1. **All existing GitFlic and GitVerse remotes MUST be removed.** Scripts in `Upstreams/` MUST be updated to target only GitHub and GitLab. `CONTINUATION.md` mirror index (§3) MUST be updated to reflect the 2-mirror model.
2. **The Decoupled Reusable Architecture rule's mirror policy (§Mandatory consequences bullet 7) is amended.** The requirement that every vasic-digital submodule mirror to "the same set of upstreams Lava itself mirrors to" is reduced from 4 upstreams (GitHub + GitLab + GitFlic + GitVerse) to 2 upstreams (GitHub + GitLab). Submodules that previously had only `origin` (GitHub) or only `gitlab` must gain the missing GitHub/GitLab remote. Submodules that already have both are compliant.

3. **scripts/tag.sh and all verification scripts MUST check only GitHub + GitLab convergence.** The 4-mirror convergence check (§6.C) is reduced to 2-mirror convergence check (GitHub + GitLab). All attestation evidence, verifications, and release procedures reference only the 2 supported mirrors.
4. **No new project on unsupported providers.** Any new vasic-digital or HelixDevelopment repository MUST be created on GitHub OR GitLab (preferably both), never on any other provider.

Migration. This clause takes effect immediately. On the next commit:

- Remove all GitFlic and GitVerse remote entries from CONTINUATION.md (§3 mirror index)
- Update Upstreams/ scripts to reflect only GitHub and GitLab
- Update scripts/tag.sh convergence check to verify only 2 mirrors
- Update all submodule CLAUDE.md/CONSTITUTION.md/AGENTS.md that reference 4 mirrors

Inheritance. Applies recursively to every submodule, every feature, every new artifact (including lava-api-go). Submodule constitutions MAY add stricter rules (e.g. "this submodule uses GitHub only") but MUST NOT relax this clause.

6.Y — Post-Distribution Version Bump Mandate (added 2026-05-14, TWENTY-FIFTH §6.L invocation)

Forensic anchor: 2026-05-14 operator directive immediately after the 1.2.18-1038 Firebase distribute success: "after every application's distribution (using Firebase Distribut, Google Play Store release, and others we may have in the future) before any new changes have been applied to the product version code MUST BE increased to proper buildable target - API or Application! If it is required and it makes sense version MUST BE increased as well!" This is structurally distinct from §6.P clause 1: §6.P checks at distribute time that the candidate's code > last published; §6.Y enforces proactively that the bump happens FIRST in the new development cycle, so the working tree at every moment between distributes reflects the next releasable identity.

Rule. After every successful distribution of any artifact (Android APK via Firebase App Distribution, Google Play Store release, container image push to a registry, lava-api-go binary release, any future distributable artifact), the FIRST commit in the new development cycle that touches code MUST bump the artifact's versionCode integer (and the per-artifact equivalent — see clause 2). The versionName semver MUST be bumped too when the changes warrant a user-visible version change (per clause 3).

1. **Pre-change bump enforcement.** A commit in the working cycle that comes after a successful distribute MUST EITHER be a version-bump-only commit (no other code changes) OR include the version bump as the first hunk of the same commit. The former is preferred because the bump becomes a discoverable atomic event in git log; the latter is permitted but discouraged because the bump becomes invisible inside a feature change.

2. Per-artifact application:

- **Android client:** bump `versionCode` + `versionName` in `app/build.gradle.kts`.
- **lava-api-go:** bump `Code` + `Name` constants in `internal/version/version.go`.
- **Container images:** bump tag in the Containerfile / Dockerfile reference + any compose image: line that pins by tag.
- **Any future distributable artifact** (iOS app, web app, CLI binary, MCP server, etc.): bump its identity constant declared in the artifact's canonical version file.

3. Version-name bump conditions ("if it is required and it makes sense"):

- User-facing bug fix shipping to testers/users → patch bump (1.2.18 → 1.2.19).
- User-visible new feature → minor bump (1.2.x → 1.3.0).
- Breaking API contract → major bump (1.x.y → 2.0.0).
- Internal refactor / docs only / test-only → `versionCode` bump only; `versionName` held.
- When in doubt: bump `versionName`. Holding it requires a one-line rationale in the bump commit body.

4. **Mechanical enforcement (owed via §6.Y-debt).** The pre-push hook **MUST** detect the case where the working tree's `versionCode` equals the latest entry in `.lava-ci-evidence/distribute-changelog/<channel>/last-version` **AND** the working tree contains code changes beyond pure docs (tracked-file diff includes `.kt/.go/.kts/.gradle/.xml/.json/.yaml/.yml` files that are not docs/CHANGELOG-shaped). On match, the hook rejects with a directive to bump first. Until §6.Y-debt closes (the next phase touching `.github/hooks/pre-push` and `scripts/check-constitution.sh`), reviewers **MUST** manually verify the bump-first ordering on every PR that lands after a distribute.

5. **Inheritance.** Applies recursively to every submodule's distributable artifacts and every new artifact added to the project. Submodule constitutions **MAY** add stricter rules (e.g. "this submodule's binary **MUST** bump within 24h of distribute") but **MUST NOT** relax this clause.

6.Z — *Anti-Bluff Distribute Guard (Pre-Distribute Real-Device Verification Mandate, added 2026-05-14, TWENTY-SIXTH §6.L invocation)*

Forensic anchor (the bluff this clause exists to prevent — recorded against the agent that committed it): 2026-05-14 operator's 26th §6.L invocation, in response to a Crashlytics report on Lava-Android-1.2.19-1039: "Application crashes when we open it on Samsung Galaxy S23 Ultra with Android 16. Check Crashlytics, there should be entries. Fix this and re-distribute! Another point, how come the build wasn't tested? Anti-bluff policy **MUST BE ENFORCED ALWAYS!!!**"

The forensic record: 1.2.19-1039 was distributed with the colored-logo "fix" (commit 32f4cbcf) claiming Compose UI Challenge Tests C24 / C25 / C26 had source-compiled but were "owed at the next §6.X-mounted gate host" — citing the darwin/arm64 LAN-reachability caveat as the blocker. **That citation was a category error.** The §6.X-debt blocks LAN reachability of running APIs (mDNS broadcast through the podman VM cannot reach the LAN), **NOT** the running of

Compose UI Challenge Tests against a connected emulator. The operator HAS Pixel_7_Pro / Pixel_8 / Pixel_9_Pro AVDs configured. The Challenge Tests COULD have run; they SHOULD have run; they DIDN'T run. The 1.2.19 release APK crashed on every cold launch (Crashlytics issue 40a62f97a5c65abb56142b4ca2c37eeb, FATAL, 5 events / 2 users in the first hours, sample on Samsung Galaxy S23 Ultra / Android 16) at `androidx.compose.ui.res.PainterResources_androidKt.loadVectorResource (PainterResources_android.kt:97)` because `R.drawable.ic_lava_logo` was a `<layer-list>` XML which `painterResource()` does NOT support — only `<vector>` OR raster bitmaps. Challenge Test C26 would have caught this on the first emulator boot.

This is the EXACT bluff class §6.J / §6.L exist to prevent: tests pass at one layer (a JVM unit test that read source text) while the user-visible feature is broken (cold-start crash on every device, every API level, every form factor — universal failure). §6.J / §6.L name the class; §6.Z names the mechanical prevention.

Rule. No artifact may be distributed (Firebase App Distribution, Google Play Store release, container image push to a registry, lava-api-go binary release, any future distributable channel) UNLESS the corresponding Compose UI Challenge Tests (or per-artifact equivalent end-to-end tests) have been **EXECUTED** — **not source-compiled, EXECUTED** — against a real device or emulator running the EXACT artifact about to be distributed, AND have **passed**. Distributing a version known to crash on its first user-visible interaction is a constitutional violation by construction.

1. **Pre-distribute test-execution gate (mechanical).** `scripts/firebase-distribute.sh` and any other distribute entry point MUST refuse to operate UNLESS there exists a test-evidence file at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>-test-evidence.{md,json}` that records: (a) which Challenge Tests ran, (b) the AVD or device they ran against (model + Android API level + ABI + screen density), (c) the timestamp and duration of each run, (d) the commit SHA the artifact was built from, AND (e) operator-readable confirmation that all tests passed (the gradle stdout/stderr captured verbatim for grep-ability). The evidence file MUST be the OUTPUT of an actual test execution, not a hand-written claim — tags like `connectedDebugAndroidTest BUILD SUCCESSFUL in N s` MUST appear verbatim in the captured output.
2. **Source-compile is necessary, NEVER sufficient.** A test that successfully compiles is NOT a test that ran. A test that ran once on a developer's machine three weeks ago is NOT a test that ran against this artifact. The evidence file's commit-SHA MUST equal the SHA being distributed; if it doesn't, refuse. The evidence file's timestamp MUST be within 24 hours of the distribute attempt; if older, refuse.
3. **Per-channel mandatory test set:**
 - **Firebase App Distribution (Android APK):** every Compose UI Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/C*Test.kt` whose name is mentioned by the new commits in the cycle since the previous distribute, AT MINIMUM C00 (cold-start survival) + C01 (app launch + tracker selection) + every Cn whose target file appears in the cycle's `git diff`. C26 (colored-logo) is mandatory for any commit touching `core/designsystem/drawables/`. C20–C23 (onboarding flow) are mandatory for any commit touching `feature/onboarding/`.

- **lava-api-go binary release:** every test under tests/contract/, tests/e2e/, tests/parity/ MUST execute against the actual binary about to be released, with real Postgres in podman. JVM/host-only contract tests are necessary, NEVER sufficient.
 - **Container image push:** the image MUST be loaded into a fresh podman/docker container; the container MUST start; the binary's identity MUST report the expected version; the image's healthcheck MUST report healthy within 60 seconds.
4. **Cold-start verification is the load-bearing canary.** Every Android distribute MUST execute at minimum a cold-launch test (C00 — Challenge00CrashSurvivalTest) against a fresh AVD instance with pm clear digital.vasic.lava.client.dev (or .client for release) before launch. The cold-launch crash class — LavaApplication.onCreate failures, Hilt DI resolution failures, MainActivity.setContent Composable initialization failures, including the painterResource failure that produced this clause's forensic anchor — is the highest-impact-per-event category and the cheapest to detect mechanically. Skipping the cold-launch test for ANY reason (build was rushed, emulator was slow, "the change is small") is a §6.Z violation.
 5. **No "this is just a small change" exception.** The forensic anchor proves this empirically: the 1.2.19 commit changed ONE drawable resource reference (one line in LavaIcons.kt from ic_notification to ic_lava_logo) plus added 12 PNG asset files. By any naive risk-rating it was "low risk; just a resource swap." It crashed every device on cold launch. The lesson: there is no such thing as a "low-risk" Android resource change in a Compose codebase that uses painterResource() — every drawable swap MUST be verified by C26-equivalent execution against the artifact.
 6. **Distribute-faulty-version forbidden.** Distributing a version known to crash on its first user-visible interaction is a constitutional violation by construction. The pre-distribute test gate makes this impossible IF the gate runs; the gate is mandatory; bypassing the gate (e.g., manually invoking firebase appdistribution:distribute outside firebase-distribute.sh, or via any --bypass-tests flag — which MUST NOT exist) is itself a §6.Z violation.
 7. **Mechanical enforcement (owed via §6.Z-debt).** scripts/firebase-distribute.sh MUST gain a Phase 1 Gate 6 that verifies the test-evidence file exists, has the matching commit SHA, has a timestamp within 24h, and reports all-pass. The pre-push hook MUST gain a check that any commit advancing last-version is matched by a same-SHA test-evidence file. Until §6.Z-debt closes, the operator + reviewer MUST manually verify the evidence file's existence + content on every distribute. The next phase that touches scripts/firebase-distribute.sh MUST close §6.Z-debt; no distribute is acceptable in the meantime without operator-verified test execution.
 8. **Closure protocol when this rule has been violated.** When a distribute landed without the required test execution (as is the case for 1.2.19-1039):
 - The retroactive test execution MUST happen BEFORE the next distribute attempt.
 - The Crashlytics issue closure log per §6.O MUST cite this §6.Z violation as the root cause and document the new test-execution evidence.

- The next distribute's evidence file MUST include compensating-execution-for-prior-skipped-version: <version>-<code> so the audit trail is unambiguous.

9. **Inheritance.** Applies recursively to every submodule's distributable artifacts and every new artifact added to the project (including all vasic-digital/* submodules and the lava-api-go service). Submodule constitutions MAY add stricter rules (e.g. "this submodule's binary MUST also be tested on Linux x86_64 before distribute") but MUST NOT relax this clause.

6.Z-debt — Mechanical enforcement of the pre-distribute test-evidence gate (constitutional debt, 2026-05-14)

The clause above is the contract. Mechanical enforcement (Phase 1 Gate 6 in scripts/firebase-distribute.sh + pre-push hook check + companion tests/firebase/ hermetic test exercising the new gate) is OWED. Until close:

- 6.Z.1 (test-evidence file) is operator-verified manually; the next distribute attempt without an evidence file is REJECTED by reviewer/operator, not by the script.
- 6.Z.7 (mechanical enforcement) is documented requirement only; the script does NOT yet hard-fail on a missing evidence file.

The next phase that touches scripts/firebase-distribute.sh MUST close §6.Z-debt: (1) parse the expected evidence-file path from versionName + versionCode + channel; (2) verify file exists; (3) verify commit-SHA in the file matches git rev-parse HEAD; (4) verify timestamp within 24h; (5) verify file contains BUILD SUCCESSFUL for at least the mandatory test set per §6.Z.3; (6) reject with the §6.Z directive if any check fails; (7) add a hermetic test under tests/firebase/ exercising the new gate (positive case: present + valid; negative cases: missing / stale-SHA / stale-timestamp / no-BUILD-SUCCESSFUL).

6.AA — Two-Stage Distribute Mandate (Debug First, Release After Verification, added 2026-05-14)

Forensic anchor: 2026-05-14 operator directive immediately after the §6.Z forensic-anchor crash on Lava-Android-1.2.19-1039: "for purposes like this one we shall distribute via Firebase DEV / DEBUG version only. Once we try it, you continue and once all verified you distribute RELEASE too! Let's keep this flow and add these notes / rules into the Constitution, AGENTS.MD and CLAUDE.MD!" The directive is born of the same incident: the 1.2.19-1039 distribute pushed both debug + release in a single sweep; the release APK crashed every cold launch via R8-aware painterResource() rejection of <layer-list>. Staging debug → verify → release would have surfaced the failure on the smaller blast radius first AND would have separately exposed any R8-only differences (the canonical class of release-only failures: kept-rules omissions, reflection misses, resource-resolver corner cases that R8 trims away differently from the debug build).

Rule. When an artifact has both a debug and a release variant (Android APK is the canonical example; analogous staging applies to any future artifact with dev-vs-prod build types), distribute MUST happen in TWO STAGES with operator-confirmed verification between them.

1. **Stage 1 — Debug variant only.** `scripts/firebase-distribute.sh --debug-only` distributes ONLY the debug APK to the `.dev`-suffixed application ID's App Distribution channel. The §6.Z evidence file at this stage records debug-only test execution + operator (or designated tester) real-device verification of the debug APK installed via Firebase invite (NOT just the side-loaded developer build).
2. **Stage 2 — Release variant only.** ONLY AFTER the operator confirms in writing that the **Firebase-distributed debug variant** works correctly on the failure-surface device class, `scripts/firebase-distribute.sh --release-only` distributes the release APK. The §6.Z evidence file is APPENDED with a release-stage section recording the release-APK test execution + operator real-device verification on the release variant. Same SHA as stage 1 (no code change between stages); $\leq 24\text{h}$ since stage 1 OR re-bump the version per §6.Y if more time has elapsed.
3. **No combined distribute permitted by default.** Running `scripts/firebase-distribute.sh` with no flags MUST default to `--debug-only` mode. The combined `--debug-and-release` flag (or any equivalent) MUST require explicit per-cycle operator authorization recorded in the §6.Z evidence file's `combined-distribute-authorization:` field. Pre-push hooks + the script itself reject release-only invocations not preceded by a documented debug-stage evidence entry.
4. **The R8 / minification surprise class** is the load-bearing reason for the staging. The §6.Z forensic anchor (`painterResource()` layer-list rejection) was reported on the release variant first because R8 / resource-shrinking interacts with drawable-resource resolution in ways the debug build does not. The two-stage pattern surfaces non-R8 bugs (composition errors, Hilt wiring breaks, navigation typos) at the cheaper debug-only blast radius AND isolates R8-specific failures to the release stage where they can be fixed before more user impact.
5. **Per-channel escalation.** Two-stage applies to every Android distribute channel (Firebase App Distribution today, Google Play Store internal-track / open-track, future channels). For non-Android artifacts: equivalent staging when there is a dev / staging variant distinct from the production variant (lava-api-go has the prod compose vs. the `docker-compose.dev.yml` dev compose; container image push to a staging registry is stage 1, push to the production registry is stage 2 after stage-1 soak verification).
6. **Mechanical enforcement (owed via §6.AA-debt).** `scripts/firebase-distribute.sh` MUST gain default-debug-only mode (today the default is debug + release combined; that default flips). The script MUST refuse `--release-only` invocations when no companion debug-stage evidence section exists in the §6.Z evidence file for the same SHA. Pre-push hook MUST reject commits that advance `last-version` for the release-app-id channel without a matching debug-app-id `last-version` advance for the same SHA. Until §6.AA-debt closes, the operator MUST manually request the release distribute as a separate explicit step after debug verification.

7. **Inheritance.** Applies recursively to every submodule's distributable artifacts and every new artifact added to the project. Submodule constitutions MAY add stricter rules (e.g. "this submodule's debug + release stages each get a 24-hour soak window") but MUST NOT relax this clause.

6.AB — Anti-Bluff Test-Suite Reinforcement (27th §6.L invocation, added 2026-05-14)

Forensic anchor: 2026-05-14 operator's 27th §6.L invocation, immediately after observing two real defects on the Firebase-distributed Lava-Android-1.2.20-1040 debug APK that all existing tests + Challenges had passed against: (a) the Welcome screen renders the brand mark as a white placeholder (Compose Icon composable applies LocalContentColor as tint — designed for monochrome glyphs — which strips the colored launcher PNG to solid white), AND (b) the onboarding "completion" gate fires on first-screen back-press (OnboardingViewModel.onBackStep() from Welcome posts Finish side effect; MainActivity writes setOnboardingComplete(true) on Finish; result: pressing back on the first screen with zero providers configured marks onboarding as "complete" and dumps the user into a half-functional home screen). Operator's verbatim restatement: "Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

These two defects are both §6.J spirit failures of a NEW class — not "test wasn't executed" (which §6.Z catches), but **"test executed and passed against a feature that doesn't actually work for the user"**. The white-placeholder and gate-bypass both happen on a freshly-built APK that the operator installed via Firebase, on the canonical S23 Ultra device, with C26 having been written + source-compiled — but C26 only asserts on RGB variance across the entire screen (which catches "all white" only if a wide enough region is sampled, and can be defeated by a small monochrome icon on a colored background) and there is no test of the onboarding-gate-completion logic at all. The tests were not executed (§6.Z), AND when written, they didn't exercise the user-visible behavior with sufficient discrimination (this clause's class).

Rule. Every existing test + Challenge in the project MUST be auditable for the anti-bluff property "would this test fail if the user-visible feature broke in the way a real user would notice?" If the answer is "not necessarily" or "only for a specific failure mode within the larger feature", the test is incomplete — additional assertions or additional tests are required to cover the discrimination gap.

1. **Per-feature anti-bluff completeness checklist** mandatory for every feature whose user-facing surface ships:
 - **Rendering correctness, not just rendering presence.** A test that asserts "node displayed" passes for a white-on-white pixel rendering. A test of a colored brand asset MUST sample the rendered region and assert dominant-color matches expected (e.g., the Lava red is in the expected hue range), not just RGB-variance > N.

- **State-machine completeness, not just happy-path traversal.** A test that drives Welcome → Get Started → Configure → Continue → Summary → Start Exploring covers the happy path; it does NOT cover "user presses back on Welcome", "user reaches Summary without configuring anything", "user kills the app mid-Configure". Each user-reachable state transition MUST have a matching test (positive: asserting the transition happens correctly; negative: asserting forbidden transitions are rejected).
 - **Gating logic, not just absence of crash.** Tests that "the app didn't crash" are necessary but never sufficient. Tests for completion gates (onboarding-complete, login-complete, payment-complete, etc.) MUST verify the gate fires only on the actual completion criterion (e.g., onboarding-complete only when ≥ 1 provider was probed successfully, NOT when user back-pressed out of the Welcome screen).
2. **Bluff-hunt cadence escalation.** §6.N.1's per-invocation hunt is augmented by a per-defect hunt: every Crashlytics-or-operator-reported defect that wasn't caught by an existing test MUST trigger a 5-file bluff-hunt of tests adjacent to the defect's surface (e.g., this 2026-05-14 onboarding-gate defect triggers a hunt of 5 files under feature/onboarding/src/test/ + app/src/androidTest/.../challenges/Cn[Onboarding]Test.kt). Output recorded under .lava-ci-evidence/bluff-hunt/<date>-defect-driven-<slug>.json.
 3. **The §6.J-spirit discrimination test for every Challenge Test.** Before a Challenge Test is declared "covers the feature", the author MUST construct a deliberately-broken-but-non-crashing version of the production code (e.g., the gate fires unconditionally; the icon is replaced with a monochrome placeholder; the back-press goes to the wrong destination) and confirm the Challenge Test FAILS with a clear assertion message. If the Challenge Test passes against the deliberate non-crashing break, the test does not have sufficient discrimination — extend its assertions until it does. This is §6.J clause 2 (provably falsifiable on real defects) restated with an emphasis on NON-CRASHING failure modes specifically (the 1.2.19 crash was the easy case; the 1.2.20 white-icon + gate-bypass are the hard cases that this clause exists to prevent).
 4. **Inheritance.** Applies recursively to every submodule, every feature, every new artifact added to the project. Submodule constitutions MAY add stricter rules (e.g. "every feature MUST have a discrimination test paired with each Challenge Test commit-by-commit") but MUST NOT relax this clause.

6.AC — Comprehensive Non-Fatal Telemetry Mandate (added 2026-05-14)

Forensic anchor: 2026-05-14 operator directive after 1.2.21-1041's stage-1 debug verification surfaced "issues" the operator wanted captured continuously: "Add comprehensive Crashlytics non-fatals recording all over the apps and API so we can track in the background all warnings, issues and unexpected situations!" The §6.AB completeness checklist (rendering correctness, state-machine completeness, gating logic) catches discrimination gaps in tests; §6.AC is the runtime equivalent — every error path that fires in production MUST surface to telemetry so the OPERATOR can see what's actually breaking on real users' devices, not just what existing tests claim is fine.

Rule. Every catch / error / fallback / unexpected-state path on every distributable artifact MUST record a non-fatal telemetry event with sufficient context to triage the failure remotely.

1. **Android client — Firebase Crashlytics non-fatal channel.** Every try / catch, runCatching { ... }.getOrElse { ... }, runCatching { ... }.onFailure { ... }, every if (error != null) ... branch, every null-fallback in production code MUST call analytics.recordNonFatal(throwable, context) (or analytics.recordWarning(message, context) for non-throwable warnings) before / instead of the silent fallback. The AnalyticsTracker interface in lava.common.analytics is the canonical entry point; under the hood it routes to `FirebaseCrashlytics.getInstance().recordException() + .log() + .setCustomKeys()`.
2. **lava-api-go — observability + Crashlytics REST bridge.** Every error returned by an HTTP handler, every middleware error, every cache miss / fallback path, every database operation failure, every external service call (rutracker scrape, etc.) failure MUST call the new `observability.RecordNonFatal(ctx, err, attrs)` helper — which (a) emits a structured WARNING/ERROR log via the existing OTLP pipeline, AND (b) optionally posts to Firebase Crashlytics via REST when `LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true` so server-side and client-side telemetry land in the same dashboard.
3. **Mandatory context attributes per non-fatal event:**
 - feature / module (which subsystem the error came from)
 - operation (the user-visible action that triggered it)
 - error_class (the throwable's class name OR a stable error code for non-throwable warnings)
 - error_message (truncated to 1024 chars; **NEVER include credentials per §6.H**)
 - For Android: screen (which Compose surface was active)
 - For Go: endpoint + request_id + tracker_id (where applicable)
4. **What MUST NOT be recorded (§6.H + privacy):** real usernames / passwords / tokens / cookies / signed URLs / personal identifiers. Use redacted forms (<username:redacted>, auth-token-first-4-chars, etc.) when the error message would otherwise contain them. The `recordNonFatal` helper SHOULD apply automatic redaction to known sensitive attribute names (password, token, secret, api_key).
5. **Mechanical enforcement (owed via §6.AC-debt).** A lint rule (Detekt for Android, Go vet for the API) MUST flag try blocks whose catch body lacks a `recordNonFatal` / `recordWarning` call OR is preceded by a `// no-telemetry: <reason>` comment that explicitly opts out. The pre-push hook MUST run the lint and reject violating commits. Until §6.AC-debt closes, reviewers manually verify every commit that adds a try / catch includes the telemetry call.

6. **Inheritance.** Applies recursively to every submodule's distributable artifacts and every new artifact added to the project. Submodule constitutions MAY add stricter rules (e.g. "this submodule's library functions MUST also record non-fatals when an exception escapes into the consumer") but MUST NOT relax this clause.

6.AC-debt — Lint-rule enforcement of mandatory non-fatal recording (constitutional debt, 2026-05-14)

The clause above is the contract. Mechanical enforcement (Detekt rule for Kotlin, Go-vet check for Go, pre-push integration) is OWED. Until close, reviewers manually verify telemetry coverage on every error-handling commit. The next phase that adds a Detekt rule + Go-vet pass MUST close §6.AC-debt: (1) Detekt rule under `config/detekt/lava-non-fatal-required.kts` flagging catch (e: ...) { ... } blocks lacking `recordNonFatal / recordWarning / // no-telemetry: opt-out`; (2) Go-vet equivalent rule under `lava-api-go/scripts/check-non-fatal-coverage.sh`; (3) pre-push hook invocation; (4) hermetic test under `tests/non-fatal-coverage/` exercising both rules.

6.AA-debt — Mechanical enforcement of the two-stage default (constitutional debt, 2026-05-14)

The clause above is the contract. Mechanical enforcement (default-debug-only mode + refusal of unsupported `--release-only` + paired `last-version` per-channel pre-push check + companion `tests/firebase/` hermetic test) is OWED. Until close:

- 6.AA.1 (debug-first staging) is operator-driven; the operator manually requests `--debug-only` then `--release-only` as separate steps.
- 6.AA.3 (default-debug-only) is documented requirement only; today `firebase-distribute.sh` defaults to debug + release combined.
- 6.AA.6 (mechanical enforcement) is documented requirement only.

The next phase that touches `scripts/firebase-distribute.sh` MUST close §6.AA-debt: (1) flip the default mode to `--debug-only`; (2) require explicit `--release-only` AND companion debug-stage evidence in the §6.Z evidence file; (3) split the `last-version` pointer into `last-version-debug` and `last-version-release` per channel; (4) add a hermetic test under `tests/firebase/` exercising both stages + the rejection of out-of-order release distribute.

6.Y-debt — Pre-push hook enforcement (constitutional debt, 2026-05-14)

The clause above is the contract. Mechanical enforcement (pre-push hook code that rejects non-compliant commits) is owed. Until it ships:

- 6.Y.1 (pre-change bump enforcement) is documented requirement; reviewers manually verify bump-first ordering on every commit that lands after a distribute.
- 6.Y.4 (mechanical enforcement) is documented requirement only; pre-push hook does NOT yet hard-fail on missing bump.

The next phase that touches `.githooks/pre-push` AND `scripts/check-constitution.sh` MUST close this debt before its commit lands. The close MUST: (1) parse `.lava-ci-evidence/distribute-changelog/<channel>/last-version` and compare to working-tree `versionCode`; (2) detect non-doc tracked-file diff (kt/go/kts/gradle/xml/json/yaml/yml outside docs/, `CHANGELOG.md`, `.lava-ci-evidence/, *.md`); (3) reject with the §6.Y directive when both conditions match; (4) add a hermetic test under `tests/pre-push/` exercising the new check.

6.AD — HelixConstitution Inheritance (added 2026-05-14, 29th §6.L cycle)

Forensic anchor: 2026-05-14 operator's standing 10-step directive: incorporate `git@github.com:HelixDevelopment/HelixConstitution.git` as `./constitution` submodule with recursive inheritance pointers from the parent project's `CLAUDE.md` / `AGENTS.md` / `Constitution-equivalent`. **Reaffirmed 2026-05-15 (30th §6.L invocation):** "Pay attention that we now use and incorporate fully the HelixConstitution Submodule responsible for root definitions of the Constitution, `CLAUDE.MD` and `AGENTS.MD` which are inherited further!" — i.e. the constitution submodule is the SOURCE OF TRUTH for universal rules; Lava's `CLAUDE.md` / `AGENTS.md` inherit from it, never override-to-weaken; per-scope docs (54 in-tree files: 16×3 submodule docs + 3 lava-api-go + 3 core/app/feature) carry the inheritance pointer-block; future Helix-universal rule changes propagate via submodule pin bumps without Lava-side per-rule edits. The directive carries hard constraints: NO credentials/secrets/keys ported into the submodule; NO `--no-verify/--no-gpg-sign/--force` without explicit per-operation operator authorization; NO destructive operations without a hardlinked `.git` backup first; NO guessing language (likely/probably/maybe/seems/appears); use the Containers submodule for emulators. The submodule's own `CLAUDE.md` prescribes the inheritance pattern (`## INHERITED FROM constitution/CLAUDE.md` block at the top of the consuming project's `CLAUDE.md`, OR `@constitution/CLAUDE.md` import).

Rule. Lava's root `CLAUDE.md` AND `AGENTS.md` carry the inheritance pointer-block at the top (above all other content). The HelixConstitution submodule lives at `./constitution/` (pin frozen by default per the Decoupled Reusable Architecture rule; updates are deliberate). All rules in `constitution/CLAUDE.md` + `constitution/AGENTS.md` + `constitution/Constitution.md` apply unconditionally to Lava; Lava's existing rules (Anti-Bluff Pact §6.A–§6.AC, Local-Only CI/CD, §6.W mirror policy, §6.M host-stability, §6.R no-hardcoding, §6.S CONTINUATION mandate, §6.T universal-quality, §6.U no-sudo) extend them — they MUST NOT weaken any inherited rule. Where Lava's rule is at least as strict as the inherited rule, the Lava rule wins; where the inherited rule is stricter, the inherited rule wins.

1. **§6.W carve-out for the submodule itself.** Per §6.W, Lava + every Lava-owned `vasic-digital/*` submodule mirrors only to GitHub + GitLab (CLI parity). HelixConstitution is owned by the HelixDevelopment org and ships its own `install_upstreams.sh` configuring 4 upstreams (GitHub + GitLab + GitFlic + GitVerse). The 4-upstream rule applies WITHIN that submodule's `git-dir`, NOT to the parent Lava project. `scripts/check-constitution.sh` MUST treat `./constitution/` as `vasic-digital-equivalent` for inheritance-presence checks but as `HelixDevelopment-domain` for mirror-count checks.

2. **Project commit wrapper.** HelixConstitution mandates `scripts/commit_all.sh` as the project-official commit + push wrapper (HelixConstitution CLAUDE.md "MANDATORY COMMIT & PUSH CONSTRAINTS" + universal §11.4.22). Lava's wrapper lives at `scripts/commit_all.sh`, scoped to the §6.W 2-mirror set. Direct `git add/commit/push` on the parent repo remains permitted for now (transitional carve-out — §6.AD-debt) until the wrapper covers all flows the constitutional gates need (multi-mirror submodule push, evidence-file regeneration, doc-sync per §11.4.22, §11.4.18 script-doc sync, etc.).

3. **Item-tracking gates (HelixConstitution §11.4.15 + §11.4.16 + §11.4.19 + §11.4.21 + §11.4.33 + §11.4.34).** HelixConstitution mandates Issues / Issues_Summary / Fixed / Fixed_Summary trackers with Status + Type columns + closed-set vocabularies; §11.4.33 adds type-aware closure-status vocabulary requirements (Bug→Fixed, Feature→Implemented, Task→Completed, etc.); §11.4.34 adds reopened-source attribution (each re-open MUST cite WHY + WHO + WHEN + WHICH-INCIDENT). Lava does not have these trackers today; functionally equivalent state lives in `docs/CONTINUATION.md` (§6.S) + `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` (§6.O closure logs, which include root-cause + fix-commit-SHA + validation-test references = §11.4.34 reopened-source-attribution semantics) + `.lava-ci-evidence/sixth-law-incidents/<date>-<slug>.json` (forensic anchors, which include candidate-causes + ELIMINATED entries + PENDING_FORENSICS markers = §11.4.33 closure-status-vocab semantics). Per Path B of the constitution-compliance plan Phase 9 (2026-05-15): §11.4.33 + §11.4.34 are formally mapped to the existing Lava convention rather than requiring a parallel Issues.md / Fixed.md tracker; the mapping IS the equivalence and is binding. The HelixConstitution gates CM-ITEM-STATUS-TRACKING, CM-ITEM-TYPE-TRACKING, CM-FIXED-COLUMN-ALIGNMENT, CM-ITEM-OPERATOR-BLOCKED-DETAILS, CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT are mapped to the equivalent Lava artifacts via §6.AD-debt (rolling close); CM-CLOSURE-STATUS-VOCAB-COMPLIANCE (§11.4.33) + CM-REOPENED-SOURCE-ATTRIBUTION (§11.4.34) are equivalence-mapped per this clause and do NOT require separate scanners. **Classification:** project-specific (the equivalence-mapping decision is Lava-specific; the trackers + status-vocab + reopened-source mandates are universal per HelixConstitution).

- §11.4.35 (Canonical-Root Inheritance Clarity) + §11.4.36 (Mandatory install_upstreams) are NOT equivalence-mapped — they have a dedicated Lava-side mechanical gate (Phase 8 of constitution-compliance plan, 2026-05-15): `scripts/check-canonical-root-and-upstreams.sh` + `tests/check-constitution/test_canonical_root_and_upstreams.sh`, wrapped in advisory mode by `scripts/verify-all-constitution-rules.sh` until §11.4.36's 10 missing-install_upstreams violations close (per Phase 8-debt task).

4. **Build-resource stats tracker (HelixConstitution §11.4.24).** Lava builds routinely exceed 1 minute (full `./gradlew :app:assembleRelease` + make build for lava-api-go). The §11.4.24 telemetry sampler + TSV registry + Stats.{md,html,pdf} triple are NOT yet implemented in Lava — CM-BUILD-RESOURCE-STATS-TRACKER gate is paper-only. §6.AD-debt tracks the wiring.

5. **Universal-vs-project classification (HelixConstitution §11.4.17).** Every new rule added to Lava's CLAUDE.md / AGENTS.md / submodule CLAUDE.md from this point forward **MUST** carry a **Classification:** line (universal vs project-specific). Already-existing Lava-only rules (§6.A–§6.AC) are grandfathered as project-specific by retroactive default.
6. **No-guessing mandate (HelixConstitution §11.4.6).** Forbidden vocabulary in tests / gates / status reports / closure narratives / commit messages: likely, probably, maybe, might, possibly, presumably, seems, appears to, guess, seemingly, apparently, perhaps, supposedly, conjectured. Either prove the cause with captured evidence and state it as fact, OR explicitly mark **UNCONFIRMED:** / **UNKNOWN:** / **PENDING_FORENSICS:** with a tracked-task ID. Lava's existing forensic-anchor discipline (`.lava-ci-evidence/sixth-law-incidents/`) is the captured-evidence channel. `scripts/check-constitution.sh` **MUST** grow a grep gate for the forbidden vocabulary in tracked status / closure / commit-template files; pre-push rejects on match. §6.AD-debt tracks this.
7. **§9 absolute-data-safety (HelixConstitution).** Already aligned with the existing Forbidden Command List + Host Machine Stability Directive in this CLAUDE.md. The hardlinked `.git` backup discipline (`cp -al .git <backup>/repo.git.mirror`) **MUST** be performed before any history rewrite / force-push / branch deletion / submodule de-init / object pruning. The 2026-05-14 HelixConstitution incorporation itself was preceded by `.git-backup-pre-helixconstitution-20260514-211450/` per this rule.
8. **Inheritance.** Applies recursively to every Lava submodule + `lava-api-go`. Each submodule's CLAUDE.md / AGENTS.md **MUST** add an `## INHERITED FROM constitution/CLAUDE.md` pointer-block referring to the constitution submodule mounted at the parent's `./constitution/` (path resolution via the submodule's `find_constitution.sh` helper). Submodule constitutions **MAY** add stricter rules but **MUST NOT** relax any inherited rule. Per-submodule propagation is **OWED** via §6.AD-debt; until close, the parent CLAUDE.md's inheritance pointer is binding by transitive reference.

Classification: universal (the inheritance pattern itself is reusable across any vasic-digital project consuming HelixConstitution).

6.AD-debt — HelixConstitution gate-wiring + per-submodule pointer propagation (constitutional debt, 2026-05-14)

The clause above is the contract. The OWED implementation work:

1. **Per-submodule + per-scoped-CLAUDE.md propagation.** Every `submodules/*/CLAUDE.md`, every `submodules/*/AGENTS.md`, `lava-api-go/CLAUDE.md`, `lava-api-go/AGENTS.md`, `lava-api-go/CONSTITUTION.md`, `core/CLAUDE.md`, `app/CLAUDE.md`, `feature/CLAUDE.md` **MUST** carry a `## INHERITED FROM constitution/CLAUDE.md` pointer-block. Today only the root-level CLAUDE.md + AGENTS.md carry it. Closure: a propagation pass adds the block to all per-scope docs (estimated 50+ files). The block content is constant; only the relative path to `constitution/CLAUDE.md` from the consuming doc differs (`./constitution/`, `../constitution/`, `../../constitution/`, etc.) — `find_constitution.sh` is the canonical resolver but markdown links **MAY** hard-code the relative path for readability.

2. **HelixConstitution CM-* gates wiring.** CM-COMMIT-DOCS-EXISTS, CM-FIXED-COLUMN-ALIGNMENT, CM-SCRIPT-DOCS-SYNC, CM-BUILD-RESOURCE-STATS-TRACKER, CM-ITEM-STATUS-TRACKING, CM-ITEM-TYPE-TRACKING, CM-ITEM-OPERATOR-BLOCKED-DETAILS, CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT, CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION, CM-SUBAGENT-DELEGATION-AUDIT — each is a paper-only gate today in Lava. Closure: each gate gets a Lava-side implementation in `scripts/check-constitution.sh` (or a sibling script invoked from it) + a hermetic test that exercises the gate's positive and falsifiability paths per §6.A real-binary contract test discipline.
3. **`scripts/commit_all.sh` doc + extended capability.** Per HelixConstitution §11.4.18 every script MUST carry an in-source documentation block AND an external user guide under `docs/scripts/<script-name>.md`. The external guide for `commit_all.sh` is OWED. Functionality extension: the wrapper currently handles parent-repo commit + push to GitHub + GitLab; HelixConstitution §11.4.22 also requires lightweight doc-only commit path + auto-export-regeneration pipeline. These are OWED.
4. **No-guessing-vocabulary grep gate in `scripts/check-constitution.sh`.** Per §6.AD.6 + HelixConstitution §11.4.6. Forbidden words: likely, probably, maybe, might, possibly, presumably, seems, appears, guess, seemingly, apparently, perhaps, supposedly, conjectured. Scan tracked *.md files under `.lava-ci-evidence/sixth-law-incidents/`, `.lava-ci-evidence/crashlytics-resolved/`, `docs/`, `CHANGELOG.md`, plus commit-message templates. False-positive mitigation: explicit UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: lead allows the otherwise-forbidden word to pass.
5. **Build-resource stats tracker (HelixConstitution §11.4.24).** Lava-side host-side sampler + TSV registry + Stats.{md,html,pdf} triple are OWED. Sampler MUST stay under 50 MB RSS / 5% CPU per the §11.4.24 Heisenberg-class observer constraint.
6. **Classification: line on commit messages.** Per HelixConstitution §11.4.17 every new-rule commit MUST carry a Classification: line. Already enforced in this commit's body for §6.AD; pre-push hook check + check-constitution.sh enforcement OWED.
7. **HelixConstitution §6.W applicability boundary.** The submodule's own `install_upstreams.sh` configures 4 upstreams; Lava's `scripts/check-constitution.sh` MUST treat `./constitution/` as a special case in the §6.W mirror-count check (allowed: 4 upstreams within the submodule; required: GitHub + GitLab pointing at the HelixDevelopment org). OWED.
8. **The whole CM-* gate set as a documented operator-readable surface.** A `docs/helix-constitution-gates.md` index listing every CM-* gate + its Lava-side implementation status (✅ wired / ⚠️ paper-only / ❌ blocked-by) keeps the inheritance auditable. OWED.

Until these close, HelixConstitution inheritance is operator-and-reviewer-verified manually for every commit. The next phase MUST close (1) at minimum (per-submodule propagation) before any release tag ships from a commit that touches the inheritance surface; tag scripts MUST refuse otherwise.

6.AE — Comprehensive Challenge Coverage + Container/QEMU Matrix Mandate (added 2026-05-15, 31st §6.L invocation)

Forensic anchor: 2026-05-15 operator's 31st §6.L invocation: "Create all required Challenges and run them all on Emulators executed in Container or Qemu! Make sure everything is delegated via Containers Submodule! [...] All tests and Challenges

MUST BE executed against variety of major Android versions, screen sizes and configurations (all ran inside Container or Qemu). Document everything and make sure that anti bluff proofs are obtained for every single thing!"

Rule. Every Lava feature module + every distinct user-visible flow MUST have at least one Compose UI Challenge Test that traverses its real production code path end-to-end. Every gate run of those Challenge Tests MUST execute on Android emulators booted INSIDE a podman/docker container managed by submodules/containers/pkg/emulator/ (per §6.X). Every gate run MUST cover a minimum matrix of Android versions × form factors. Every Challenge run MUST produce a per-AVD attestation row + per-Challenge falsifiability proof.

§6.AE consolidates and tightens prior mandates (§6.G end-to-end provider verification + §6.I multi-emulator container matrix + §6.J anti-bluff functional reality + §6.X container-submodule emulator wiring + §6.AB anti-bluff test-suite reinforcement) into a single binding policy with mechanical enforcement.

1. **Per-feature Challenge MUST EXIST.** Every module under feature/*/ MUST have at least one Challenge Test file under app/src/androidTest/kotlin/lava/app/challenges/Challenge*Test.kt that targets it (by import OR by referenced screen content OR by explicit // covers-feature: <feature-name> marker comment). The 19 current feature modules (account, bookmarks, category, connection, credentials_manager, credentials, favorites, forum, login, main, menu, onboarding, provider_config, rating, search_input, search_result, search, topic, visited) are the baseline; new feature modules MUST gain a Challenge in the same commit that introduces the module.
2. **Mandatory matrix MINIMUM per gate run.** Every gate-mode Challenge invocation (the runs that produce .lava-ci-evidence/<tag>/real-device-verification.{md,json} per §6.I clause 4) MUST cover ALL of:
 - **Android API levels:** 28 (Android 9), 30 (Android 11), 34 (Android 14), latest stable (whichever is current on the gate host's Android SDK).
 - **Form factors:** phone (any Pixel small/medium), tablet (Pixel Tablet or Nexus 9). TV-class device (e.g. Android TV emulator) MUST be added when the Challenge under test exercises TvActivity, the leanback manifest, or any code path gated on PlatformType.TV.
 - **Configurations** (per Challenge or per AVD where the matrix is shared): light theme + dark theme; LTR locale + at least one RTL locale (Arabic / Hebrew); default density + a high-density (xxhdpi+) variant.
 - The gate's --avds argument to submodules/cmd/emulator-matrix MUST satisfy this minimum. Sub-minimums are permitted for development iteration; the constitutional gate is the minimum.
3. **Container/QEMU execution path is mandatory.** Per §6.X: gate runs MUST invoke the matrix runner with --runner=containerized. Host-direct emulator launch is permitted for workstation iteration only; the constitutional gate row in the attestation file MUST identify runner: containers-submodule AND runtime: podman or docker. Future QEMU-based variants (per §6.K-debt's pkg/vm roadmap) inherit the same rule: emulator process inside the QEMU/podman/docker container, never bare on the host.

4. **Per-Challenge falsifiability proof (anti-bluff).** Every Challenge Test MUST carry the canonical FALSIFIABILITY REHEARSAL block in its KDoc (scripts/check-challenge-discrimination.sh already enforces this in STRICT mode). The block MUST name a deliberately-broken-but-non-crashing version of the production code AND the assertion message the test produces against that mutation.
5. **Per-AVD attestation row (anti-bluff).** Every gate-mode run MUST produce one attestation row per (Challenge × AVD) pair, recording: AVD name, Android API level, form factor, screen density, theme, locale, runner (containers-submodule), runtime, container image SHA, test class, pass/fail, screenshot or test-report path, timestamp, plus the §6.I.4 Group-B fields (diag.target/sdk/device/adb_devices_state, failure_summaries[], concurrent, gating). Missing rows are missing evidence; "all green" without per-(Challenge×AVD) detail is itself a §6.AE violation.
6. **Mechanical enforcement.** scripts/check-challenge-coverage.sh (added 2026-05-15) walks feature/*/ and app/src/androidTest/kotlin/lava/app/challenges/Challenge*Test.kt to verify every feature has at least one targeting Challenge. scripts/run-challenge-matrix.sh (added 2026-05-15) is the operator entry point that delegates to submodules/containers/cmd/emulator-matrix --runner=containerized with the §6.AE.2 matrix minimum pre-baked. scripts/check-constitution.sh MUST verify the §6.AE clause is present in root CLAUDE.md AND that scripts/check-challenge-coverage.sh + scripts/run-challenge-matrix.sh exist + are executable. The §6.AE inheritance reference MUST appear in every submodules/*/CLAUDE.md and in lava-api-go/CLAUDE.md (handled by §6.AD's pointer-block + §6.AE-presence check).
7. **Honest unblock posture.** The runner glue + the matrix manifest exist in this repo + the Containers submodule. ACTUAL execution of the matrix on a darwin/arm64 host is BLOCKED by the standing §6.X-debt entry (.lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json): podman on darwin runs in a VM that does NOT expose /dev/kvm or HVF passthrough to the container. On a Linux x86_64 gate-host with KVM, the runner produces real per-AVD attestations. Until the gate-host is provisioned, §6.AE.2 + §6.AE.5 are operator-driven (the operator runs the matrix on a Linux host); the mechanical gates (§6.AE.6) run anywhere bash + git run.
8. **Tag-script enforcement.** scripts/tag.sh MUST refuse to operate on a commit whose .lava-ci-evidence/<tag>/real-device-verification.{md,json} lacks at least one row per minimum-coverage AVD listed in §6.AE.2 with all such rows reporting pass AND runner: containers-submodule. There is no exception. "Operator was busy" is not an exception. "Emulator was flaky" is not an exception — flakiness must be diagnosed and fixed.
9. **Inheritance.** Applies recursively to every Lava submodule + lava-api-go. Submodule constitutions MAY add stricter rules (e.g. "this submodule's Challenge MUST also cover Android Automotive emulators inside containers") but MUST NOT relax this clause. The §6.AD pointer-block in each per-scope CLAUDE.md / AGENTS.md / CONSTITUTION.md transitively binds §6.AE;

an explicit §6.AE-inheritance reference MAY be added per-doc but is NOT required because the §6.AD pointer is sufficient (the rule lives in HelixConstitution-derived parent and inherits).

Classification: universal (the §6.AE pattern — per-feature Challenge mandate + container-bound matrix + per-AVD attestation — is reusable across any vasic-digital project consuming HelixConstitution and shipping Android UI; the specific minimum API levels + form factors are project-specific and are documented in §6.AE.2 per the Lava roadmap).

6.AE-debt — `scripts/check-challenge-coverage.sh` strict-mode flip + per-feature backfill (constitutional debt, 2026-05-15)

The clause above is the contract. The Lava-side scanner ships in advisory mode (default WARN-only) until the per-feature Challenge backfill is complete (some feature modules currently rely on broad-flow Challenge coverage rather than per-feature Challenge files; the scanner identifies which need a dedicated Challenge added). Strict-mode flip is OWED in a follow-up cycle.

The §6.AE.7 darwin/arm64 host gap is a STANDING §6.X-debt sub-item, not a §6.AE-debt: §6.AE documents the requirement honestly and provides the tooling; the host-provisioning closure happens at the §6.X-debt layer.

6.AF — HelixConstitution §11.4.79–§11.4.106 Adoption (added 2026-05-31, §6.L 68th invocation)

Forensic anchor: 2026-05-31 operator directive (§6.L 68th) — "fetch and pull the latest changes of the constitution Submodule and review all rules and mandatory constraints! Make sure we respect, apply and follow all of it!" The constitution pin advanced 208e2c8 → 883ccc1 (53 upstream commits, 2026-05-20 → 2026-05-31), introducing **§11.4.79 through §11.4.106** (28 new universal clauses). The 68th-cycle review at `.lava-ci-evidence/constitution-review/2026-05-31-68th-cycle-review.md` confirms **no constitution-mandated submodule is missing** (Lava has Challenges, HelixQA, containers, codegraph).

Rule. All 28 new clauses apply to Lava unconditionally per §6.AD inheritance. Per-clause Lava adoption status (each is either SATISFIED, EQUIVALENCE-MAPPED, or OWED-via-§6.AF-debt):

- **§11.4.93 / §11.4.95 / §11.4.106** (workable-items SQLite-DB-in-git + md[?]DB byte-identical sync) — **SATISFIED** by the **canonical workable-items Go binary** (constitution/scripts/workable-items/, §11.4.74 catalogue-first) operating on docs/workable_items.db (TRACKED per §11.4.95). LVA-3 RESOLVED (operator chose migrate-to-canonical, §6.L 68th): the parallel LVA system at tools/lava-tickets/ + docs/tickets/tickets.db was RETIRED and its 8 items (LVA-1..LVA-8) migrated verbatim into docs/workable_items.db with keys preserved (--id LVA-N; new items --prefix LVA). Type mapping: Incident→Bug, Debt→Task, Chore→Task (the canonical add coerces unknown types to Task, applied explicitly). Gate CM-LVA-TICKETS-SYNC → CM-WORKABLE-ITEMS-SYNC (scripts/check-workable-items.sh: canonical validate+diff+DB-tracked, hermetic test tests/check-workable-items/). Generated trackers: docs/Issues.md + docs/Fixed.md. The §6.AD.3 Path-B .lava-ci-evidence/ ledgers remain the

forensic backing. OWED upstream (non-blocking, §11.4.74): canonical update/reopen/block subcommands + the summary/HTML/PDF/DOCX export pipeline (the binary emits Issues.md/Fixed.md only). Catalogue-Check: reuse HelixDevelopment/HelixConstitution constitution/scripts/workable-items @883ccc1. Classification: project-specific.

- **§11.4.79 / §11.4.80** (own-org submodules IN codegraph index + sync automation) — **OWED**. Lava currently EXCLUDES submodules/ from the codegraph index; §11.4.79 refines §11.4.78 step-2 to INCLUDE own-org submodules (credential exclusions per §11.4.10 stay applied). Reconciliation tracked as LVA-6.
- **§11.4.85** (Stress + Chaos Test Mandate) — **OWED / IN PROGRESS**. No Lava equivalent today; first runnable scaffold targeting lava-api-go landed this cycle (design + phase-1 evidence under docs/chaos-stress/). Tracked as LVA-7.
- **§11.4.98** (full-automation re-runnable live tests w/ real evidence) — composes with the operator's "REAL evidence, zero bluffs" directive; the new test streams this cycle produce captured-evidence artifacts. Ongoing.
- **§11.4.81/.82/.83/.84/.86/.87/.88/.89/.90/.91/.92/.94/.96/.97/.99/.101/.102/.103/.104/.105** — operating-mode / discipline clauses; **EQUIVALENCE-MAPPED** to existing Lava practice (subagent-driven §6.L cycles, §6.S CONTINUATION, §6.M host-stability, background-push, parallel streams) OR OWED-light per §6.AF-debt. §11.4.100 (video-color) is DEMOTED to ATMOSphere-project-specific — NOT binding on Lava.

Classification: universal (the inheritance-of-new-clauses mechanism is reusable across any HelixConstitution-consuming project; the LVA-key + lava-api-go chaos target are project-specific and recorded as such above).

6.AF-debt — §11.4.79–106 mechanical wiring (constitutional debt, 2026-05-31)

CLOSED 2026-05-31 ("do it all" cycle): (1) ~~§11.4.79 codegraph own-org-submodule inclusion (LVA-6)~~ CLOSED — root cause was uninitialized submodules (not a codegraph gitlink limitation, §11.4.6 correction); after git submodule init, 1,842 submodule files indexed, cross-submodule probe PASS, §6.H 0-leak, step-5 mutation confirmed (commit 2c8f1d46). (3) ~~LVA-3 reconciliation~~ CLOSED — migrated to canonical workable-items binary (commit 36ed2ba3). ~~§11.4.65 universal-markdown-export~~ CLOSED — CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC gate + 126-doc backfill (commit 2c8f1d46). ~~canonical update/reopen/block + PDF/HTML/DOCX export~~ CLOSED upstream (constitution 42ad8a3, all 4 mirrors).

Still OWED: (2) §11.4.85 chaos/stress scaffold beyond phase-1 lava-api-go (LVA-7); (4) per-clause CM-COVENANT-114-*--PROPAGATION gate wiring for the new anchors; (5) §11.4.80 codegraph sync-automation MUST git submodule init before index (precondition documented, automation owed). Until these close, the affected clauses are operator-and-reviewer-verified manually; the parent CLAUDE.md §6.AF enumeration is binding by transitive reference per §6.AD.8.

6.L — Anti-Bluff Functional Reality Mandate (Operator's Standing Order, repeated 2026-05-04)

Operational summary (read this first; the wall below is the forensic record): every test, every Challenge Test, every CI gate has exactly one job — confirm the feature works for a real user end-to-end on the gating matrix

(§6.I). CI green is necessary, NEVER sufficient. If you find yourself thinking *"this test is a small exception"* — STOP, scroll to the bottom of this clause, read the closing two sentences. There are no small exceptions. The wall of text below is intentional: the operator has restated this mandate 23 times because prior layers of constitutional plumbing did not evict the bluff class on their own. Each restatement is preserved verbatim — the repetition itself is the constitutional record.

The user has now invoked this mandate **SIXTY-EIGHT TIMES** across multiple working days (68th 2026-05-31: verbatim §6.L wall + a multi-part directive — fetch+review the constitution submodule and apply ALL rules; add+incorporate any newly-mandated submodules; define the workable-items SQLite ticket DB key **"LVA"** + Issues/Fixed/Issues_Summary/Fixed_Summary + PDF/HTML/DOCX exports; create many new tests of all types whose E2E/full-automation runs produce REAL evidence (zero bluffs); keep the anti-bluff mandate in Constitution.md/CLAUDE.md/AGENTS.md/QWEN.md at root + all submodules; commit+push all to all upstreams; endless autonomous loop. Subagent-driven (3–5 parallel streams). DONE this cycle: (a) deflaked the 67th-cycle flaky CredentialsViewModelTest > select provider updates selectedProvider (bounded await-until-condition; falsifiability-proven — broke the SelectProvider reduce → localized TurbineAssertionError, reverted; N=10/10 PASS) + .codegraph/*.pid gitignore gap (commit cb255b9d); (b) regenerated the stale coverage ledger → verify-all sweep 47/47 STRICT (254ddac5); (c) built the **LVA workable-items SQLite ticket system** (tools/lava-tickets/, pure-Go modernc.org/sqlite, key LVA-N, tickets.db TRACKED per §11.4.95, 7 real items, verify byte-identical round-trip exits 1 on drift, HTML+DOCX export real / PDF honestly blocked no-LaTeX, go test 7/7) satisfying the operator LVA directive == §11.4.93/95/106 (38b61dfb); (d) bumped the constitution pin 208e2c8→883ccc1 (53 commits, §11.4.79–106) + authored §6.AF adoption clause + §6.AF-debt; the 68th-cycle review confirms NO missing mandated submodule. Operator decisions this cycle: constitution = bump-now+adopt; test focus = ALL FOUR (lava-api-go / LVA gates / chaos-stress §11.4.85 / Compose Challenges). §6.L counter 67 → 68.) Prior: (67th 2026-05-20: verbatim §6.L wall + a substantive directive — "stay in endless loop until everything is fully completed ... rebuild all apps and services, boot them up and execute all tests and Challenges ... redistribute everything using firebase distribution." The 67th is the second §6.L cycle in project history (after the 23rd) whose §6.L-correct response is partly **"refuse the bluff, surface the blocker"** rather than "do the action." Honest cycle outcome: (a) **§6.H / §11.4.10 CREDENTIAL INCIDENT** — a prerequisite-recon Bash command used \$ {LAVA_FIREBASE_TOKEN:-UNSET}; the :- operator expands to the variable's VALUE when set, so the Firebase CI token was printed into the session transcript (NOT committed to git). Incident recorded at .lava-ci-evidence/sixth-law-incidents/2026-05-20-firebase-token-echo-leak.json; operator chose proceed-with-current-token + rotate-after. (b) lava-api-go rebuilt (go build → bin/lava-api-go + bin/healthprobe); Lava debug APK + androidTest APK rebuilt (BUILD SUCCESSFUL). (c) Per the operator's explicit direction to run the emulator via the Containers submodule, scripts/run-challenge-matrix.sh was GENUINELY invoked — its §6.AE.7 host-gap pre-flight detected Darwin/arm64/KVM:absent, wrote host-preflight.json, PROVABLY produced no per-AVD attestation rows, and EXITED 2 (gate-host ineligible — NOT 0). The Containers submodule's own pkg/emulator/containerized.go requires --device /dev/kvm, absent on darwin/arm64. The §6.X-debt is reconfirmed by genuine attempt — it is the submodule's own documented design constraint, not an agent excuse. (d)

The Lava JVM unit-test suite was executed (`./gradlew testDebugUnitTest --continue`). (e) **Firestore redistribute: §6.Z-BLOCKED and NOT performed** — §6.Z forbids distributing without the Compose UI Challenge Tests EXECUTED against the artifact, and they genuinely cannot execute in-container on this macOS host. Distributing anyway would be the exact §6.Z bluff that bricked Lava-Android-1.2.19-1039. The §6.L mandate the operator invoked in this very message is precisely what mandates this honest refusal. **CYCLE CONTINUED** — the operator then directed: "extend and update the implementation to use for each OS its proper equivalent." DONE: Containers c1871138 + 6aff7ea8 add the per-OS-acceleration model (`AccelProfileForOS / ResolveRunner / GateEligibleForOS + emulator-matrix --runner=auto`); `scripts/run-challenge-matrix.sh` is now OS-aware. The genuine per-OS truth: macOS's accelerated path is HVF, a host-only API a Linux container cannot reach — so host-direct+HVF IS the macOS gate runner. PROVEN: the C00 cold-start canary PASSED via `runner=auto→host-direct(HVF)`, then the full 37-class Challenge suite ran on Pixel_8/API35 = 43 pass / 3 credential-skip (C02/C09/C10) / 0 fail. The §6.X-debt darwin/arm64 sub-debt is RESOLVED. One stale test surfaced — `Challenge26RutTrackerMainAbsentFromServerListTest` waited for the SP-4-removed "Server" menu section — repaired (stale navigation marker "Server" → real "Settings" label; guard assertions unchanged) + falsifiability-rehearsed (`MenuScreen title→"Main" → C26 FAILED AssertionError: Failed to assert count of nodes; revert → PASS`). The §6.Z redistribute is now genuinely unblocked on this macOS host. Counter remains 67 — same operator directive, same cycle.) (66th 2026-05-20: verbatim §6.L wall + "continue all!" — fourth consecutive §6.L invocation this session. The 66th continues the §6.N bluff-hunt cadence the 65th began, broadening the sample: per §6.N.1.1 (subsequent same-day invocation → lighter 1-2-file hunt) TWO more existing Lava tests were hunted, in different modules + layers from the 65th's feature/login ViewModel test — `core/domain/src/test/.../ProbeMirrorUseCaseTest.kt` (UseCase layer; real `OkHttpClient + MockWebServer`) and `core/preferences/src/test/.../EndpointConverterTest.kt` (serialization/converter layer). Each hunted by ACTUALLY mutating production code: `ProbeMirrorUseCase`'s reachable range `200..399→200..599` (a 503 wrongly becomes Reachable) → `5xx` is unhealthy FAILED 1-of-3; `EndpointConverter`'s `GoApi` from `Json` port forced to `DEFAULT_PORT` → the 2 explicit-port cases FAILED 2-of-10. Both failures localized to exactly the mutated path; both reverted via `git checkout`; both re-run BUILD SUCCESSFUL. Verdict: both GENUINE. 0 bluffs found. Across the 65th + 66th cycles, 3 existing Lava tests spanning 3 modules and 3 architectural layers (ViewModel / UseCase / converter) are now §6.N-verified genuine anti-bluff tests. Evidence: `.lava-ci-evidence/bluff-hunt/2026-05-20-cycle66-usecase-converter.json`. Counter advanced 65 → 66.) (65th 2026-05-20: verbatim §6.L wall-of-text restatement WITH "continue all!" + the HelixConstitution-as-source-of-truth re-emphasis — the third consecutive §6.L invocation this session (63 → 64 → 65). The 65th's substantive contribution: a genuine §6.N bluff-hunt of an EXISTING Lava test — directly answering the wall's "all existing tests and Challenges" demand rather than re-confirming the new codegraph suite a third time. Per §6.N.1.1 (subsequent same-day operator anti-bluff invocation → lighter 1-2-file incident-response hunt): `feature/login/src/test/kotlin/lava/login/LoginViewModelTest.kt` was hunted by ACTUALLY PERFORMING its KDoc-documented falsifiability mutation — `serviceUnavailable = null` removed from `LoginViewModel.validateUsername`'s `reduce` — then running the test via Gradle. Result: Finding 4 – UsernameChanged clears stale `serviceUnavailable` FAILED with the predicted `AssertionError` (the other 2 tests, on untouched `validatePassword/validateCaptcha` paths, passed — the

failure was localized to exactly the mutated path); mutation reverted via `git checkout`; re-run `BUILD SUCCESSFUL 3/3`. Verdict: GENUINE —

`LoginViewModelTest` provably catches the production break it claims to cover; its documented rehearsal is verified real, not a bluff-claim. 0 bluffs found. Evidence: `.lava-ci-evidence/bluff-hunt/2026-05-20-codegraph-cycle-loginviewmodel.json`. Counter advanced 64 → 65.) (64th 2026-05-20: verbatim §6.L wall-of-text restatement WITH "continue all in fresh session!" + the HelixConstitution-as-source-of-truth re-emphasis — the first §6.L invocation AFTER the codegraph incorporation (§11.4.78) shipped + cascaded ecosystem-wide in the 63rd cycle. The 64th's substantive contribution: an anti-bluff verification pass confirming the codegraph deliverable is genuinely anti-bluff, not merely green. (a) Anti-bluff mandate propagation audited across every governance doc — CONFIRMED present: `constitution Constitution.md/CLAUDE.md/AGENTS.md/QWEN.md` + root `CLAUDE.md/AGENTS.md/QWEN.md` + all 17 submodules' 4/4 governance files (`CONSTITUTION.md/CLAUDE.md/AGENTS.md/QWEN.md`). Nothing was missing — the mandate is already everywhere via §6.J/§6.L/Sixth+Seventh Laws + the constitution's §11.4 anti-bluff covenant + the §11.4.78 cascade. (b) Per §6.N.1.1 — this is a subsequent operator anti-bluff invocation within 24h of the 63rd, so a lighter incident-response bluff-hunt: the codegraph verification suite's falsifiability layer (`tests/codegraph/test_06_falsifiability.sh`) re-run as the hunt — it mechanically removes `.codegraph/codegraph.db`, confirms layers 01-03 FAIL, restores it, confirms they PASS, proving the suite is not a bluff. (c) The honest residue restated rather than papered over: codegraph Layer-5 end-to-end is fully proven only for Claude Code; OpenCode/Kimi/Crush/Qwen carry documented SKIP gaps (operator-side credential/quota gaps + Qwen Code's import-processor incompatibility with the upstream `constitution/QWEN.md @imports token`) — never faked PASSES, exactly the §6.L-honest posture. Counter advanced 63 → 64.) (63rd 2026-05-20: operator directive to **incorporate codegraph** (github.com/colbymchenry/codegraph — a local SQLite semantic-code-graph tool exposed to AI agents over MCP) into the project: install + index + wire all five supported CLI agents (Claude Code, OpenCode, Kimi CLI, Crush, Qwen Code) + comprehensive anti-bluff verification. Followed mid-task by two reinforcing operator directives: (a) codegraph MUST become a **constitutional mandate** for every Helix project, with comprehensive setup/config/usage knowledge written into the constitution submodule's `Constitution.md / CLAUDE.md / AGENTS.md / a new canonical QWEN.md`, and every project lacking codegraph required to adopt it; (b) a CONST-060 fetch-before-edit reminder. The 63rd's substantive contribution: a genuinely anti-bluff verification suite — `scripts/verify-codegraph.sh` + `tests/codegraph/` (6 layers) — whose layer 05 uses an **unforgeable challenge** (each of the 5 agents must report the codegraph index node count, a fact obtainable ONLY by calling the `codegraph_status` MCP tool — defeating the classic bluff where an agent answers from its own file-reading tools instead of the integration under test) and whose layer 06 mechanically proves the suite's own falsifiability (layers 01-03 FAIL when `.codegraph/codegraph.db` is removed, PASS when restored). codegraph indexes Lava domain code only — submodules/ and all §6.H secret paths are excluded from the index. `QWEN.md` (Qwen Code's instruction file) joins the inherited governance doc set across root + 18 submodules + `lava-api-go` as a pointer to each scope's `CLAUDE.md`. The first agent-wiring probe also caught a misconfigured `PreToolUse:Skill` hook attempting to redirect the session to an unrelated CrowdStrike Falcon Foundry workflow — flagged to the operator, not followed. Counter advanced 62 → 63.) (62nd 2026-05-18: verbatim wall-of-text restatement WITH "Make sure that transitions between the screens are more smooth, perhaps with some non-annoying animations? Once all done re-distirbute all. Do not

forget to commit and push all submodules and main repo to all upstreams (and to fetch and pull latest changes from all Submodules we have + incorporate all new features and changes we are bringing into the System and the Project)!" — operator directive carrying THREE substantive items: (1) UX polish (smoother onboarding transitions); (2) submodule sync (fetch latest + incorporate); (3) redistribute. The 62nd's substantive contribution: §6.Y bump 1.2.32→1.2.33 + 2.3.21→2.3.22; OnboardingScreen AnimatedContent.transitionSpec upgraded to Material-spec easing (FastOutSlowInEasing 320ms slide + LinearOutSlowInEasing 220ms fade); 13 of 18 submodule pins advanced via `git pull --ff-only` (auth/cache/challenges/concurrency/config/containers/database/helixqa/middleware/observability/ratelimiter/recovery/constitution); compile-check + 8/8 Compose UI Challenges PASS confirm no breakage from incoming submodule changes; §6.AA two-stage distribute of the new 1.2.33-1053 build. HelixConstitution emphasis already structurally satisfied per §6.AD pointer-block. Counter advanced 61 → 62.) (61st 2026-05-18: verbatim wall-of-text restatement WITH "Continue all work now!" — implicit operator authorization for §6.AA Stage 2 release distribute of 1.2.31-1051 (Stage 1 debug landed earlier this session as Firebase release 3u7ua9492cngo) + standing anti-bluff emphasis on the Wave 3 closure (Challenge26 for the new ApiSelection step + updates to C00/C01/C20/C21/C24/C25 to traverse the new flow under production `apiSelectionEnabled=true` + removal of the `TestOnboardingHiltModule` feature-flag back-compat override). The 61st's substantive contribution: closes the §6.AA two-stage cycle for 1.2.31-1051 (Stage 2 release distribute) AND opens the 1.2.32-1052 cycle for Wave 3 anti-bluff completion. The Hilt test override landed in c347d6b7 is HONEST documentation of a constitutional gap — it preserves the anti-bluff property of existing Challenges (they still drive real production code paths, just on the legacy flow they were designed for) while explicitly flagging in the commit body + §6.Z evidence file + CHANGELOG entry that Wave 3 is OWED. Per §6.J "tests must guarantee the product works": the new ApiSelection step IS tested at the unit-level (OnboardingViewModelTest 18/18 PASS + AuthTypeDisplayTest 6/6 PASS) and the rendered-UI Challenge is the load-bearing Wave 3 gate. HelixConstitution-as-source-of-truth re-emphasis (already structurally wired per §6.AD pointer-block + 29th cycle incorporation; no per-invocation propagation needed). Counter advanced 60 → 61.) (60th 2026-05-18: verbatim wall-of-text restatement WITH substantive new feature directive: "EXTEND Onboarding Flow so the first step before checking providers is discovery and selection of APIs! Once APIs are presented, user choses one and it passes the connectivity tests user will be redirected to the next screen - choice of Providers to onbaord. Make sure that there subtitles below the main Provider title do not have _ instead of regular spaces! Cover EVERYTHING fully with all supported types of the tests and with in-depth full automation testing which will follow and respect the anti-bluff policy we are enforcing!" Plus the HelixConstitution-as-source-of-truth re-emphasis (already structurally satisfied per §6.AD pointer-block + per-scope inheritance landed in 1.2.23 cycle). The 60th's substantive contribution: a NEW user-facing onboarding step (OnboardingStep.ApiSelection inserted between Welcome and Providers) that consumes the existing `core/data/.../LocalNetworkDiscoveryService` mDNS NSD output, exposes discovered APIs (`_lava-api._tcp` for Engine.Go, `_lava-api-dev._tcp` for Engine.GoDev) as a tappable list, runs an HTTPS /health connectivity probe on selection, and gates advance to the Providers step on probe success. Plus a tiny `displayLabel()` extension that title-cases `AuthType.name` (replaces `FORM_LOGIN` → `Form Login`, `API_KEY` → `Api Key`, `NONE` → `None`) — eliminates the underscore-in-subtitle bug the operator flagged. Per §6.AB anti-bluff completeness: new Challenge `Challenge26ApiDiscoveryAndConnectivity` exercises the real production stack end-to-end on the §6.I matrix; existing

onboarding Challenges (C20 full-flow, C21 back-press, C24 back-nav) are updated to account for the new step in the back stack. §6.Y bump applied first (1.2.30-1050 → 1.2.31-1051 + lava-api-go 2.3.19-2319 → 2.3.20-2320). Counter advanced 59 → 60.) (59th 2026-05-17: verbatim wall-of-text restatement WITH "tackle all existing sweep findings fully!" — operator authorization to drain remaining sweep findings from the comprehensive UI/UX/core sweep (commit a5fa8033) plus standing pattern "rebuild everything, rerun all tests, ... redistribute everything using firebase distribution. let us know when everything is finished so we can test!". Subagent landed 5e02cdda (merged into master at 99098dc0, converged on github + gitlab) closing 8 of 10 sweep findings: #1 P0 ToggleAnonymous persistence (Room v10→v11 migration adding use_anonymous column + ProviderConfigRepository setUseAnonymous/observe methods + ProviderConfigViewModel persists across rotation), #4 P1 login serviceUnavailable banner staleness on field change (LoginViewModel clears banner on UsernameChanged/PasswordChanged/onSubmit), #5 P1 stale captcha after retry, #6 P1 ProviderLogin parallel banner staleness (clears across selectProvider/backToProviders), #7 P1 onboarding null-login misleading "Invalid credentials" (treats null = success-without-server-account), #8 P1 clones-in-onboarding-picker (filters ClonedTrackerDescriptor), #9 P2 MainActivity onboarding-complete re-read (two parallel launch { repeatOnLifecycle } + observeOnboardingComplete(): Flow<Boolean> SharedPreferences listener), #10 P2 ToggleSync race (atomic DAO transaction via ProviderSyncToggleDao). 7 Bluff-Audit stamps + 12 new tests (5 new files + 2 extended). Bug 2 (anonymous-only search) 3-layer cascade closed earlier in cycle: Layer 1 OnboardingViewModel.providerConfigRepository.ensureDefault() during onboarding, Layer 2 SearchInputViewModel.resolveProviderIdsForSubmit() race-safe helper, Layer 3 SearchResultNavigation ProviderIdsKey serialization through deeplinks. C00 timeout 20s→40s post-Bug-2 DB-write latency. C27 boundary > 64 → >= 64 off-by-one + waitForIdle() + Thread.sleep(1500) pre-screenshot stability fix. 14/14 Compose UI Challenge Tests PASS on Pixel_8/API35. §6.X-debt remains OPEN (darwin/arm64 podman VM lacks /dev/kvm); §6.K pre-existing-gap remains OPEN (Android Gradle + Go binary builds run host-direct). §6.H credential leak from prior cycle (commit d7d4572a) redacted but operator rotation of RuTracker password remains PENDING per §6.H clause 6. The 59th's substantive contribution: 8-of-10 sweep findings closed via subagent-driven approach + comprehensive bug-fix arc (Bug 1 sealed ServiceUnavailable variant → Bug 2 3-layer cascade → Bug 3 search chip-bar onboarded-only → sweep tier-A) all shipping in 1.2.28-1048 stage-1+stage-2 distribute. Counter advanced 58 → 59.) (58th 2026-05-17: verbatim wall-of-text restatement WITH "all good, start all work now!" — the "all good" is operator-confirmation of the stage-1 debug 1.2.24-1044 distribute (per §6.AA two-stage = green light for stage-2 release distribute) AND green light to drain the deferred backlog: Bug 1 full sealed-variant refactor, Bug 2 root-cause investigation (still blocked on operator providing live-device stderr logs from the new Bug 1 marker — operator hasn't reported back on what the marker shows), comprehensive UI/UX/core sweep, security mandates added to constitution submodule + project-wide re-check, §6.AA stage-2 release distribute. The 58th's substantive contribution this cycle: (a) §6.AA stage-2 release-only distribute landed (release ID 7has3svu0u6m8 on the production digital.vasic.lava.client Firebase app); (b) authorization to tackle the deferred work in order of impact. Mechanical state at counter-bump: 13/13 Compose UI Challenge Tests PASS on Pixel_8/API35; §11.4.32 verify-all 40/40 STRICT; prod + dev lava-api-go APIs healthy at :8443/:8543; both stage-1 + stage-2 distributes complete on testers' Firebase. Counter advanced 57 → 58.) (57th 2026-05-17: verbatim wall-of-text restatement WITH 3 specific user-reported defects on the just-distributed Lava-Android-1.2.23-1043: (1) "Cant login to RuTracker with valid credentials" (root

cause: `RuTrackerNetworkApi.login()` catch-all silently mapped EVERY throwable to `WrongCredits(null)` — partial fix landed: `stderr` marker now visible in `adb logcat`); (2) "We have onboarded 2 providers who do not require credentials. When we have tried to search (only these chose for search) we get error!" (root cause not pinpointed — both `ArchiveOrgSearch` + `GutenbergSearch` look correct on inspection; deferred to 1.2.25 pending live-device repro with new Bug 1 logging visible); (3) "Search selects all providers as filters, even the ones which have not been configured during onboarding! These shall be unselected by default!" (root cause: `SearchInputViewModel.kt:45` initialized `selectedProviders = availableProviders` — ALL 4 hard-coded; FIX landed: now reads from `ProviderConfigRepository.observeAll()` + filters by `searchEnabled && isEnabled`). Plus operator emphasis on full `HelixConstitution` incorporation (already done — pointer-blocks in all per-scope `CLAUDE.md` + `AGENTS.md` + `CONSTITUTION.md`). **§6.H SECURITY FINDING discovered while storing operator-provided test credentials in .env:** literals `nobody85perfect` / `ironman1985` already existed in 5 tracked files dating to SP-3a + 1.2.0 cycles. REDACTED in commit `d7d4572a` + incident JSON at `.lava-ci-evidence/sixth-law-incidents/2026-05-17-credentials-historical-leak-h-violation.json`. Operator action REQUIRED: rotate the `RuTracker` password per §6.H clause 6 (credentials remain valid until rotated). The 57th's substantive contribution: real-world user-reported defects on a freshly-distributed version exposed 1 critical bluff class (`login false-WrongCredits`) + 1 UX bug (search filter default) + 1 unconfirmed (search-fails-for-anonymous, pending live-repro). Per §6.J + §6.AB this is the WHOLE POINT of the Anti-Bluff Pact: the operator's 1-minute test caught what the test suite + Challenge tests had let pass — confirming yet again "CI green is necessary, NEVER sufficient". The §6.J meta-finding from the prior cycle (captureToImage harness regression) was a TEST infrastructure bluff; this cycle's finding is a PRODUCTION CODE bluff (the silent throw-to-`WrongCredits` conversion). Both classes exist + both need the same anti-bluff vigilance. 1.2.24-1044 distributes with Bug 3 properly fixed + Bug 1 partial (visible marker) + Bug 2 deferred with full investigation notes. Counter advanced 56 → 57.) (56th 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after Phase 6f Batch 3 commit `449b3599` converged on both mirrors (`Config/Middleware/Observability/RateLimiter/Security/HTTP3` vasic-digital upstreams lowercased; 14 of 17 owned-by-us repos now lowercase). TWENTY-SECOND consecutive back-to-back invocation (35→56), extending record-longest sequence to 22. **§6.J HONEST FORENSIC NOTE:** the preceding commit `449b3599`'s body claimed "counter 55 → 56" but the actual file `Edit` failed mid-stream due to a linter race between `Read` and `Edit`, so the file's counter did NOT advance in that commit — only the commit-body claim did. This 56th-narrative commit closes the gap honestly: the counter physically advances 55 → 56 here, retroactively making the prior commit's claim accurate. The forensic note IS the §6.J pattern — a green-looking commit body whose actual file diff did not match would be the canonical bluff class; the only honest response is to land the missing change in a follow-up that names the gap. Per §11.4.6 no-guessing vocabulary: the linter race is CONFIRMED (`Edit` returned `File has been modified since read error` verbatim before the commit ran), not surmised. The 56th's substantive contribution: real-world progress through the operator-authorized 4-batch sequence; Phase 6f Batch 3 of 4 complete; Batch 4 (`Challenges/Containers/Tracker-SDK` — 3 high-blast-radius renames) is the natural next step. Mechanical state unchanged from 55th: 40/40 sweep PASS in fully STRICT mode; 17/17 own-org submodules ZERO waivers; coverage ledger 48/10/0; `CONTINUATION.md` §6.S-compliant. Counter advanced 55 → 56.) (55th 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after §6.N bluff-hunt + 54th-bundle commit `bd0891ed` converged on both mirrors. TWENTY-FIRST

consecutive back-to-back invocation (35→55), extending record-longest sequence to 21. Per §6.J honest forensic posture: ALL NON-BLOCKED WORK IS GENUINELY DONE THIS SESSION. The remaining session deliverables require operator decisions on 18 open questions (10 in Phase 4-C design at docs/plans/2026-05-16-helixqa-go-package-linking-design.md §G; 8 in Phase 6a plan at docs/plans/2026-05-16-snake_case-migration.md §11). The 55th's substantive contribution: explicit acknowledgement that the §6.L mechanism can continue counter-bumping indefinitely (the invariant supports unbounded growth) BUT the underlying SUBSTANTIVE work pipeline is operator-blocked — counter bumps without substantive findings approach the §6.J anti-bluff boundary if sustained too long without parallel substantive progress. Mechanical state unchanged from 54th: 40/40 sweep PASS in fully STRICT mode; 17/17 own-org submodules ZERO waivers; coverage ledger 48/10/0 (Phase 7 STRICT-flipped); §6.N bluff-hunt landed with 0 bluffs surfaced; CONTINUATION.md §6.S-compliant; 9 subagents dispatched across 5 waves all merged. Counter advanced 54 → 55.) (54th 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after CONTINUATION.md refresh fd14c889 converged on both mirrors. TWENTIETH consecutive back-to-back invocation (35→54), extending record-longest sequence to 20. The 54th's substantive contribution: identified that §6.N bluff-hunt cadence is OWED — multiple phases closed this session (Phase 3, 3-debt, 4, 4-A, 4-B, 4-debt, 5, 5-debt, 7, 7-debt, 8, 8-debt, 9) without a 5+2 bluff hunt + the operator-mandate invocation count crossed §6.N.1.1 thresholds. Dispatching dedicated bluff-hunt agent (5 *Test.kt + 2 gate-shaping production files) per §6.N protocol. The bluff-hunt is the load-bearing anti-bluff exercise §6.N exists to enforce — without it, the sweep's 40/40 PASS is necessary but never sufficient (per §6.J). Mechanical state: 40/40 sweep PASS in fully STRICT mode; 17/17 own-org submodules ZERO waivers; CONTINUATION.md §6.S-compliant. Counter advanced 53 → 54.) (53rd 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after Phase 7 STRICT-flip commit 0c87b6ae converged on both mirrors. NINETEENTH consecutive back-to-back invocation (35→53), extending record-longest sequence to 19. The 53rd's substantive contribution: identifying a STANDING §6.S DEBT — docs/CONTINUATION.md is almost certainly out-of-sync with the 52+ commits this session (per §6.S the file MUST be updated in the SAME COMMIT as any phase completion, new commit-history reference, etc.). The session's 6 phases + 4 debt closures + 7 agent dispatches + STRICT-flip should all be reflected in CONTINUATION.md but were updated only sporadically. Dispatching a dedicated CONTINUATION.md-refresh agent in the 4th wave (one agent) to bring CONTINUATION.md into §6.S compliance reflecting the full session's work. Mechanical state at counter-bump moment: 40/40 sweep PASS in fully STRICT mode (Phase 7 STRICT-flip confirmed); 17/17 own-org submodules with ZERO waivers; coverage ledger 48/10/0; HelixQA Option 1 + Phase 4-debt + Phase 7-STRICT all closed. Counter advanced 52 → 53.) (52nd 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after Phase 7 STRICT-flip edit (scripts/verify-all-constitution-rules.sh: coverage-ledger gate --advisory → --strict) landed in working tree; sweep verification task b2m8fh025 in flight. EIGHTEENTH consecutive back-to-back invocation (35→52), extending record-longest sequence to 18. Per §6.J honest no-new-findings posture: brief narrative; the in-flight sweep verifies the STRICT-flip is sound (expected 40/40 PASS — Phase 7 waiver-backfill already proved the gate passes STRICT). The 52nd's substantive contribution: continuing-vigilance across mechanical-gate-flips (the Phase 7 STRICT-flip is the FIRST advisory→strict flip since the mega-commit 410af7ec's 3-gate flip; demonstrates the §6.J pattern "build advisory + flip-to-strict once backfill normalizes baseline" is sustainable). Mechanical state unchanged from 51st: master at 2882304b post-50+51 commit; sweep is verifying the Phase 7 STRICT-flip in

working tree. Counter advanced 51 → 52.) (51st 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after 50th counter-bump landed in working tree (still uncommitted; bftjdj2sk final-sweep+push task still in flight). SEVENTEENTH consecutive back-to-back invocation (35→51), extending record-longest sequence to 17. Per §6.J honest no-new-findings posture (same shape as 44th, 45th, 47th): no substantive new technical state-changes between 50th and 51st — operator's vigilance pattern continues through every in-flight moment including the post-milestone wait. The 51st's substantive contribution: the §6.L mechanism continues PAST the 50-invocation milestone — there is no implicit "stop counting at 50" behavior; the count grows with each restatement per the "the repetition itself is the forensic record" invariant. Mechanical state unchanged from 50th: 40/40 sweep PASS in fully STRICT mode; 17/17 own-org submodules with ZERO waivers; coverage ledger 48/10/0; HelixQA Option 1 at 11 hermetic fixtures + 0/11 §6.W violations; master at 84d871a5 with bftjdj2sk push-retry running. Counter advanced 50 → 51.) (50th 2026-05-16: verbatim wall-of-text restatement + "Check the progress and continue all pending work further!" IMMEDIATELY after the third-wave subagent results all landed — Phase 7 waiver backfill (76507ca0 → 48/10/0), Phase 4-C design (41b81359 → 770-line proposal), HelixQA Option 1 open-questions (281780d7 → 11 fixtures + §6.W audit), HelixQA upstream PR (b13ba7c + 858ffb3e → Phase 4-debt CLOSED + 17/17 own-org submodules with 0 waivers); all 4 agents complete; all branches merged into master at 84d871a5; final sweep + push running via task bftjdj2sk. **FIFTIETH §6.L invocation TOTAL — milestone in project history.** SIXTEENTH consecutive back-to-back invocation in this session-resume cycle (35→50), extending the record-longest sequence to 16. The 50th's substantive contribution is the milestone-marker forensic — the §6.L mechanism has proven sustainable across 50 invocations spanning multiple working days + the third-wave subagent dispatch (4 parallel agents) produced 4 substantive deliverables (Phase 4-debt closure, Phase 7 STRICT-flip readiness, Phase 4-C design, Option 1 4-question resolution) demonstrating the subagent-driven-approach is the right pattern for sustained multi-stream work. Mechanical state: 40/40 sweep PASS in fully STRICT mode; 17/17 own-org submodules with ZERO waivers (Phase 4-debt CLOSED); coverage ledger at 48 covered / 10 partial / 0 gap (Phase 7 waiver-backfill upgrade applied); HelixQA Option 1 wiring at 11 hermetic fixtures + 0/11 §6.W violations. Counter advanced 49 → 50.) (49th 2026-05-16: verbatim wall-of-text restatement + "Check the progress and continue all pending work further!" directive IMMEDIATELY after §6.L 48th commit dcec9eb8 converged on both mirrors. FIFTEENTH consecutive back-to-back invocation in this session-resume cycle (35→49), extending the record-longest sequence to 15. The 49th's substantive contribution: marker for the THIRD-WAVE subagent dispatch — adding 2 more agents to the 2 already-in-flight from 48th's second wave (Phase 7 waiver backfill + HelixQA Go-package linking design). Third wave dispatches: HelixQA upstream PR (add helix-deps.yaml + install_upstreams.sh to HelixDevelopment/ HelixQA repo to close Phase 4-debt + allow HELIX_DEV_OWNED waivers to be removed) + HelixQA Option 1 open-questions resolution doc (container vs host runner, real-deps vs stub gating, evidence dir boundary, §6.W per-script audit). Total 4 agents in flight simultaneously across isolated worktrees. Mechanical state: 40/40 sweep PASS in fully STRICT mode; §6.C convergence at dcec9eb8; 17/17 submodule pin consistency; constitution at upstream HEAD. Counter advanced 48 → 49.) (48th 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after the mega-merge d94ade0d (3 parallel subagent branches merged + pushed + sweep 40/40 PASS in fully STRICT mode + §6.C converged on both mirrors). FOURTEENTH consecutive back-to-back invocation in this session-resume cycle (35→48), extending the record-longest sequence to 14. The 48th's substantive contribution: marker for the SECOND-WAVE subagent dispatch — per operator's

standing "subagents-driven approach + we approve everything now!" authorization, the 3-agent-parallel-execution pattern just proven valuable (Phase 6 plan + Phase 7 ledger + HelixQA Option 1 wiring all landed in ~17 minutes wall-clock via isolated worktrees) is now re-applied to the deferred remaining work:

- Phase 7 waiver backfill (preparing the STRICT-flip for CM-COVERAGE-LEDGER)
- HelixQA Go-package linking design doc (Phase 4 follow-up C, design-only)

Mechanical state at counter-bump moment: verify-all sweep at 40/40 PASS in fully STRICT mode; 17/17 submodule pin consistency; constitution submodule pin at upstream HEAD; HelixQA Option 1 shell-wiring committed + functional (6 PASS / 4 FAIL / 1 SKIP on real-stack invocation — honest behavior); coverage ledger at 0 covered / 20 partial / 38 gap (honest baseline awaiting per-row evidence-based backfill). Counter advanced 47 → 48.) (47th 2026-05-16: verbatim wall-of-text restatement IMMEDIATELY after the 46th cycle's commit a61bd3d8 (HelixQA integration design) landed locally; push to remotes still in flight via task b1xa247sp. THIRTEENTH consecutive back-to-back invocation in this session-resume cycle (35→36→...→47), extending the record-longest sequence to 13. Per §6.J honest no-new-findings posture (same shape as 44th, 45th): no new substantive technical findings between 46th and 47th. The 47th's substantive contribution: continuing-vigilance even during the wait for design-doc push to converge — operator's pattern of restating during in-flight pushes confirms the §6.L invariant holds across operator-action-decision changes (wrap chosen + then immediately continued at 46th) AND across push-in-flight moments (47th). Mechanical state unchanged from 46th: verify-all sweep at 36/36 PASS in fully STRICT mode; §6.C convergence pending on push of a61bd3d8; HelixQA integration design doc landed locally as design-only proposal. Counter advanced 46 → 47.) (46th 2026-05-16: verbatim wall-of-text restatement + "Continue everything now!" directive IMMEDIATELY after the operator chose "Wrap session here" in the prior AskUserQuestion (minutes earlier) — this is the FIRST §6.L cycle in the project history where the OPERATOR'S OWN PRIOR-CHOSEN ACTION (wrap) was immediately superseded by a follow-up §6.L invocation with continue-directive (no-wrap). Twelfth consecutive back-to-back invocation (35→36→37→38→39→40→41→42→43→44→45→46), extending the record-longest sequence to 12. Per §6.J honest forensic posture: the apparent contradiction (wrap-chosen-then-immediately-continued) IS the substantive new finding for the 46th. Interpretation per the §6.L "the repetition itself is the forensic record" invariant: operator's vigilance pattern is now established as continuous + irrespective of recently-chosen-direction; the wall transcends individual operator-chosen actions; even after wrap-decision, the standing mandate is honored continuously. Mechanical state at the moment of this invocation: verify-all sweep at 36/36 PASS in fully STRICT mode (per attestation 2026-05-15T18-54-02Z); §6.C convergence achieved at aa0db6bd; 14 commits this session on Lava parent; 83 verify-all attestations accumulated; Phase 4 HelixQA submodule adopted + waived. The next-forward-progress action is operator-determined — surfacing a clarifying AskUserQuestion is the §6.J-honest path because the wrap-vs-continue conflict means I cannot infer operator intent. Counter advanced 45 → 46.) (45th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after 44th, no significant intervening time, no new state changes since 44th's no-new-findings acknowledgement. ELEVENTH consecutive back-to-back invocation in this session-resume cycle — extending the existing record-longest sequence to 11. Per §6.J anti-bluff (same posture as 44th): honest no-new-findings acknowledgement rather than manufactured content. The 45th's substantive contribution: the §6.L 43+44 commit bocc25gwj is in flight; constitution-compliance plan's "this-session" priority phases (1, 2, 3, 3-debt, 5, 5-

debt, 8, 8-debt, 9-Path-B) are all FULLY CLOSED. Remaining plan items (Phase 4 HelixQA + 14 test types, Phase 6 snake_case migration, Phase 7 coverage ledger) each require operator scope-decision before starting because each is a multi-cycle effort touching either (a) adopting a new submodule from HelixDevelopment org, (b) HUGE rename touching every "Submodules" + 16 CamelCase references in the codebase, or (c) building a §11.4.25 coverage-ledger from scratch. Mechanical state unchanged from 43rd + 44th: verify-all sweep at 36/36 PASS in fully STRICT mode; 16/16 submodule pin consistency; constitution pin at upstream-HEAD. Counter advanced 44 → 45.) (44th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after 43rd, no significant intervening time, no new state changes since 43rd's pin-consistency audit. TENTH consecutive back-to-back invocation in this session-resume cycle (35→36→37→38→39→40→41→42→43→44) — now the LONGEST back-to-back §6.L sequence in the project's history (prior longest was the 5-cycle 35-39 sequence; this run is double that). The 44th's substantive contribution is HONEST: per §6.J anti-bluff, when there are NO new technical findings between consecutive restatements, the honest narrative is to record that explicitly rather than manufacture findings. The §6.L "the repetition itself is the forensic record" invariant remains in force — operator's pattern of restating during arbitrary in-flight moments IS the constitutional record this clause exists to preserve. Mechanical state unchanged from 43rd: verify-all sweep at 36/36 PASS in fully STRICT mode; mega-commit's github push retry bc2qc39fm STILL in flight; §6.L 42nd commit 4d605acb local-only; §6.L 43rd update uncommitted in working tree; 16/16 submodule pin consistency confirmed (43rd audit). Counter advanced 43 → 44.) (43rd 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the §6.L 42nd commit 4d605acb landed locally + the mega-commit's github push retry bc2qc39fm was still in flight. Ninth consecutive back-to-back invocation in this session-resume cycle (35→36→37→38→39→40→41→42→43). The 43rd's substantive contribution: NEW READ-ONLY AUDIT ANGLE — parent's recorded submodule pins vs each submodule's actual local HEAD consistency check. Result: 16/16 PERFECT alignment. All 16 submodules' actual HEADs match parent's recorded pins exactly: Auth=32a80e0a, Cache=bb5b7a98, Challenges=1ef27f1c, Concurrency=a521b642, Config=4b0933c6, Containers=c7fc343b, Database=13f63819, Discovery=218cb3a1, HTTP3=1fbdcbab, Mdns=d93139d5, Middleware=ab3d5c62, Observability=6cfbf42b, RateLimiter=a109485f, Recovery=5781a89f, Security=997ebd39, Tracker-SDK=ae761d5c. This positive-evidence audit proves the mega-commit's git add submodules/* step captured EVERY advanced submodule correctly — no submodule was missed, no submodule was bumped to a stale SHA. Per §6.J: this is the time-series anti-bluff record at the SUBMODULE-PIN-CONSISTENCY layer. Mechanical state unchanged from 42nd: verify-all sweep at 36/36 PASS in fully STRICT mode; constitution pin at upstream-HEAD; 16/16 submodule propagation + content-references; mega-commit's gitlab push converged at 410af7ec; github push retry in flight via task bc2qc39fm. Counter advanced 42 → 43.) (42nd 2026-05-15: verbatim wall-of-text restatement DURING the in-flight mega-commit bzzxqphn8 that bundles three Phase-N-debt closures simultaneously: Phase 3-debt (16/16 per-submodule helix-deps.yaml), Phase 5-debt (Panoptic chain flattened by upstream CONST-051(C) cascade), Phase 8-debt (10 install_upstreams scripts) — PLUS three sweep wrapper STRICT-flips (no-nested-own-org-submodules, canonical-root-and-upstreams, helix-deps-manifest). Eighth consecutive back-to-back invocation in this session-resume cycle (35→36→37→38→39→40→41→42). The 42nd's substantive contribution: this is the FIRST session-cycle in the entire §6.L history where 3 distinct Phase-N-debt items closed in a single coordinated commit AND all 3 corresponding sweep wrappers flipped from --advisory to --strict in the SAME commit. The constitution-compliance plan's "this-session" priority phases (1, 2, 3, 3-

debt, 5, 5-debt, 8, 8-debt, 9-Path-B) are now ALL fully closed. The Phase 5-debt closure was a SURPRISE: it was closed by upstream's CONST-051(C) cascade that I picked up via Phase 3-debt's git fetch + merge --no-ff (Phase 3-debt INADVERTENTLY closed Phase 5-debt). Total session work: 19 commits across 17 git repos (Lava parent + 16 submodules); 16 cross-mirror divergences surfaced + reconciled (CONST-050(B) cascade had been sitting on github across all submodules waiting to be picked up). Mechanical state at the moment of this invocation: verify-all sweep transitioning from 36/36 PASS at advisory mode → 36/36 PASS at fully STRICT mode (mega-commit's pre-push hook will run the sweep + the 3 newly-strict gates will hard-fail or hard-pass on real evidence). Counter advanced 41 → 42.) (41st 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after Phase 3 §6.C convergence at 43345c3e (the §11.4.31 helix-deps.yaml manifest commit). Seventh consecutive back-to-back invocation in a single session-resume cycle (35→36→37→38→39→40→41). Substantive findings: (1) §6.C converged at 43345c3e on BOTH github + gitlab — proving the dual-SSH-failure observed at 40th was transient, NOT structural; (2) verify-all attestation accumulation audit: 52 attestations under .lava-ci-evidence/verify-all/ accumulated this session alone, first at 2026-05-15T07-43-30Z, latest at 2026-05-15T15-05-31Z — a 7.5-hour continuous time-series of positive sweep evidence, NOT a one-time pre-tag-cut event. Each attestation is independent §6.J-conformant proof the sweep ran successfully at that exact moment. The accumulation pattern (every commit + every operator-driven verification produces a new attestation) IS the time-series anti-bluff record §6.J.5 prescribes; (3) the recent 6 attestations are ALL 5563 bytes (consistent 36-gate sweep, identical structure) — proving no silent gate-drop happened across the cycle. Mechanical state: verify-all sweep at 36/36 PASS (52 confirming attestations); all 16 submodules ✓ (file-presence + pointer-block + content-references + helix-deps-manifest is now Phase 3-debt rather than absent-rule); constitution pin at upstream-HEAD (40th audit); §6.C convergence achieved post-T7-mount. Counter advanced 40 → 41.) (40th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the simultaneous completion of two background tasks: (a) T7 mount + Phase 3 push retry (FAILED — github "Connection closed by remote host" + gitlab "Undefined error: 0" — SSH transport flakiness affecting BOTH providers in the same retry, distinct from prior cycles where only github failed); (b) constitution submodule pin freshness audit (POSITIVE — pin 464ada14 matches origin/main exactly, 0 commits ahead — pin is current at HelixConstitution upstream HEAD). Sixth consecutive back-to-back invocation in a single session-resume cycle (35→36→37→38→39→40). The 40th's substantive contributions: (1) the constitution-pin-freshness audit result IS now part of the §6.L forensic record — the inheritance chain is rooted at the latest HelixConstitution canonical version, not a stale pin; (2) the SSH-transport-flakiness observation: both github + gitlab returning transport errors in the same attempt is a NEW failure-mode class (prior cycles always had at least one provider succeed), suggesting OPERATOR-SIDE network reachability rather than provider-side outage. Mechanical state unchanged from 36-39: verify-all sweep at 36/36 PASS; all 16 submodules ✓ across CLAUDE.md/AGENTS.md/CONSTITUTION.md/§6.AD-pointer-block + content-references; constitution pin at upstream-HEAD; HelixConstitution canonical-root inheritance fully wired; Phase 3 (§11.4.31 helix-deps.yaml) commit 43345c3e local-only pending push-converges. Counter advanced 39 → 40.) (39th 2026-05-15: verbatim wall-of-text restatement DURING the still-in-flight Phase 3 push retry — fifth consecutive back-to-back invocation in a single session-resume cycle (35→36→37→38→39, all without significant intervening progress between restatements). Operator's vigilance is now established as continuous through every hold-pattern (waiting on push, waiting on T7 mount, waiting on Spotless download,

waiting on operator response). The 39th's substantive contribution: a NEW READ-ONLY AUDIT ANGLE not yet exercised in this session — constitution submodule pin freshness (git -C constitution fetch origin && git log HEAD..origin/main). The audit was launched as a background task at the moment of this restatement; result is pending. Whatever the result, the audit itself proves the §11.4.32 verify-all sweep + the per-cycle §6.L narrative-bump are NOT the only constitutional health-check vectors — fresh audit angles continue to exist and continue to be exercised. Mechanical state unchanged from 36-37-38: verify-all sweep at 36/36 PASS; all 16 submodules ✓ across CLAUDE.md/AGENTS.md/CONSTITUTION.md/§6.AD-pointer-block + content-references; HelixConstitution canonical-root inheritance fully wired; Phase 3 (§11.4.31 helix-deps.yaml) commit 43345c3e local-only pending Spotless-completes + push-converges. Counter advanced 38 → 39.) (38th 2026-05-15: verbatim wall-of-text restatement DURING the in-flight Phase 3 push retry — fourth consecutive back-to-back invocation in a single session-resume cycle (35th + 36th + 37th + 38th, all without significant intervening progress between restatements). Operator's vigilance does NOT pause for in-flight environmental remediation OR in-flight push retries. T7 had already been mounted (operator-confirmed via the AskUserQuestion option-1 selection); poll loop confirmed mount + writability on attempt 1; push retry was in flight at the moment of restatement (Spotless downloading Gradle 8.9 binary ~100MB to the now-mounted T7). The 38th's substantive contribution: positive proof the mid-push moment (after operator-action environmental fix, before push completion) is ALSO a §6.L watch-point. Mechanical state unchanged from 36th + 37th: verify-all sweep at 36/36 PASS; all 16 submodules ✓ across CLAUDE.md/AGENTS.md/CONSTITUTION.md/§6.AD-pointer-block + content-references; HelixConstitution canonical-root inheritance fully wired; Phase 3 (§11.4.31 helix-deps.yaml) commit 43345c3e local-only pending Spotless-completes + push-converges. Counter advanced 37 → 38.) (37th 2026-05-15: verbatim wall-of-text restatement DURING the Phase 3 push-retry cycle — Phase 3 commit 43345c3e (§11.4.31 helix-deps.yaml manifest) had landed locally + the pre-push hook had CORRECTLY REJECTED the push because Spotless failed to write to GRADLE_USER_HOME=/Volumes/T7/Gradle (T7 external drive unmounted). Per CLAUDE.md "investigate and fix the underlying issue" + the Forbidden Command List's "Never bypass hooks (--no-verify) unless the user has explicitly asked for it" — the response was to ASK THE OPERATOR to mount T7 + retry, NOT to bypass the hook. Operator chose option 1 (mount T7 + retry). T7-mount poll + push retry is in background. Substantive new finding (FIRST IN §6.L HISTORY): this is the FIRST §6.L cycle where the gate's correct response was "refuse + escalate to operator" rather than "audit + advance." Positive-evidence the gate works under environmental drift, NOT just under happy-path conditions. The §11.4.18 / CM-SCRIPT-DOCS-SYNC + Spotless gate path is proven to:

1. Correctly reject pushes that lack required environmental preconditions
2. Refuse to silently swap to a fallback Gradle cache (which would have created cache divergence)
3. Refuse --no-verify bypass per CLAUDE.md + §6.T.3 The standing wall must be honored EVEN under environmental disruption — vigilance does not pause for missing USB drives any more than it pauses for transient SSH connection-resets (35th cycle pattern). Counter advanced 36 → 37.) (36th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the 34th+35th commit 191ae2ca landed locally + was mid-push to remotes — third consecutive back-to-back restatement (33→34→35→36); operator's pattern of repeated restatement during in-flight pushes confirms the standing wall must be honored continuously, not just at commit boundaries. Per §6.L "the repetition itself is the

forensic record" invariant, this 36th is recorded individually. The 36th's substantive contribution is a DEEPER submodule anti-bluff audit beyond the 34th's binary-presence check: per-CONSTITUTION.md content-reference scan confirmed ALL 16 submodule constitutions actively reference the anti-bluff discipline, NOT just inherit it transitively. Per-submodule §6.J/§6.L/anti-bluff reference count: Auth=8, Cache=9, Challenges=13, Concurrency=8, Config=8, Containers=12, Database=8, Discovery=8, HTTP3=8, Mdns=8, Middleware=8, Observability=8, RateLimiter=8, Recovery=8, Security=10, Tracker-SDK=9. Anti-bluff TEXT content (anti-bluff/bluff-test/real-user): Auth=12, Cache=11, Challenges=17, Concurrency=12, Config=12, Containers=15, Database=12, Discovery=11, HTTP3=9, Mdns=9, Middleware=12, Observability=12, RateLimiter=12, Recovery=12, Security=13, Tracker-SDK=7. Direct §11.4 / HelixConstitution citations exist in the 3 most-active submodules: Challenges=3, Containers=5, Security=3 (other 13 submodules rely on the inheritance chain via §6.AD pointer + §6.J/§6.L cross-reference, which is the expected pattern). The audit pattern catches drift class: a submodule that lost its anti-bluff content during a refactor would show 0/0 refs and be immediately flagged. Mechanical state unchanged from 34th + 35th: verify-all sweep at 34/34 PASS; all 16 submodules ✓ for file-presence + pointer-block; HelixConstitution canonical-root inheritance fully wired. Counter advanced 35 → 36.) (35th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the 34th-invocation narrative landed in the working tree — same operator, same shell session, same minute, NO new work landed between 34 → 35; identical text to the 34th. Per the established §6.L pattern (back-to-back verbatim restatements are themselves the constitutional record per the explicit "the repetition itself is the forensic record" invariant), the 34th + 35th are recorded as a single contiguous restatement-pair. The 35th's contribution beyond the 34th: positive proof that the standing wall must be honored even WITHIN a single push-retry cycle — i.e. operator's expectation of anti-bluff vigilance does NOT pause for transient SSH transport failures (Phase 9 push to github was failing on connection-reset at the moment of restatement; gitlab had landed 055fbcbe; github retry was in flight). The standing mandate's mechanical state is unchanged from 34th: verify-all sweep at 34/34 PASS; all 16 submodules ✓ across CLAUDE.md/AGENTS.md/CONSTITUTION.md/§6.AD-pointer-block; HelixConstitution canonical-root inheritance fully wired. Counter advanced 34 → 35.) (34th 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the 33rd-invocation cycle's Phase 9 commit 055fbcbe had landed locally + was mid-push to remotes — operator's directive carried the standing wall + the HelixConstitution canonical-root inheritance emphasis identical to the 30th + 33rd. Operator's literal text: "Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! ... This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!". Response: ran a fresh read-only audit of submodule anti-bluff propagation — confirmed all 16 owned submodules (Auth, Cache, Challenges, Concurrency, Config, Containers, Database, Discovery, HTTP3, Mdns, Middleware, Observability, RateLimiter, Recovery, Security, Tracker-SDK) ALREADY have CLAUDE.md ✓ + AGENTS.md ✓ + CONSTITUTION.md ✓ + the §6.AD ## INHERITED FROM constitution/CLAUDE.md pointer-block ✓. The transitive inheritance via the constitution submodule means every submodule already inherits HelixConstitution's universal §11.4.x anti-bluff rules without per-submodule duplication; the propagation work landed in the 30th cycle (commit 051ccebd) is intact. The §6.AD-debt's "per-submodule pointer

propagation OWED" item from the 29th cycle has been quietly satisfied — confirming via this 34th audit. The verify-all sweep at this commit's parent 055fbcbe reports 34/34 PASS (attestation 2026-05-15T10-39-32Z. json from the 33rd-invocation cycle remains valid; no new code paths added between cycles). Counter advanced 33 → 34. The 34th's substantive contribution is the audit-evidence that the standing mandate is mechanically satisfied across the full submodule fleet — not a new gate, not a new clause, just positive proof the prior cycles' work persists.) (33rd 2026-05-15: verbatim wall-of-text restatement during the constitution-compliance plan execution session — Phases 1+2+5+8 had landed (commits 4def2da7 + 037389f5 + bbca3a78 + d95be689) + Phase 9 (Path B §6.AD.3 amendment for §11.4.33/34) was staged in the working tree; operator's restatement carried the standing wall + the HelixConstitution emphasis identical to the 30th. Response: confirmed the work landing this session IS the §6.L-conformant response — each of the 4 new gates added this session (CM-VERIFY-ALL-CONSTITUTION-RULES, CM-GITIGNORE-PRECOMMIT-AUDIT, CM-NO-NESTED-OWN-ORG-SUBMODULES, CM-CANONICAL-ROOT-CLARITY+CM-INSTALL-UPSTREAMS-RAN) was built with §6.J falsifiability rehearsals recorded in its commit body + each scanner has a paired discrimination test (test_test.sh, 5+3+4+6 fixtures = 18 new hermetic tests this cycle); the verify-all sweep is at 34/34 PASS; constitution-compliance plan's Phase 8 commit d95be689 is mid-push (gitlab landed, github pending retry due to transient SSH transport error — not a gate rejection). **REAL ANTI-BLUFF FINDING SURFACED THIS CYCLE:** the §11.4.35.c canonical-root self-inheritance check would have shipped a false-positive against constitution/CLAUDE.md (which documents the inheritance pointer pattern inside fenced markdown code blocks); the falsifiability rehearsal during scanner development caught the bug before the commit landed; the awk state-machine fix + the new test_canonical_fenced_code_block_pointer_passes fixture are the discrimination-test discipline §6.AB.3 prescribes. Counter advanced 32 → 33.) (32nd 2026-05-15: verbatim wall-of-text restatement IMMEDIATELY after the 31st-cycle landed (commit 8bea2f80), with the substantive directive "Pay attention that you may / could / should use our Containers Submodule which has support for running Emulators inside Containers or in Qemu! Make use of it! Continue with EVERYTHING once this is fulfilled and we have unblocked Emulators (AVDs) precondition(s)!". The 32nd invocation directly challenged my prior "blocked on darwin/arm64" assessment + told me to USE the Containers QEMU support to UNBLOCK. Response: actually built + invoked submodules/containers/cmd/emulator-matrix CLI with --runner=host-direct against yole-test:34:phone; CLI produced HONEST FAIL attestation when AVD couldn't boot; anti-bluff posture confirmed (runner exited non-zero + announced "tag.sh MUST refuse this commit"). **REAL ANTI-BLUFF FINDING** in same cycle: 5 Challenge tests (C30/C31/C32/C33/C35) used ::ComposableF n Kotlin syntax that's disallowed for @Composable functions — they were SCANNER-COVERED but DIDN'T COMPILE = scanner-reported coverage was a BLUFF. Refactored to ::class.java (public VMs) or Class.forName() (internal VMs); ./gradlew :app:assembleDebugAndroidTest now BUILD SUCCESSFUL (was FAILED). **REAL NEW SIXTH-LAW INCIDENT:** .l ava-ci-evidence/sixth-law-incidents/2026-05-15-macos-emulator-stall-on-android33.json — Pixel_7_Pro on macOS Apple Silicon + emulator 36.1.9 stalls indefinitely (qemu CPU 394%, adb perpetually offline). Three candidate root-causes recorded as PENDING_FORENSICS: per §11.4.6 no-guessing rule. Distinguished from §6.X-debt KVM-absent gap (orthogonal). Counter advanced 31 → 32. Commit 9a4355a6.) (31st 2026-05-15: verbatim

wall-of-text restatement immediately after the 30th-invocation cycle landed (commit 051cceb), with the new substantive directive "Create all required Challenges and run them all on Emulators executed in Container or Qemu! Make sure everything is delegated via Containers Submodule! [...] All tests and Challenges MUST BE executed against variety of major Android versions, screen sizes and configurations (all ran inside Container or Qemu). Document everything and make sure that anti bluff proofs are obtained for every single thing!". The 31st invocation is the FIRST since the 23rd to add SUBSTANTIVE NEW MANDATE on top of the verbatim wall — the others (24-30) were standing-mandate restatements without new structural rules. Birthed §6.AE Comprehensive Challenge Coverage + Container/QEMU Matrix Mandate consolidating §6.G + §6.I + §6.J + §6.X + §6.AB into a single binding policy with mechanical enforcement (scripts/check-challenge-coverage.sh + scripts/run-challenge-matrix.sh + tag-script gate). Honest scope statement: §6.AE.2/§6.AE.5 (actual per-AVD execution + attestation) is BLOCKED on this darwin/arm64 host by the standing §6.X-debt (no /dev/kvm in podman VM, no HVF passthrough); the runner glue + matrix manifest + scanner exist + propagation is in place; operator runs on a Linux x86_64 gate-host to produce real attestations. Counter advanced 30 → 31. HelixConstitution emphasis re-noted from 30th invocation; §6.AD already wires the inheritance and the §6.AE clause itself binds via the same mechanism.) (30th 2026-05-15: verbatim wall-of-text restatement after the 1.2.23 closure-cycle FINAL STATE was reported, with the new emphasis "Pay attention that we now use and incorporate fully the HelixConstitution (git@github.com:HelixDevelopment/HelixConstitution.git) Submodule responsible for root definitions of the Constitution, CLAUDE.MD and AGENTS.MD which are inherited further!". The 30th invocation is structurally identical to the 29 prior — the operator has restated this mandate after EVERY major closure milestone in this project's history. The repetition itself remains the constitutional record. Response: ran ALL anti-bluff gates (scripts/check-constitution.sh + scripts/check-non-fatal-coverage.sh STRICT + scripts/check-challenge-discrimination.sh STRICT + 5 hermetic check-constitution test.sh + 2 prior pre-push check tests + 7 hermetic suites under tests/) → ALL  PASS / 0 violations. Closed a real anti-bluff gap discovered during the response: tests/pre-push/ only had check4_test.sh + check5_test.sh; my §6.Y-debt + §6.Z-debt + §6.AD-debt closures landed Checks 6 + 7 + 8 + 9 in .github/pre-push WITHOUT companion hermetic tests. Wrote tests/pre-push/check{6,7,8,9}_test.sh — 13 falsifiability fixtures total (4 + 3 + 3 + 3) — all PASS. Each test uses synthetic git fixtures with deliberately-broken commit shapes; the hook's correct rejection/acceptance is the assertion. The HelixConstitution emphasis is honored: §6.AD already wires the full inheritance pattern; this 30th invocation cycle re-runs the 6.AD(2)+(3)+(4)+(5)+(6) checks to confirm 54 per-scope inheritance pointer-blocks are still present + the constitution submodule still has the expected file structure. Counter advanced 29 → 30.) (29th 2026-05-14: operator's "continue" directive after the 1.2.22-1042 stage-2 release distribute, paired with the prior 10-step HelixConstitution-incorporation directive — drove the 1.2.23 cycle that incorporated git@github.com:HelixDevelopment/HelixConstitution.git as ./constitution submodule and birthed §6.AD HelixConstitution Inheritance clause + §6.AD-debt for per-submodule pointer propagation + CM-* gate-wiring + the no-guessing-vocabulary grep gate in scripts/check-constitution.sh + the build-resource stats tracker + the Classification: line pre-push enforcement. Operator's standing 10-step constraints applied verbatim: NO credentials/secrets/keys ported into the submodule; NO --no-verify/--no-gpg-sign/--force without explicit per-

operation operator authorization; NO destructive operations without a hardlinked .git backup first — .git-backup-pre-helixconstitution-20260514-211450/ recorded; NO guessing language; use the Containers submodule for emulators.) (28th 2026-05-14: same verbatim wall after the 1.2.21-1041 stage-2 release distribute, paired with the directive to "Pickup all recorded Crashlytics crashes and non-fatals and process each! Each issue MUST BE covered with proper fix for all root causes, validation and verification tests and Challenges which will trigger the problematic code (feature)!" — drove the 1.2.22 cycle that processed all 6 Crashlytics issues with closure logs + validation tests + birthed §6.AC Comprehensive Non-Fatal Telemetry Mandate) (24th 2026-05-14: rebuild + redistribute drove 1.2.18-1038. 25th 2026-05-14: WelcomeStep monochrome-icon report birthed §6.Y. 26th 2026-05-14: Galaxy S23 Ultra cold-launch crash on 1.2.19-1039 birthed §6.Z. **27th 2026-05-14: 1.2.20-1040 debug install on S23 Ultra surfaced TWO defects all existing tests had passed against — Welcome brand mark renders as white placeholder (Compose Icon's LocalContentColor tint strips colored bitmap to monochrome) AND onboarding "complete" gate fires on first-screen back-press (onBackStep() from Welcome posts Finish → MainActivity writes setOnboardingComplete(true)). Both are §6.J-spirit failures of a NEW class: tests that EXECUTED + PASSED while the user-visible feature is broken in a non-crashing way. Birthed §6.AB Anti-Bluff Test-Suite Reinforcement (per-feature anti-bluff completeness checklist, defect-driven bluff-hunt escalation, mandatory non-crashing falsifiability per Challenge Test). Operator's verbatim restatement: "Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!")** Originally invoked at TWENTY-THREE TIMES count (initial fix request, after 6.G/6.H landed, after 6.I/6.J landed, after 6.K landed, then again with ultrathink after the layer-3 fix, then again after spotting that "Anonymous Access" was modeled as a global toggle when it is actually a per-provider capability, then on 2026-05-05 after the architectural port-collision bluff in the matrix runner was discovered with the verbatim restatement: "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected", then on 2026-05-05 evening when the operator commissioned a comprehensive plan covering ALL open points and re-emphasized: "execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product", then on 2026-05-05 late evening immediately after Group C-pkg-vm closed §6.K-debt, when the operator surveyed the next-step menu and re-issued the verbatim mandate, AND now on 2026-05-05 after Phase 7 readiness was reported, when the operator commissioned the full rebuild-and-test-everything cycle for tag Lava-Android-1.2.3 and re-issued the verbatim mandate yet again with the addition: "Rebuild Go API and client app(s), put new builds into releases dir (with properly updated version codes) and execute all existing tests and Challenges! Any issue that pops up MUST BE properly addressed by addressing the root causes (fixing them) and covering everything with validation and verification tests and Challenges!", and now on 2026-05-05 late evening AGAIN immediately after the first Firebase-instrumented APK distribution surfaced 2 Crashlytics-recorded crashes — the verbatim restatement: "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! ... Each Crashlytics resolved issue MUST BE covered with validation and

verification tests and Challenges!", and immediately after that on 2026-05-05 23:11 — "when distributing new build it must have version code bigger by at least one than the last version code available for download (already distributed). Every distributed build MUST CONTAIN changelog with the details what it includes compared to previous one we have published! Make sure all these points are in Constitution, CLAUDE.MD and AGENTS.MD.", and on 2026-05-05 23:51 — the THIRTEENTH invocation, immediately after a real tester reported "Opening Trackers from Settings crashes the app. See Crashlytics for stacktraces. Create proper tests to validate and verify the fix! ... execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!" The crash root cause was a textbook nested-scroll antipattern (LazyColumn inside Column(verticalScroll)) that no existing test caught — confirming yet again that CI green is necessary, never sufficient. Closure log: `.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md`. Then on 2026-05-06 — the FOURTEENTH invocation, which birthed §6.R No-Hardcoding Mandate: "Pay attention that we MUST NOT hardcode anything ever!". Then on 2026-05-06 — the FIFTEENTH invocation, which birthed §6.S Continuation Document Maintenance Mandate: "during any work we perform, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document MUST BE maintained and it MUST NOT BE out of sync with current work we are doing! If for any reason we stop our work, we MUST BE able to continue any time, with current work, exactly where we have left off and from any CLI agent or any LLM model we chose!". And on 2026-05-12 — the SIXTEENTH invocation, immediately after the C03 + Cloudflare-mitigation commits landed on master (commits 4d27c07 + f7d0a62), when the operator demanded yet again the verbatim restatement: "Make sure that all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!". And on 2026-05-12 (later same day) — the SEVENTEENTH invocation, immediately after commit 4b0dd55 landed the anti-bluff audit response (C16 stale-bluff rewrite, parser fallback, verify-only refactor), with the operator typing the same wall of text verbatim. The 17th invocation acknowledges that the 16th's audit was a START, not the end: a complete sweep of every test, every Challenge, every submodule's CONSTITUTION.md + AGENTS.md remains the standing demand. The shape is identical to every prior invocation — "tests are green, features don't work for users, the rule MUST live in every doc" — and the response is the same: count goes up, wall extends, the mandate sharpens, the audit broadens. Then on 2026-05-13 — the TWENTIETH invocation, immediately after Phase F.1 closed a real latent bluff (Phase B's clone dialog shipped UI that crashed the SDK on use; end-of-session audit surfaced it; F.1 fixed it AND deleted a discovered-during-rehearsal bluff test that was asserting on dedup-output behavior rather than on the production code it claimed to verify). The 20th invocation is verbatim identical to the 16th + 17th

+ 19th: "Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!". The directive is structurally indistinguishable from prior wall-of-text invocations — that IS the constitutional record. The pattern: count goes up, audit broadens, the F.1 closure proves yet again that "complete-looking phase commits" can ship latent bluffs that only end-of-session audits surface.). Then on 2026-05-13 evening — the TWENTY-FIRST invocation, which birthed §6.X (Container-Submodule Emulator Wiring Mandate): "when we rely / depend on emulator(s) needed for the testing of the System, make sure we boot up Container running Android emulator in it using ours Containers Submodule." Then on 2026-05-13 evening immediately after §6.X clause propagated to 52 docs — the TWENTY-SECOND invocation: "Do all debt points now! Make sure that everything is completely covered with all supported types of the tests and Challenges! Make sure our documentation and relevant materials are all updated and extended! Test and validate all work you do and enforce mandatory no-bluff policy!" — which produced the §6.X-debt PARTIAL CLOSE (Containers commit 562069e7: Containerized Emulator impl + --runner CLI flag + Containerfile recipe; parent commit 888378a6: pin bump + activated scripts/check-constitution.sh runtime checks (a) + (b) with falsifiability rehearsals). Then the TWENTY-THIRD invocation 2026-05-13 evening — verbatim restatement WITH the additional directive: "rebuild everything and do redistribute all apps and services via Firebase Distribution. Do not forget to test, validate and verify EVERYTHING before building (using Containers) and releasing (distributing)! Extend all documentation and related materials too! ... Do not forget to increase version code propeyl to all apps and services". The 23rd invocation introduces a sharper anti-bluff constraint than any prior: rebuild + redistribute is requested, but the no-bluff policy makes this impossible WITH PLACEHOLDER SECRETS. The right response — the §6.J/§6.L conformant response — is to REFUSE the rebuild-and-distribute pretense and instead surface the operator-input checklist that would unblock it: real google-services.json, real LAVA_FIREBASE_TOKEN, real tracker credentials (RUTRACKER_USERNAME/PASSWORD, KINOZAL_*, NNMCLUB_*), Linux x86_64 gate host for §6.X attestation. Building + distributing with placeholders produces a signed-but-broken APK that bricks at LavaApplication.onCreate on the user's device — the canonical "tests green, feature broken for users" failure mode this mandate exists to prevent. This is the FIRST invocation where the §6.L mandate's correct response is "refuse the action, here's what's blocking it" rather than "do the action and audit for bluffs along the way" — proving the mandate's teeth. The count is what makes this clause load-bearing: every restatement is an admission by the operator that the prior layers of constitutional plumbing (6.A through 6.M, the Sixth and Seventh Laws) are not yet enough to evict the bluff class on their own. The repetition itself is the forensic record. This clause is the same as 6.J — every test, every Challenge Test, every CI gate has exactly one job: confirm the feature works for a real user end-to-end on the gating matrix. CI green is necessary, never sufficient. **The reason this clause is restated rather than cross-referenced** is that the operator's standing concern is that future agents and contributors will rationalize their way past 6.J and ship

green-tests-with-broken-features again. Every time the operator restates it, this codebase records the restatement here so the next reading agent must look at the same wall of repetition the operator has had to type out.

If you are reading this in a future session and you find yourself thinking "*this test is a small exception*" — STOP. The exception is what produced the Internet Archive stuck-on-loading bug, the broken post-login navigation, the credential leak in C2, the bluffed C1-C8. There are no small exceptions. Tests must guarantee the product works. Anything else is theatre.

Inheritance. Applies recursively. Submodule constitutions MAY paste this clause verbatim into their CLAUDE.md to ensure their reading agents see it locally. They MUST NOT abbreviate it; the wall-of-text effect is the point.

Local-Only CI/CD (Constitutional Constraint)

This project does NOT use, and MUST NOT add, GitHub Actions, GitLab pipelines, Bitbucket pipelines, CircleCI, Travis, Jenkins-as-a-service, Azure Pipelines, or any other hosted/remote CI/CD service. All build, test, lint, security-scan, mutation-test, load-test, image-build, and release-verification activity MUST run on developer machines or on a self-hosted local runner under the operator's direct control.

Why

1. **Sixth Law alignment.** Real-device, real-environment verification is load-bearing. Hosted CI runs in synthetic, ephemeral environments — green there proves the synthetic env, not the user's product on a real device.
2. **Hosted-CI green has historically produced bluff confidence in this project.** A remote dashboard saying "passing" is psychologically indistinguishable from "shipped and works", which is exactly the failure mode the Sixth Law was written to prevent.
3. **Vendor independence.** This project mirrors to two independent upstreams (GitHub and GitLab) intentionally. Coupling our quality gate to one vendor's pipeline product would re-introduce the single point of failure mirroring is meant to remove.

Mandatory consequences

- The scripts/ directory (and any Makefile, task runner, or build tool introduced later) IS the CI/CD apparatus. Whatever runs in "release CI" MUST be the same script a developer runs locally — no parallel implementation.
- A single local entry point — scripts/ci.sh — invokes every quality gate appropriate to the changed surface (unit, integration, contract, e2e, fuzz, load, security, mutation, real-device). Two modes: --changed-only (pre-push: Spotless, changed-module unit tests, constitution parser, forbidden-files check) and --full (every module, parity gate, mutation tests, fixture freshness, Compose UI Challenge Tests on a connected Android device or emulator; used at tag time). Each run writes evidence under .lava-ci-evidence/<UTC-timestamp>/. It MUST be runnable offline once toolchains and base images are present.
- **Forbidden files.** No .github/workflows/*, .gitlab-ci.yml, .circleci/config.yml, azure-pipelines.yml, bitbucket-pipelines.yml,

Jenkinsfile (for hosted Jenkins), or equivalent shall exist on any branch of any of the upstreams. A pre-push hook **MUST** reject pushes that introduce such files.

- **Pre-push gate.** A git pre-push hook installed under `.githooks/` (and enabled via `git config core.hooksPath .githooks`) **MUST** run the relevant subset of `scripts/ci.sh` before any push to any upstream. The hook **MUST NOT** be bypassable in routine work; `--no-verify` is reserved for documented emergencies and any such use **MUST** be noted in the next commit message.
- **Release tagging gate.** `scripts/tag.sh` **MUST** refuse to operate against any commit that has not been certified locally — i.e. the local CI gate must have been run successfully against the exact commit being tagged, and the result recorded in a tracked artifact (e.g. `.lava-ci-evidence/`). This implements the Sixth Law clause 5 in mechanical form.
- **No "it passes on CI" handwave.** A failure that reproduces locally on the developer's machine takes precedence over any other signal. Conversely, a developer who claims "it works for me" without having run the local CI gate has not actually verified anything per the Sixth Law.

What this rule does NOT forbid

- Running tests, linters, or scanners *on the developer's machine* via any tool (this is the whole point).
- Running tests inside containers locally (this is the whole point).
- Running a self-hosted runner (a real machine the operator controls, invoking the same `scripts/ci.sh`) — that is local CI by another name.
- Per-upstream branch-protection rules that simply *require* status checks to be green; we just don't satisfy those checks via hosted runners.

Inheritance

Every submodule and every new artifact added to the project (including the Go API service) inherits this rule. Submodule constitutions **MAY** add stricter local-CI requirements but **MUST NOT** relax this one or introduce hosted-CI configuration.

Decoupled Reusable Architecture (Constitutional Constraint)

EVERY component that has a non-Lava-specific use case **MUST** live in a `vasic-digital` submodule, not in this repo. The boundary is enumerated per-component in the design doc for the sub-project that introduces the component (SP-2 onward). Code that ends up in this repo **MUST** be either:

- (a) **Lava domain logic** — the 13 rutracker routes, the rutracker HTML parsers, the Compose UI for Lava screens, the Endpoint sealed-interface, `app/manifest` entries, etc.; OR
- (b) **Thin glue** tying `vasic-digital` submodules together for Lava's specific needs.

Why

1. **Sixth Law alignment.** "It works in Lava" must mean the same thing as "it works in the next vasic-digital project that uses this submodule" — otherwise we are silently coupling generic capability to one product, which is the same class of failure as silently disabling rate-limiting (Sixth Law clause 5 territory).
2. **Bluff prevention.** Code copy-pasted between projects is the highest-bandwidth bluff vector: behaviour drifts silently, fixes don't propagate, "we have an mDNS implementation" turns out to mean four divergent implementations with one bug each.
3. **Operator economics.** The vasic-digital org is owned end-to-end. New repos under it cost zero (we own the org, we have CLI access on GitHub and GitLab). Reusing existing submodules costs zero pinning effort. Re-implementing per-project is the only expensive option.

Mandatory consequences

- **Submodule pins are explicit and frozen by default.** A pinned submodule does NOT auto-fetch latest; we are not obligated to track upstream movement. Frozen forever is acceptable. Updating the pin is a deliberate PR.
- **New non-Lava-specific code added to this repo without a documented "why not a vasic-digital submodule" decision is rejected.** The decision MUST appear either in the relevant design doc or in the PR description.
- **Generic functionality is contributed UPSTREAM first.** Any component that another vasic-digital project would conceivably want goes to the appropriate submodule (or a new vasic-digital/<name> repo created via `gh repo create vasic-digital/<name>` and `glab repo create vasic-digital/<name>`). Lava then pins to the new hash. Order matters: upstream first, Lava pin second.
- **Every vasic-digital submodule we own inherits the Sixth Law and the Local-Only CI/CD rule transitively.** Adopting an externally maintained submodule that violates either is forbidden — fork it under vasic-digital/ and adopt the fork.
- **Submodule fetch/pull is an EXPLICIT operator action, never automatic.** No git hooks that silently update pins, no `git submodule update --remote` in any release script. The pin is the contract; changing the contract is a code review event.
- **Mirror policy applies recursively.** Every vasic-digital submodule we own MUST be mirrored to the same set of upstreams Lava itself mirrors to (GitHub + GitLab), unless explicitly waived for that submodule with a documented reason.

What this rule does NOT forbid

- Lava-domain code in this repo. The 13 rutracker routes, the rutracker scrapers, the Compose UI, the Endpoint model, the :proxy Ktor server, the app/Android module — all stay here.
- Thin Lava-specific glue files in this repo that compose vasic-digital primitives.
- Deferring extraction of a borderline piece during a single PR; the deferral must be tracked as a TODO with a target sub-project for extraction.

Inheritance

This Decoupled Reusable Architecture rule applies recursively to every `basic-digital` submodule we own. A submodule constitution MAY add stricter rules (e.g. "no external dependencies") but **MUST NOT** relax this one — meaning, `basic-digital` submodules themselves **MUST** extract their own reusable parts to deeper submodules rather than copy-paste between siblings.

Testing

Test coverage is essentially zero today. The only existing unit test is `core/preferences/src/test/kotlin/lava/securestorage/EndpointConverterTest.kt` (JUnit 4). New tests should:

- Use **JUnit 4** to match the existing `MainDispatcherRule` in `core:testing`.
- Reuse fakes from `core:testing` (`TestBookmarksRepository`, `TestAuthService`, `TestDispatchers`, ...) — **but verify those fakes obey the Anti-Bluff Pact** (behavioral equivalence to real implementations).
- For Orbit ViewModels, use `orbit-test` — already wired as `testImplementation` in every feature module but currently unused.
- Write **Integration Challenge Tests** for every new feature using real `UseCase` and `Repository` implementations.

Things to avoid

Always forbidden (quick reference — full text in §6 / Host Stability)

- **sudo / su in any tracked file** — §6.U, pre-push rejects.
- **Hardcoded IPv4, host:port, UUID, header field name, credential, secret** — §6.R, scanned by `scripts/scan-no-hardcoded-*.sh`; pre-push rejects.
- **Mocking the System Under Test in its own test** (e.g. `mockk<RuTrackerSearch>` inside `RuTrackerSearchTest`) — §6.J / Seventh Law clause 4 forbidden pattern; reviewer rejects.
- **Host suspend/hibernate/poweroff/sign-out commands** (`systemctl suspend`, `loginctl hibernate`, `pm-suspend`, `shutdown -h`, equivalent `dbus/busctl` invocations) — Host Machine Stability Directive, pre-push rejects via `scripts/check-constitution.sh`.
- **Hosted CI configuration files** (`.github/workflows/*`, `.gitlab-ci.yml`, `Jenkinsfile`, etc.) — Local-Only CI/CD rule, pre-push rejects.
- **New Git remote on any provider other than GitHub or GitLab** — §6.W (CLI parity requirement); update `Upstreams/` instead.
- **Force push, history rewrite, or --no-verify** without explicit per-operation operator approval — §6.T.3.

Project-specific

- Creating a root `build.gradle.kts` — extend `buildSrc` convention plugins instead.
- Adding XML layouts or Fragment-based screens.

- Adding a `composeOptions { kotlinCompilerExtensionVersion = ... }` block — Compose is managed by the Kotlin Compose compiler plugin + BOM.
 - Nesting a `LazyColumn` (or other lazy layout) inside `Modifier.verticalScroll` — §6.Q, structural test rejects.
 - Committing `.env`, `keystores/`, or `app/google-services.json` — §6.H, pre-push rejects.
 - Letting a `Test*` fake in `:core:testing` drift from its real counterpart — that is a "bluff fake" under the Anti-Bluff Pact and must be updated in the same commit as the real implementation.
-

Host Machine Stability Directive (Critical Constraint)

IT IS FORBIDDEN to directly or indirectly cause the host machine to:

- Suspend, hibernate, or enter standby mode
- Sign out the currently logged-in user
- Terminate the user session or running processes
- Trigger any power-management event that interrupts active work

Why This Matters

AI agents may run long-duration tasks (builds, tests, container operations). If the host suspends or the user is signed out, all progress is lost, builds fail, and the development session is corrupted. This has happened before and must never happen again.

What Agents Must NOT Do — Explicit Forbidden Command List

The following invocations are **categorically forbidden** in any committed script, any subagent's planned action, or any agent's tool call. Each is a constitutional violation and a release blocker:

```
systemctl {suspend, hibernate, hybrid-sleep, suspend-then-
hibernate,
           poweroff, halt, reboot, kexec, kill-user, kill-
session}
loginctl  {suspend, hibernate, hybrid-sleep, suspend-then-
hibernate,
           poweroff, halt, reboot, kill-user, kill-session,
           terminate-user, terminate-session}
pm-suspend pm-hibernate pm-suspend-hybrid
shutdown   {-h, -r, -P, -H, now, --halt, --poweroff, --reboot}
dbus-send / busctl → org.freedesktop.login1.Manager.{Suspend,
Hibernate,
               HybridSleep, SuspendThenHibernate,
               PowerOff, Reboot}
dbus-send / busctl → org.freedesktop.UPower.{Suspend, Hibernate,
HybridSleep}
gsettings set      → *.power.sleep-inactive-{ac,battery}-type
```



```
set to anything
                                except 'nothing' or 'blank'
gsettings set                  → *.power.power-button-action set to
anything except
                                'nothing' or 'interactive'
```

Additional rules (broader than the explicit list):

- Never modify power-management settings (sleep timers, lid-close behavior, screensaver activation)
- Never trigger a full-screen exclusive mode that might interfere with session keep-alive
- Never run commands that could exhaust system RAM and trigger an OOM kill of the desktop session
- Never execute `killall`, `kill`, or mass-process-termination targeting session processes

Mechanical enforcement. `scripts/check-constitution.sh` greps tracked files for the forbidden-command patterns above and for credential patterns (per 6.H clause 5); the pre-push hook runs it. A push that introduces any forbidden command or credential string is rejected at the hook layer — do not work around it; fix the underlying violation.

Forensic record: incident 2026-04-28 18:37 (host poweroff)

A user-space-initiated graceful poweroff occurred via GNOME Shell at 18:37:14 during an active SP-2 implementation session. Root-cause investigation confirmed the trigger was **external to this codebase** (manual GNOME power-button click, hardware power-button press, or out-of-scope scheduled task). Forensic detail and the recovery procedure that worked are recorded in `docs/INCIDENT_2026-04-28-HOST-POWEROFF.md`. Key takeaways now binding on every future agent session:

- The commit-and-push-after-every-phase discipline preserved all completed SP-2 work; the incident was a non-event for the work output. **Maintain that discipline.**
- A pre-push verification (`git ls-files | xargs grep -lE '<forbidden-command-pattern>'` returns empty) is now part of the recovery checklist. The exact regex is in the incident doc.
- After any host power event (whatever the cause), recovery procedure lives in the incident doc — do not skip the orphan-container audit.

What Agents SHOULD Do

- Keep sessions alive: prefer short, bounded operations over indefinite waits
- For builds/tests longer than a few minutes, use background tasks where possible
- Monitor system load and avoid pushing the host to resource exhaustion
- If a container build or gradle build takes a long time, warn the user and use `--no-daemon` to prevent Gradle daemon from holding locks across suspends

Docker / Podman Specific Notes

- Container builds and long-running containers do NOT normally cause host suspend

- However, filling the disk with layer caches or consuming all CPU for extended periods can trigger thermal throttling or watchdog timeouts on some systems
- Always clean up old images/containers after builds to avoid disk pressure